

Partie 17 — Créer ses propres Hooks

FORMATION REACT / TYPESCRIPT

GAMEHUB PROJECT

Partie 17

Custom Hooks — encapsuler et réutiliser la logique React

Partie 18

Gestion robuste des erreurs — loading, error, empty states

17.1 — Pourquoi créer un Hook personnalisé ?

Objectif de cette section

Comprendre la motivation derrière la création de Hooks personnalisés dans une application React/TypeScript comme GameHub.

Questions clés

- Qu'est-ce qu'un Hook React ?
- Pourquoi ne pas se contenter des Hooks natifs ?
- Quand la duplication de logique devient-elle un problème ?
- Comment un custom Hook résout-il ces problèmes ?

Qu'est-ce qu'un Hook ?

Un Hook est une fonction fournie par React qui permet d'utiliser certaines fonctionnalités de React dans un composant fonctionnel.

Hooks natifs — exemples

`useState()`

`useEffect()`

`useContext()`

`useReducer()`

Ce que les Hooks permettent de gérer

- L'état du composant
- Les effets de bord
- Le contexte partagé
- Certaines logiques liées au cycle de vie d'un composant

Exemple : useState

Nous avons déjà utilisé un Hook fondamental :

```
const [count, setCount] = useState(0);
```

Ce que useState permet au composant de faire

- Créer un état
- Lire sa valeur
- Modifier cette valeur
- Provoquer un nouveau rendu lorsque l'état change

Rôle du Hook

Le Hook est une **interface** entre notre composant et une fonctionnalité de React. Il abstrait la mécanique interne et expose une API simple et prévisible au développeur.

Pourquoi créer nos propres Hooks ?

React fournit des Hooks génériques. Mais notre application peut avoir des besoins spécifiques.

Exemple GameHub

```
const { games, loading, error } = useGames();
```

`useGames` n'est pas fourni par React. C'est un Hook que nous pouvons créer pour encapsuler la logique liée aux jeux.

Principe

Un custom Hook permet de donner un **nom métier** à une logique applicative, de la centraliser, et de l'exposer sous une interface claire à tous les composants qui en ont besoin.

Le problème de la duplication

Supposons que plusieurs composants doivent charger des données. On pourrait recopier dans chacun d'eux :

```
const [data, setData] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  // chargement des données
}, []);
```

Beaucoup de code répété

La même logique est dupliquée dans chaque composant concerné.

Corrections multiples

Une correction doit être appliquée à plusieurs endroits simultanément.

Comportements divergents

Risque de comportements différents entre composants similaires.

Lisibilité dégradée

Les composants deviennent plus difficiles à lire et à maintenir.

Principe d'un custom Hook

On peut déplacer cette logique dans une fonction dédiée, puis l'utiliser dans un composant.

Définition du Hook

```
function useSomething() {  
  // logique React  
  
  return {  
    // données utiles  
  };  
}
```

Utilisation dans un composant

```
function GamesPage() {  
  const data = useSomething();  
  
  return <div>...</div>;  
}
```

📌 Le composant utilise la logique sans avoir besoin de connaître tous ses détails. C'est le principe d'encapsulation appliqué à la logique React.

Custom Hook : définition

Un custom Hook est une fonction JavaScript/TypeScript créée par le développeur.

Caractéristiques essentielles

- Créée par le développeur
- Dont le nom **commence par**

17.2 — Règles des Hooks

Pourquoi des règles ?

React doit pouvoir identifier correctement les Hooks à chaque rendu. Pour cela, leur ordre d'appel doit rester prévisible et constant.

Les deux règles essentielles

1 Appeler les Hooks au niveau supérieur

Jamais dans une condition, une boucle ou une fonction imbriquée.

2 Appeler uniquement depuis un composant React ou un custom Hook

Jamais depuis une fonction JavaScript ordinaire.

Pourquoi les Hooks ont-ils des règles ?

React doit pouvoir identifier correctement les Hooks à chaque rendu. Pour cela, leur ordre d'appel doit rester prévisible.

1

Appeler les Hooks au niveau supérieur

Pas dans une condition, une boucle ou une fonction imbriquée.

2

Appeler uniquement depuis un composant React ou un custom Hook

Pas depuis une fonction JavaScript ordinaire.

Ne pas appeler un Hook dans une condition

✗ Incorrect

```
if (isLoggedIn) {  
  useEffect(() => {  
    // ...  
  }, []);  
}
```

Le Hook n'est appelé que si la condition est vraie — l'ordre d'appel devient imprévisible.

✓ Correct

```
useEffect(() => {  
  if (isLoggedIn) {  
    // ...  
  }  
}, [isLoggedIn]);
```

Le Hook est appelé dans les deux cas. C'est le **contenu** de l'effet qui dépend de la condition.

Ne pas appeler un Hook dans une boucle

 Incorrect

```
games.forEach(() => {  
  useState(false);  
});
```

Contextes interdits pour les Hooks

- Dans `for`
- Dans `while`
- Dans `forEach`
- Dans une fonction imbriquée

 Les Hooks restent au niveau supérieur du composant ou du custom Hook — jamais à l'intérieur d'une structure de contrôle ou d'une fonction imbriquée.

Un Hook dans un composant ou un Hook

✓ Dans un composant

```
function GamePage() {  
  const [favorite, setFavorite] =  
    useState(false);  
  
  return <GameCard />;  
}
```

✓ Dans un custom Hook

```
function useGames() {  
  const [games, setGames] =  
    useState<Game[]>([]);  
  
  return games;  
}
```

✗ Dans une fonction ordinaire

```
function calculateGames() {  
  const [games, setGames] =  
    useState([]);  
}
```

`calculateGames` n'est pas un composant ni un Hook. React ne peut pas gérer les Hooks appelés depuis une fonction ordinaire.

17.3 — Premier custom Hook

Objectif

Créer un premier custom Hook simple pour comprendre la mécanique de base avant d'aborder des cas plus complexes comme `useFetch` ou `useGames`.

Ce que nous allons construire

- Un Hook `useCounter` minimal
- Encapsulant `useState` et la logique d'incrément
- Exposant une interface claire au composant

Créer useCounter

Commençons par un exemple très simple.

```
function useCounter() {  
  const [count, setCount] = useState(0);  
  
  function increment() {  
    setCount((current) => current + 1);  
  }  
  
  return {  
    count,  
    increment,  
  };  
}
```

Analyse

Notre fonction utilise `useState`. Elle est donc elle-même un custom Hook — React la reconnaît comme telle grâce au préfixe `use`.

Ce qu'elle expose

- `count` — la valeur actuelle
- `increment` — la fonction pour l'augmenter

Utiliser useCounter

Composant utilisant le Hook

```
function Counter() {  
  const { count, increment } =  
    useCounter();  
  
  return (  
    <button onClick={increment}>  
      {count}  
    </button>  
  );  
}
```

Ce que le composant ne gère plus

- `useState` directement
- La fonction `increment`
- La logique de modification de l'état

Il utilise simplement l'interface fournie par le Hook — le composant est réduit à sa responsabilité d'affichage.

Que retourne un custom Hook ?

Un Hook peut retourner ce dont le composant a besoin — on définit ainsi une petite API pour notre Hook.

Exemple d'interface retournée

```
return {  
  count,  
  increment,  
  decrement,  
};
```

Principe de conception

Cacher les détails inutiles

Le composant n'a pas besoin de connaître l'implémentation interne.

Exposer uniquement le nécessaire

L'interface du Hook doit être minimale et explicite.

17.4 — useFetch

Contexte

Dans GameHub, de nombreux composants doivent récupérer des données depuis une API. Ce besoin récurrent est un excellent candidat pour un custom Hook générique.

Ce que nous allons construire

- Un Hook `useFetch` générique
- Gérant les états `data`, `loading`, `error`
- Réutilisable pour toute ressource API

Un besoin fréquent : récupérer des données

Dans GameHub, nous devons récupérer des données depuis une API. Un composant a généralement besoin de trois informations :

data

Les données récupérées depuis l'API, prêtes à être affichées.

loading

Indique si la requête est encore en cours d'exécution.

error

Contient le message d'erreur si la requête a échoué.

Le même principe peut être utilisé pour différentes ressources — c'est un bon candidat pour un custom Hook.

Première version de useFetch

```
function useFetch(url: string) {  
  const [data, setData] =  
    useState<unknown>(null);  
  const [loading, setLoading] =  
    useState(true);  
  const [error, setError] =  
    useState<string | null>(null);  
  
  useEffect(() => {  
    // effectuer la requête  
  }, [url]);  
  
  return { data, loading, error };  
}
```

État centralisé

data, loading, error dans un seul endroit

Effet centralisé

useEffect géré une seule fois dans le Hook

URL paramétrable

Le Hook s'adapte à n'importe quelle ressource

Utiliser useFetch

Composant GamesPage simplifié

```
function GamesPage() {  
  const { data, loading, error } =  
    useFetch("/api/games");  
  
  if (loading) {  
    return <p>Chargement...</p>;  
  }  
  
  if (error) {  
    return <p>Erreur : {error}</p>;  
  }  
  
  return <GameGrid games={data} />;  
}
```

Bénéfice immédiat

Le composant se concentre maintenant sur l'**affichage**. Toute la logique de récupération, de gestion du chargement et des erreurs est déléguée au Hook.

La séparation des responsabilités est claire et explicite.

17.5 — Typage d'un custom Hook

Enjeu TypeScript

Sans typage précis, `data` est de type `unknown`. TypeScript ne peut pas valider les accès aux propriétés ni fournir l'autocomplétion.

Objectif

Rendre `useFetch` générique pour que TypeScript connaisse le type exact des données retournées selon le contexte d'utilisation.

```
useFetch<Game[]>();
```

Pourquoi typer le Hook ?

Avec TypeScript, nous voulons connaître le type des données retournées.

Exemple GameHub

```
useFetch<Game[]>();
```

Cela permet notamment à TypeScript de savoir que `data` contient des `Game[]`.

Avantages du typage précis

- Autocomplétion sur les propriétés de `data`
- Détection d'erreurs à la compilation
- Documentation implicite de l'interface du Hook
- Refactoring plus sûr

Un Hook générique

On peut rendre `useFetch` générique en définissant une interface typée :

Interface de retour

```
interface UseFetchResult<T> {  
  data: T | null;  
  loading: boolean;  
  error: string | null;  
}
```

Signature du Hook

```
function useFetch<T>(  
  url: string  
) : UseFetchResult<T> {  
  // ...  
}
```

`T` représente le type des données récupérées — il sera inféré ou spécifié à l'appel.

Utiliser le type générique

```
const result = useFetch<Game[]>("/api/games");
```

Ici : `T = Game[]`

useFetch<Game[]>

```
useFetch<Game[]>("/api/games");
```

useFetch<User>

```
useFetch<User>("/api/user");
```

useFetch<Review[]>

```
useFetch<Review[]>("/api/reviews");
```

Même logique, types différents — le Hook est pleinement réutilisable à travers toute l'application.

17.6 — useGames

Du générique au métier

`useFetch` est un Hook générique. `useGames` est un Hook métier spécifique à GameHub — il encapsule la connaissance de l'URL et du type de données attendu.

Principe de spécialisation

On construit un Hook métier **au-dessus** d'un Hook générique. C'est une composition de Hooks — une pratique courante et recommandée.

Spécialiser le Hook pour GameHub

`useFetch` est générique. Mais GameHub peut avoir besoin d'une logique spécifique aux jeux.

Définition de `useGames`

```
function useGames() {  
  return useFetch<Game[]>("/api/games");  
}
```

Utilisation dans un composant

```
const {  
  data: games,  
  loading,  
  error,  
} = useGames();
```

Le composant n'a plus besoin de connaître l'URL de l'API.

Pourquoi useGames est intéressant ?

useFetch — besoin générique

« Récupérer des données depuis une URL. »

Réutilisable dans tout contexte de l'application.

useGames — besoin métier GameHub

« Récupérer les jeux de GameHub. »

Encapsule l'URL, le type, et toute logique spécifique aux jeux.

Cela permet d'encapsuler progressivement la logique de l'application — du générique vers le spécifique, couche par couche.

17.7 — Réutiliser la logique

Principe fondamental

Plusieurs composants peuvent utiliser le même Hook. La logique est définie une seule fois, mais chaque composant gère son propre affichage.

Point d'attention critique

Réutiliser un Hook ne signifie **pas** partager automatiquement son état. Chaque appel crée une instance indépendante.

Plusieurs composants peuvent utiliser le même Hook

GamesPage

```
function GamesPage() {  
  const { data, loading, error } =  
    useGames();  
  
  // affichage...  
}
```

HomePage

```
function HomePage() {  
  const { data, loading, error } =  
    useGames();  
  
  // affichage...  
}
```

La logique de chargement est définie dans `useGames`. Chaque composant peut ensuite gérer son propre affichage de manière indépendante.

Réutiliser la logique $\neq$ partager le state

Deux composants, deux états distincts

```
function ComponentA() {  
  const { data } = useGames();  
}  
  
function ComponentB() {  
  const { data } = useGames();  
}
```

Les deux composants utilisent le même code de Hook, mais chaque appel crée sa propre utilisation du Hook et donc son propre état.

Pour partager réellement un état

- Le **Context** React
- Un state placé dans un composant parent commun
- Une solution de gestion d'état adaptée (Zustand, Redux...)

17.8 — Quand créer — et quand ne pas créer — un custom Hook ?

La question à se poser

Un custom Hook doit répondre à un vrai besoin. Créer un Hook pour déplacer deux lignes de code n'apporte pas de valeur — il peut même nuire à la lisibilité.

Critères de décision

- La logique est-elle réutilisée ?
- Le composant est-il difficile à lire ?
- Plusieurs Hooks fonctionnent-ils ensemble pour une même fonctionnalité ?
- Veut-on donner un nom clair à une logique complexe ?

Quand créer un custom Hook ?

Un custom Hook est particulièrement intéressant lorsque :



Logique réutilisée

La même logique apparaît dans plusieurs composants.



Plusieurs Hooks combinés

Plusieurs Hooks fonctionnent ensemble pour une même fonctionnalité.



Composant difficile à lire

Extraire la logique dans un Hook améliore la lisibilité du composant.



Nommage d'une logique complexe

On veut donner un nom clair et métier à une logique complexe.

Quand ne pas créer de Hook ?

Il ne faut pas créer un custom Hook simplement pour déplacer quelques lignes.

Exemple contre-productif

```
const total = price * quantity;
```

Créer :

```
useTotal(price, quantity);
```

serait probablement inutile. Une simple variable est suffisante.

Règle pratique

Si la logique ne contient pas de Hook React (`useState`, `useEffect`...), ce n'est pas un custom Hook — c'est simplement une fonction utilitaire.

Créer un Hook lorsque cela rend la logique plus claire, réutilisable ou mieux organisée.

Un Hook doit avoir une responsabilité claire

✗ Hook difficile à comprendre

`useEverything()` qui gère :

- les jeux
- le profil
- le thème
- les notifications
- les formulaires

✓ Hooks spécialisés

`useGames()`

`useProfile()`

`useTheme()`

`useNotifications()`

Chaque Hook possède une responsabilité identifiable et limitée.

Hook générique ou Hook métier ?

Deux niveaux sont possibles — ils sont complémentaires, pas exclusifs.

Générique

```
useFetch<Game[]>("/api/games");
```

Réutilisable dans beaucoup de contextes différents de l'application.

Métier

```
useGames();
```

Simplifie spécifiquement le code de GameHub — encapsule l'URL et le type.

Exercice — Créer useCounter

Consigne

Créer un custom Hook : `useCounter()`

Il doit fournir :

- `count`
- `increment`
- `decrement`
- `reset`

Puis créer un composant qui utilise ce Hook.

Objectif pédagogique

Comprendre la séparation entre :

- La **logique** — encapsulée dans le Hook
- Le **composant** — réduit à l'affichage
- Les **données retournées** par le Hook — l'interface exposée

Correction — useCounter

Implémentation du Hook

```
function useCounter() {  
  const [count, setCount] = useState(0);  
  
  return {  
    count,  
    increment: () =>  
      setCount((c) => c + 1),  
    decrement: () =>  
      setCount((c) => c - 1),  
    reset: () => setCount(0),  
  };  
}
```

Utilisation dans un composant

```
const {  
  count,  
  increment,  
  decrement,  
  reset,  
} = useCounter();
```

Le composant dispose d'une interface claire et complète sans connaître les détails de l'implémentation.

Exercice — useGames

Consigne

Créer un Hook : `useGames()` qui :

1. Récupère les jeux depuis `/api/games`
2. Expose les jeux
3. Expose `loading`
4. Expose `error`

Puis utiliser ce Hook dans `GamesPage`.

Indice

S'appuyer sur `useFetch` déjà créé. `useGames` est une spécialisation de `useFetch` pour le contexte GameHub.

Penser à typer correctement avec `Game[]`.

Correction — useGames

Implémentation

```
function useGames() {  
  return useFetch<Game[]>(  
    "/api/games"  
  );  
}
```

Dans GamesPage

```
const {  
  data: games,  
  loading,  
  error,  
} = useGames();
```

Le composant utilise maintenant une logique métier clairement nommée. L'URL de l'API est encapsulée dans le Hook — le composant n'en a pas connaissance.

À retenir — Partie 17

Définition

Un custom Hook commence par `use`, peut utiliser d'autres Hooks, et encapsule une logique réutilisable.

Règles

Les règles des Hooks doivent être respectées — niveau supérieur, composant ou Hook uniquement.

Interface

Un Hook peut retourner données et fonctions. TypeScript permet de typer précisément son interface avec les génériques.

State indépendant

Réutiliser un Hook ne signifie pas partager automatiquement son state — chaque appel crée une instance indépendante.

Pertinence

Un custom Hook doit répondre à un vrai besoin — créer un Hook lorsque cela rend la logique plus claire, réutilisable ou mieux organisée.

Partie 18 — Gestion robuste des erreurs

FORMATION REACT / TYPESCRIPT

GAMEHUB PROJECT

Objectif

Construire des interfaces qui gèrent explicitement tous les états possibles d'une opération asynchrone.

Périmètre

Erreurs réseau, erreurs API, états de chargement, empty states, Error Boundaries.

18.1 — Pourquoi une application peut échouer ?

Constat fondamental

Même si notre code est correct, une application interagit avec des systèmes externes qu'elle ne contrôle pas. Les défaillances sont inévitables — il faut les anticiper.

Ce que cette section couvre

- Les sources d'échec d'une application
- Les états d'une interface complète
- La distinction entre les types d'erreurs
- Les stratégies de gestion adaptées

Une application ne contrôle pas tout

Même si notre code est correct, une application peut rencontrer des problèmes.

Réseau

- Absence de connexion Internet
- Serveur inaccessible
- Délai d'attente dépassé

API

- API qui renvoie une erreur
- Données inattendues
- Service externe temporairement indisponible

Utilisateur

- Saisie d'une valeur invalide
- Action dans un état inattendu

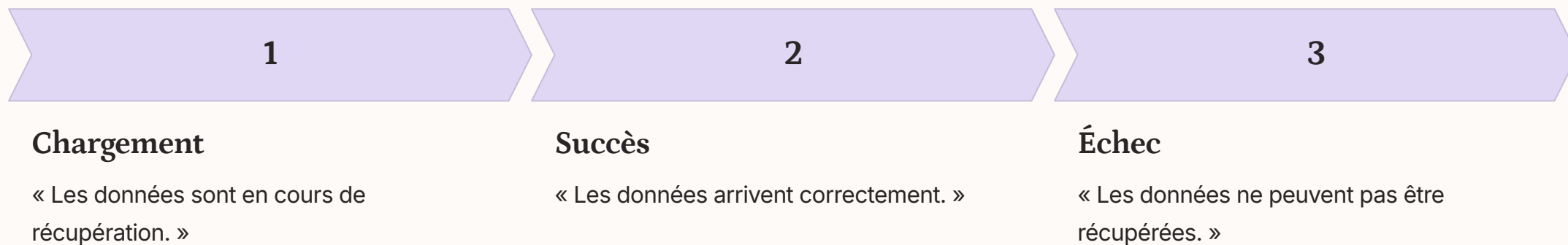
Code

- Erreur dans le code JavaScript
- Erreur de rendu React

Une application robuste doit prévoir les situations d'échec.

Succès ou échec : prévoir les deux

Pour une requête API, on ne doit pas seulement penser au cas nominal.



Une interface complète doit donc gérer plusieurs états — pas uniquement le cas nominal.

Les états d'une interface

Pour une page GameHub qui récupère des jeux, on peut avoir :

Loading

Requête en cours → indicateur de chargement

Success

Données reçues → liste des jeux

Error

Requête échouée → message d'erreur actionnable

Empty

Requête réussie mais aucune donnée → message adapté

📌 Ces états ne sont pas des erreurs de programmation. Ce sont des états normaux de l'application — ils doivent tous être gérés explicitement.

18.2 — Erreurs réseau

Définition

Une erreur réseau signifie que le client n'arrive pas correctement à communiquer avec le serveur — indépendamment du code de l'application.

Comportement de fetch

Dans ce cas, `fetch` peut rejeter la Promise — ce qui déclenche le bloc `catch` d'un `try/catch`.

Attention : une réponse HTTP 404 ou 500 ne provoque **pas** automatiquement un rejet de la Promise.

Qu'est-ce qu'une erreur réseau ?

Une erreur réseau signifie que le client n'arrive pas correctement à communiquer avec le serveur.

Exemples d'erreurs réseau

- Connexion Internet interrompue
- Serveur inaccessible
- Problème DNS
- Délai d'attente dépassé
- Connexion bloquée

Comportement de fetch

Dans ce cas, `fetch` peut rejeter la Promise.

La Promise rejetée sera capturée par le bloc `catch` d'un `try/catch`.

Gérer une erreur avec try/catch

```
try {  
  const response = await fetch("/api/games");  
  
  if (!response.ok) {  
    throw new Error("Erreur HTTP");  
  }  
  
  const games = await response.json();  
} catch (error) {  
  console.error(error);  
}
```

try

Contient le code susceptible d'échouer — la requête et le traitement de la réponse.

catch

Permet de réagir à l'erreur — journalisation, mise à jour de l'état, affichage d'un message.

fetch et erreur HTTP

Point important

```
const response =  
  await fetch("/api/games");
```

Une réponse HTTP 404 ou 500 ne provoque **pas** automatiquement un `catch`.

Solution : vérifier `response.ok`

```
if (!response.ok) {  
  throw new Error("Erreur HTTP");  
}
```

Puis l'erreur pourra être récupérée par `catch`. `response.ok` est `true` pour les codes 200–299 uniquement.

 Ne pas vérifier `response.ok` est une erreur fréquente — l'application peut sembler fonctionner alors que l'API a retourné une erreur.

18.3 — Erreurs API

Distinction clé

Une erreur API est différente d'une erreur réseau. Le serveur est joignable, mais il répond avec un code d'erreur HTTP indiquant que la requête n'a pas abouti comme prévu.

Implication pour le frontend

Le frontend doit distinguer les différents types d'erreurs pour afficher un message adapté à chaque situation.

Une API peut répondre avec une erreur

Codes HTTP d'erreur courants

400

Bad Request

401

Unauthorized

403

Forbidden

404

Not Found

500

Internal Server Error

Ce que le frontend doit distinguer

- Succès
- Erreur serveur
- Erreur d'autorisation
- Ressource inexistante
- Problème réseau

Ne pas afficher les détails techniques à l'utilisateur

✗ Mauvaise UX

- `TypeError: Failed to fetch`
- `HTTP 500 Internal Server Error`

Ces informations peuvent être utiles au développeur, mais sont incompréhensibles pour l'utilisateur.

✓ Message adapté à l'utilisateur

Impossible de charger les jeux. Veuillez réessayer.

Simple, compréhensible, et actionnable — l'utilisateur sait quoi faire.

Séparer erreur technique et message utilisateur

On peut conserver l'erreur technique pour le développeur tout en affichant un message adapté à l'utilisateur.

```
catch (error) {  
  console.error(error);  
  setError("Impossible de charger les jeux.");  
}
```

Pour le développeur

`console.error(error)` — l'erreur technique complète est disponible dans la console pour le diagnostic.

Pour l'utilisateur

`setError("...")` — un message simple et exploitable est affiché dans l'interface.

18.4 — États de chargement

Pourquoi c'est important

Une requête réseau n'est pas instantanée. Sans indication visuelle, une page vide peut donner l'impression que l'application est bloquée ou cassée.

Objectif

L'utilisateur doit toujours comprendre ce que fait l'application — même lorsqu'elle attend une réponse du serveur.

Pourquoi afficher un état de chargement ?



Requête non instantanée

Une requête réseau n'est pas instantanée — le délai peut varier de quelques millisecondes à plusieurs secondes.



Éviter la confusion

Sans indication, une page vide peut donner l'impression que l'application est bloquée ou que quelque chose s'est mal passé.



Expérience utilisateur

Pendant ce temps, l'utilisateur doit comprendre que quelque chose est en cours — c'est une exigence de base de l'UX.

Exemple avec loading

Déclaration des états

```
const [loading, setLoading] =  
  useState(true);  
const [error, setError] =  
  useState<string | null>(null);  
const [games, setGames] =  
  useState<Game[]>([]);
```

Affichage conditionnel

```
if (loading) {  
  return (  
    <p>Chargement des jeux...</p>  
  );  
}
```

Le composant retourne immédiatement un indicateur de chargement tant que la requête n'est pas terminée.

Une interface doit expliquer ce qu'elle fait

Critères d'un bon état de chargement

- Visible
- Compréhensible
- Cohérent avec l'action en cours

Exemples adaptés au contexte

Chargement des jeux...

Chargement du profil...

Enregistrement...

Éviter : *Loading...* si toute l'interface est destinée à des utilisateurs francophones.

18.5 — États d'erreur

Principe

Un état d'erreur ne doit pas seulement informer l'utilisateur qu'un problème est survenu — il doit lui permettre de continuer à utiliser l'application.

Ce qu'un bon état d'erreur contient

- Un message compréhensible
- Une action possible (réessayer, revenir en arrière...)
- Éventuellement une alternative

Afficher une erreur utile

Exemple d'état d'erreur actionnable

```
if (error) {  
  return (  
    <div>  
      <p>  
        Impossible de charger les jeux.  
      </p>  
      <button onClick={loadGames}>  
        Réessayer  
      </button>  
    </div>  
  );  
}
```

Ce que l'utilisateur comprend

1

Ce qui s'est passé

Les jeux n'ont pas pu être chargés.

2

Quelle action il peut effectuer

Il peut cliquer sur « Réessayer ».

Une bonne erreur doit être actionnable

Un bon message répond autant que possible aux questions suivantes :

Que s'est-il passé ?

Les jeux n'ont pas pu être chargés.

Que peut faire l'utilisateur ?

Réessayez dans quelques instants.

Y a-t-il une alternative ?

Vous pouvez également consulter votre bibliothèque.

L'objectif n'est pas seulement d'afficher une erreur. L'objectif est de permettre à l'utilisateur de continuer.

Réessayer une requête

```
async function loadGames() {  
  try {  
    setLoading(true);  
    setError(null);  
  
    const response = await fetch("/api/games");  
  
    if (!response.ok) {  
      throw new Error("Erreur HTTP");  
    }  
  
    const data = await response.json();  
    setGames(data);  
  } catch (error) {  
    setError("Impossible de charger les jeux.");  
  } finally {  
    setLoading(false);  
  }  
}
```

❏ `finally` est exécuté que la requête réussisse ou échoue. Il est donc pratique pour terminer l'état de chargement dans tous les cas.

18.6 — Empty states

Définition

Un empty state est la situation où une requête a réussi, mais la liste retournée est vide. Ce n'est pas une erreur — c'est un état normal qui mérite une interface dédiée.

Pourquoi c'est important

Confondre un empty state avec une erreur conduit à des messages trompeurs. L'utilisateur doit comprendre que la requête a fonctionné, mais qu'il n'y a rien à afficher.

Une liste vide n'est pas forcément une erreur

Situation

```
games.length === 0
```

Cela peut simplement signifier :

La requête a fonctionné, mais aucun jeu ne correspond au résultat.

Distinction fondamentale

Ce n'est pas la même situation qu'une erreur API. La requête a abouti — il n'y a simplement rien à afficher dans ce contexte.

Traiter un empty state comme une erreur est une erreur de conception UX.

Empty state vs error

Error

Les données n'ont pas pu être récupérées.

Impossible de récupérer les jeux. Réessayez.

Empty

Les données ont été récupérées, mais la liste est vide.

Aucun jeu trouvé. Essayez une autre recherche.

La distinction est importante — le message et l'action proposée à l'utilisateur sont différents dans chaque cas.

Empty state dans GameHub

Affichage conditionnel

```
if (results.length === 0) {  
  return (  
    <p>  
      Aucun jeu ne correspond  
      à votre recherche.  
    </p>  
  );  
}
```

Avec une action proposée

On peut également proposer une action pour aider l'utilisateur à sortir de cet état :

Aucun jeu trouvé. Modifiez votre recherche ou supprimez les filtres.

L'utilisateur comprend la situation et sait comment y remédier.

18.7 — Error Boundaries

Un type d'erreur différent

Jusqu'ici, nous avons traité les erreurs réseau et API. Mais un composant React peut également provoquer une erreur pendant son rendu — un cas différent qui nécessite un mécanisme différent.

Solution React

Un Error Boundary permet à React de détecter certaines erreurs provenant de l'arbre de composants et d'afficher une interface de secours plutôt qu'une page complètement inutilisable.

Toutes les erreurs ne sont pas des erreurs API

Erreurs traitées jusqu'ici

- API
- Réseau
- Données

Erreur de rendu React

```
function GameCard() {  
  throw new Error(  
    "Erreur dans GameCard"  
  );  
  
  return <div>...</div>;  
}
```

Cette erreur est différente d'une erreur réseau — elle survient pendant le rendu du composant, pas pendant une requête.

Le rôle d'un Error Boundary

Un Error Boundary permet à React de détecter certaines erreurs provenant de l'arbre de composants et d'afficher une interface de secours.

Sans Error Boundary

Une erreur de rendu non gérée peut rendre toute l'interface inutilisable — l'écran devient blanc ou affiche une erreur technique.

Avec Error Boundary

Au lieu d'avoir une interface complètement inutilisable, on peut afficher :

Une erreur est survenue. Veuillez réessayer.

L'idée : prévoir une interface de secours lorsqu'une partie de l'application ne fonctionne plus.

Error Boundary : à quoi ça sert ?

Protéger l'interface

Protéger une partie importante de l'interface contre les erreurs de rendu.

Afficher un fallback

Afficher une interface de secours compréhensible à la place du composant en erreur.

Préserver l'expérience

Éviter qu'une erreur de rendu détruise toute l'expérience utilisateur.

Faciliter le diagnostic

Faciliter le diagnostic d'une erreur en isolant la partie défailante.

Il ne remplace pas la gestion des erreurs réseau ou API.

Ce qu'un Error Boundary ne remplace pas

Un Error Boundary n'est pas une solution universelle. Il ne faut pas l'utiliser pour remplacer la gestion des erreurs réseau.

Ce qu'il ne remplace pas

```
try {  
  await fetch(...);  
} catch {  
  // gérer l'erreur réseau  
}
```

Tableau des responsabilités

try/catch

Requête réseau

response.ok

Réponse HTTP invalide

loading state

Chargement en cours

empty state

Données absentes

Error Boundary

Erreur de rendu React

18.8 — Construire une UX résiliente

Définition

Une interface résiliente ne suppose pas que tout fonctionnera toujours. Elle anticipe les défaillances et offre à l'utilisateur les moyens de continuer malgré les problèmes.

Ce que cette section couvre

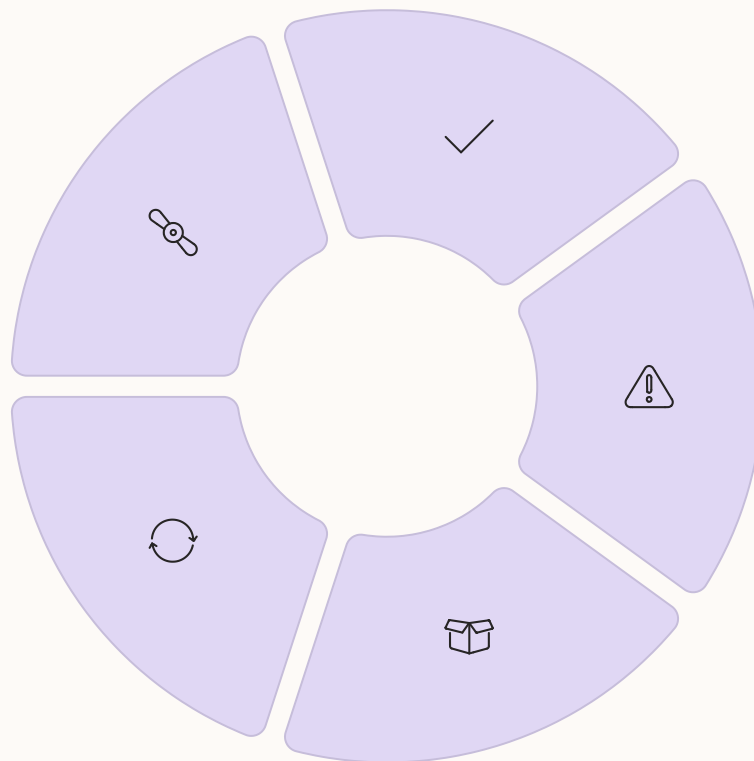
- La structure complète d'une page résiliente
- Les erreurs à ne jamais masquer
- La checklist GameHub avant livraison

Une interface résiliente

Une interface résiliente ne suppose pas que tout fonctionnera toujours.

Chargement
Indicateur visible pendant la requête

Réessayer
Possibilité de relancer l'opération



Succès

Affichage des données récupérées

Erreur

Message compréhensible et action possible

Absence de données

Message adapté et suggestion d'action

Exemple : page GameHub complète

Une page de jeux peut suivre cette logique — chaque situation possède son interface dédiée.

```
if (loading) {  
  return <Loading />;  
}  
  
if (error) {  
  return <GameLoadError onRetry={loadGames} />;  
}  
  
if (games.length === 0) {  
  return <EmptyGames />;  
}  
  
return <GameGrid games={games} />;
```

Erreur

Montrer l'erreur et bouton de réessai.

Vide

Indiquer absence de jeux et suggestions.

Chargement

Afficher spinner et message d'attente.

Succès

Présenter la liste de jeux chargée.

Ne pas masquer les erreurs

✗ À éviter absolument

```
catch (error) {  
  // rien  
}
```

Une erreur ignorée peut produire :

- Une interface vide
- Un comportement incompréhensible
- Des données incohérentes
- Une difficulté à diagnostiquer le problème

✓ Une erreur doit être

1

Traitée

Mise à jour de l'état
d'erreur

2

Affichée si nécessaire

Message compréhensible
pour l'utilisateur

3

Journalisée si utile

console.error pour le développeur

Exercice — Rendre GamesPage robuste

Point de départ

```
const [games, setGames] =  
  useState<Game[]>([]);
```

À ajouter

- loading
- error
- try/catch
- Vérification de response.ok
- Message si aucun jeu n'est disponible
- Bouton « Réessayer »

Objectif pédagogique

Gérer explicitement les différents états de la page :

- Loading
- Error (avec retry)
- Empty
- Success

Chaque état doit avoir une interface dédiée et compréhensible.

Correction — structure de la page

```
if (loading) {  
  return <p>Chargement...</p>;  
}  
  
if (error) {  
  return (  
    <div>  
      <p>{error}</p>  
      <button onClick={loadGames}>  
        Réessayer  
      </button>  
    </div>  
  );  
}  
  
if (games.length === 0) {  
  return <p>Aucun jeu disponible.</p>;  
}  
  
return <GameGrid games={games} />;
```

📌 La structure devient immédiatement lisible : **loading** → **error** → **empty** → **success**. Chaque état est traité explicitement, dans l'ordre.

Exercice — Identifier le bon traitement

Pour chaque situation, choisir le traitement adapté.

A

L'utilisateur perd Internet pendant le chargement.

B

L'API répond 500.

C

La recherche ne retourne aucun jeu.

D

Un composant React provoque une erreur pendant son rendu.

E

Les données sont encore en cours de récupération.

Correction

A — Réseau interrompu

→ **Erreur réseau** — catch de la Promise rejetée par fetch

B — HTTP 500

→ **Erreur API** — vérification de response.ok

C — Aucun résultat

→ **Empty state** — games.length === 0

D — Erreur de rendu

→ **Error Boundary** — mécanisme React dédié

E — Requête en cours

→ **Loading state** — indicateur de chargement

La première compétence est donc de **reconnaître le type de problème** avant de choisir une solution.

Checklist GameHub

Avant de considérer une page comme terminée, vérifier :

États de l'interface

- ☐ Que voit l'utilisateur pendant le chargement ?
- ☐ Que se passe-t-il si l'API échoue ?
- ☐ Que se passe-t-il sans connexion ?
- ☐ Que se passe-t-il si la liste est vide ?

Qualité de l'expérience

- ☐ Peut-on réessayer ?
- ☐ Les messages sont-ils compréhensibles ?
- ☐ Les erreurs techniques restent-elles utiles au développeur ?
- ☐ Les erreurs de rendu importantes sont-elles protégées ?

À retenir — Partie 18

Une application robuste ne gère pas uniquement le cas nominal.

Loading

Les données sont en cours de récupération — afficher un indicateur visible et compréhensible.

Success

Les données sont disponibles — afficher le contenu attendu.

Error

Quelque chose a empêché l'opération — message clair, action possible, erreur technique journalisée.

Empty

L'opération a réussi, mais il n'y a rien à afficher — distinguer de l'erreur, proposer une action.

Error Boundary

Protège l'interface contre certaines erreurs de rendu React — complément, pas substitut.

- ❏ Une bonne gestion des erreurs ne consiste pas seulement à empêcher les crashes : elle permet à l'utilisateur de comprendre la situation et, lorsque c'est possible, de continuer à utiliser l'application.

FORMATION REACT / TYPESCRIPT

GAMEHUB PROJECT

Partie 19 — Performance React

Re-renders

React.memo

useMemo

useCallback

Lazy loading

19.1 — Pourquoi une application peut ralentir ?

Contexte

Même avec un code propre et une API rapide, les applications React peuvent être ralenties par un nombre excessif de "re-renders" des composants. Cette section explore les causes de ces re-renders et les stratégies d'optimisation.

Ce que cette section couvre

- Comprendre le mécanisme de re-render
- Identifier les causes courantes de performance
- Optimiser avec `React.memo`, `useMemo`, `useCallback`
- Techniques de Lazy Loading pour les composants et les données

Une application peut devenir coûteuse

Une interface React peut ralentir lorsque le navigateur doit effectuer trop de travail.

Exemples

- beaucoup de composants
- beaucoup de données à afficher
- calculs coûteux
- trop de mises à jour
- images lourdes
- trop de JavaScript chargé au démarrage

À retenir

Mais une application avec beaucoup de composants n'est pas automatiquement lente. Avant d'optimiser, il faut identifier ce qui coûte réellement du temps.

Performance : quel est le problème ?

Une interface peut être lente à plusieurs moments :

Situation	Exemple
Chargement initial	beaucoup de JavaScript
Interaction	clic ou saisie peu réactive
Rendu	trop de composants recalculés
Calcul	filtre ou tri coûteux
Réseau	données longues à récupérer
Images	fichiers trop lourds

La performance ne concerne donc pas uniquement React.

19.2 — Comprendre les re-renders

Qu'est-ce qu'un re-render ?

Le re-render est le processus par lequel React met à jour l'interface utilisateur. Comprendre quand et pourquoi les composants re-rendent est essentiel pour diagnostiquer et optimiser les problèmes de performance dans votre application React.

Le cycle de vie du rendu

Dans React, lorsqu'un composant est affiché pour la première fois, on parle de "render" initial. Par la suite, si des changements surviennent, le composant peut être "re-rendered" pour refléter ces modifications.

Les principales causes de re-renders sont :

- La mise à jour de l'état local du composant (via `useState` ou `useReducer`).
- La modification des props reçues de son composant parent.
- Le re-render d'un composant parent, qui entraîne le re-render de tous ses enfants par défaut.
- La modification du contexte (`useContext`) que le composant consomme.

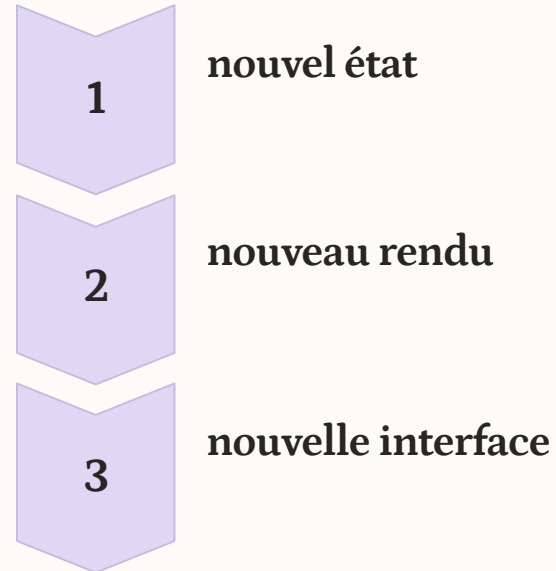
Chaque re-render implique une exécution de la fonction de rendu du composant et une comparaison du DOM virtuel. Si le travail est excessif ou inutile, cela peut entraîner des ralentissements perceptibles pour l'utilisateur.

Qu'est-ce qu'un re-render ?

Lorsqu'une donnée utilisée par un composant change, React peut effectuer un nouveau rendu du composant.

```
const [count, setCount] = useState(0);

return (
  <button onClick={() => setCount(count + 1)}>
    {count}
  </button>
);
```



Le re-render est donc un mécanisme normal de React.

Un re-render n'est pas forcément un problème

Il ne faut pas penser : « Re-render = mauvaise performance ». React est justement conçu pour mettre à jour l'interface lorsque les données changent.

Le problème apparaît lorsque :

- les rendus sont très nombreux
- les composants sont très coûteux
- des calculs importants sont répétés inutilement

Exemple GameHub

```
<GameGrid games={games} />
```

Si un état de la page change, certains composants peuvent être rendus à nouveau.

Avec 10 jeux, ce travail est généralement insignifiant. Avec plusieurs milliers de cartes complexes, il peut devenir intéressant d'analyser le comportement. La taille réelle du problème compte.

19.3 — Identifier un problème de performance

Avant de plonger dans les techniques d'optimisation, il est crucial de savoir si votre application souffre réellement d'un problème de performance et, si oui, d'identifier la source exacte de ce problème.

Le principe d'abord

L'optimisation prématurée est la racine de tous les maux. Avant d'optimiser, il faut savoir si un problème existe et où il se situe.

Comment identifier les goulots d'étranglement ?

Les outils de développement du navigateur et les extensions React sont vos meilleurs alliés pour débusquer les problèmes de performance :

- Utiliser le **React DevTools Profiler** pour visualiser les temps de rendu des composants.
- Surveiller les **métriques de performance du navigateur** (FPS, temps de chargement, utilisation CPU) via l'onglet Performance.
- Rechercher les **re-renders inutiles** de composants qui n'auraient pas dû se mettre à jour.
- Identifier les **calculs coûteux** ou les opérations bloquantes dans le code.

Ne pas optimiser au hasard

Avant d'ajouter :

```
memo()  
useMemo()  
useCallback()
```

L'optimisation doit répondre à un problème observé.

il faut se demander :

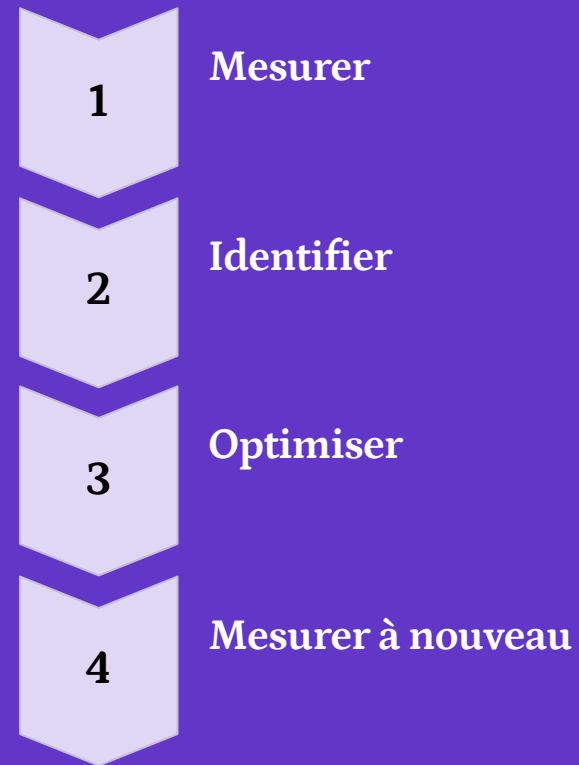
- Où est le ralentissement ?
- Quand apparaît-il ?
- Quel composant travaille trop ?
- Quel calcul est coûteux ?
- Est-ce réellement React qui pose problème ?

Mesurer avant de modifier

Pour analyser une application React, on peut utiliser les outils de développement du navigateur et les outils de développement React. On cherche notamment :

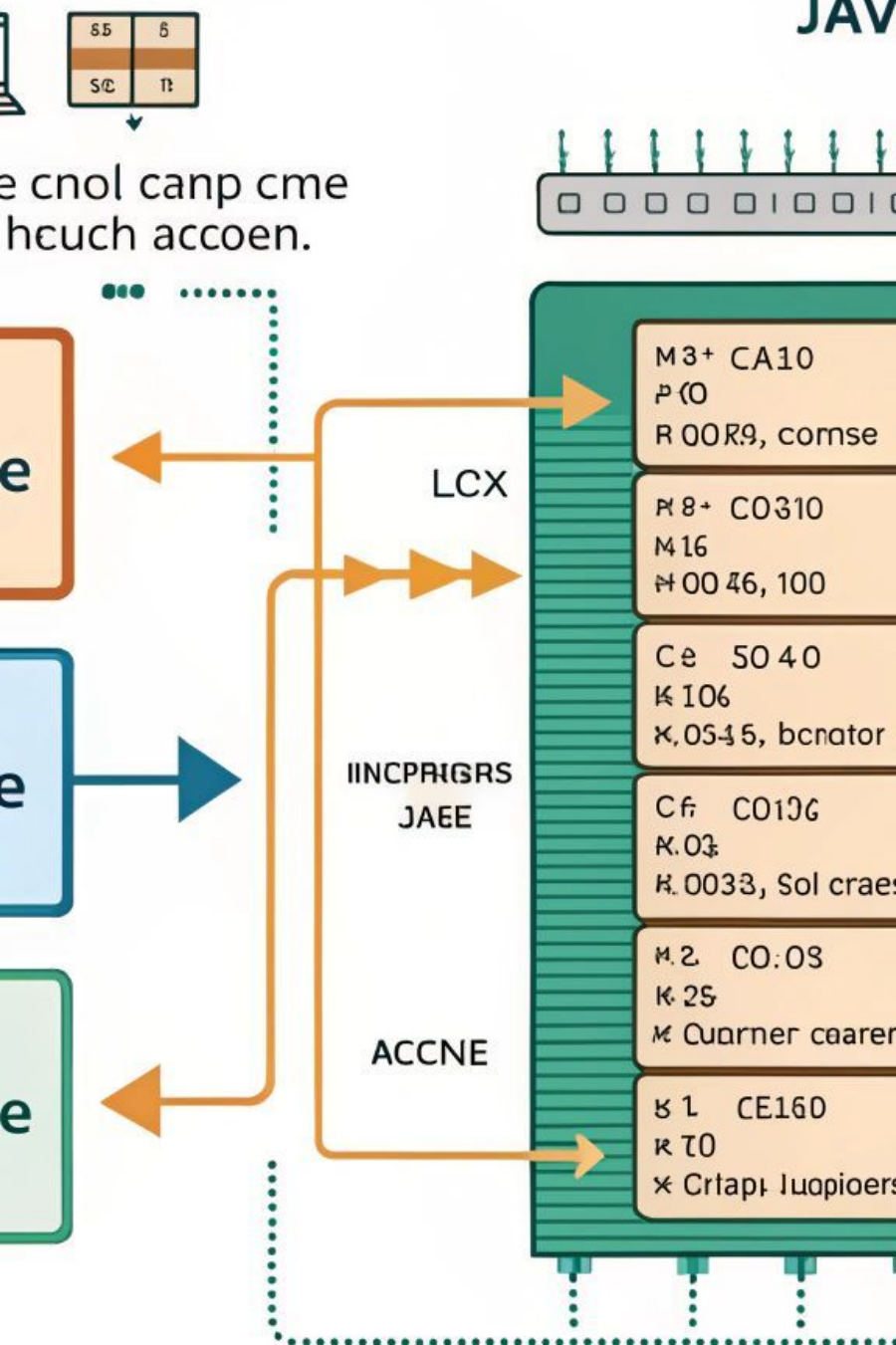
- les composants qui se rendent
- la fréquence des rendus
- les opérations coûteuses
- les interactions lentes

Principe :



Cache

CPU Cache
JAV



19.4 — React.memo

React.memo est une fonction d'ordre supérieur (Higher-Order Component - HOC) que vous pouvez utiliser pour optimiser les performances de rendu des composants fonctionnels dans React.

Le concept

React.memo mémorise le résultat d'un composant et, par défaut, ne le re-rend pas si ses props n'ont pas changé. Cela permet d'éviter des re-renders inutiles et coûteux pour les composants qui reçoivent les mêmes props.

Comment ça marche ?

Lorsque votre composant est enveloppé dans React.memo, React effectue une comparaison superficielle (shallow comparison) des props actuelles et des props précédentes. Si cette comparaison révèle que les props sont identiques, React saute le re-render du composant et réutilise le dernier rendu mémorisé.

- Idéal pour les composants "purs" (qui rendent le même résultat pour les mêmes props).
- Permet de réduire la charge de travail du navigateur.
- Ne pas utiliser partout : le coût de la comparaison des props peut parfois dépasser le bénéfice de la mémorisation.

Le problème des props inchangées

```
const GameCard = ({ game }: GameCardProps) => {  
  return
```

```
{game.title}
```

```
; };
```

Si le parent est rendu à nouveau, GameCard peut lui aussi être rendu à nouveau. Mais si ses props n'ont pas changé, ce travail peut parfois être évité.

Utiliser memo

```
const GameCard = memo(  
  function GameCard({ game }: GameCardProps) {  
    return (  
      <article>  
        {game.title}  
      </article>  
    );  
  }  
);
```

memo permet à React de réutiliser le résultat précédent lorsque les props sont considérées comme inchangées.

memo ne bloque pas tous les renders

memo ne signifie pas : « Ce composant ne sera plus jamais rendu. »

memo est donc un outil d'optimisation, pas une garantie absolue.

Le composant peut toujours être rendu si :

- ses props changent
- son propre state change
- un contexte utilisé par le composant change

19.5 — useMemo

`useMemo` est un hook React qui vous permet de mémoriser des valeurs calculées coûteuses.

Il prend une fonction de "création" et un tableau de dépendances. `useMemo` ne recalcule la valeur mémorisée que si l'une des dépendances a changé.

Cela est utile pour éviter des calculs intensifs sur chaque rendu, améliorant ainsi la performance de l'application.

Le concept

`useMemo` permet de "mémoriser" le résultat d'une fonction et de ne le recalculer que lorsque ses dépendances changent. Si les dépendances n'ont pas changé entre les rendus, React réutilisera la dernière valeur mémorisée sans exécuter la fonction.

Comment ça marche ?

Lorsqu'un composant est rendu, React vérifie si les dépendances passées à `useMemo` ont changé. Si non, la valeur précédemment calculée est renvoyée. Si oui, la fonction est exécutée, la nouvelle valeur est renvoyée et mémorisée pour le prochain rendu.

- Optimise les calculs coûteux et les traitements de données.
- Peut prévenir des re-renders inutiles de composants enfants si la valeur mémorisée est une prop.
- Ne pas utiliser pour des calculs non coûteux, car la mémorisation a un coût de mémoire et de comparaison.

Le problème des calculs coûteux

Imaginons que GameHub possède beaucoup de jeux. Nous voulons filtrer et trier les résultats.

```
const filteredGames = games
  .filter((game) => game.title.includes(query))
  .sort((a, b) => a.title.localeCompare(b.title));
```

Ce calcul est effectué pendant le rendu. Pour quelques éléments, ce n'est généralement pas un problème.

À quoi sert useMemo ?

useMemo permet de mémoriser le résultat d'un calcul entre les rendus.

```
const filteredGames = useMemo(() => {  
  return games  
    .filter((game) =>  
      game.title.includes(query)  
    )  
    .sort((a, b) =>  
      a.title.localeCompare(b.title)  
    );  
}, [games, query]);
```

Le calcul est réévalué lorsque games ou query change.

Comprendre les dépendances

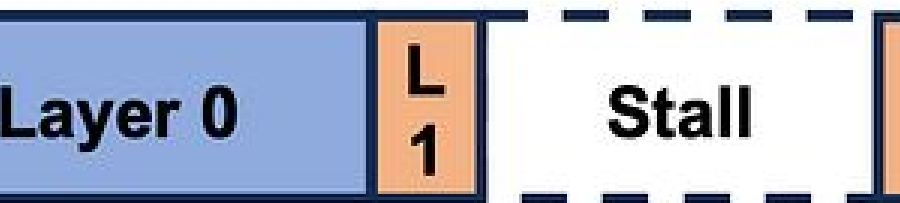
```
useMemo(() => {  
  return expensiveCalculation();  
}, [games, query]);
```

Les dépendances répondent à la question : « Dans quels cas le résultat doit-il être recalculé ? » Si games et query ne changent pas, React peut réutiliser le résultat mémorisé.

Full Layer

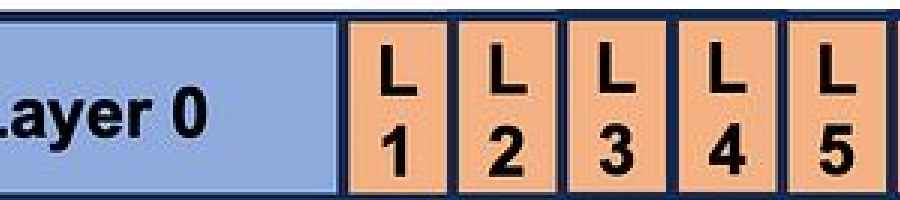


SV



GPU waiting for KVCache lo

er 1-6 Layer 8



b) GPU running without wait

19.6 — useCallback

`useCallback` est un hook React qui vous permet de mémoriser des fonctions de rappel (callbacks) entre les rendus.

Il prend une fonction et un tableau de dépendances. `useCallback` ne recrée la fonction mémorisée que si l'une de ses dépendances a changé.

Cela est utile pour éviter des créations inutiles de fonctions qui pourraient déclencher des re-renders de composants enfants optimisés par `React.memo`.

Le concept

`useCallback` permet de "mémoriser" une fonction de sorte qu'elle ne soit recréée que lorsque ses dépendances changent. Si les dépendances n'ont pas changé, React réutilise la même instance de fonction du rendu précédent.

Comment ça marche ?

Lorsqu'un composant est rendu, React vérifie si les dépendances passées à `useCallback` ont changé. Si non, la même instance de fonction précédemment mémorisée est renvoyée. Si oui, la fonction est recréée, la nouvelle instance est renvoyée et mémorisée pour le prochain rendu.

- C'est utile pour optimiser les performances des composants enfants qui dépendent de cette fonction (ex: un enfant enveloppé dans `React.memo`).
- Préserve l'identité référentielle des fonctions, évitant ainsi des re-renders inutiles.
- Ne pas utiliser partout : comme `useMemo`, la mémorisation a un coût et ne devrait être appliquée qu'aux fonctions qui sont coûteuses à recréer ou qui causent des re-renders inutiles de composants enfants.

Une fonction est une nouvelle valeur

À chaque rendu, une fonction définie dans le composant peut être recréée.

```
function GamesPage() {  
  const handleFavorite = (game: Game) => {  
    // ...  
  };  
  
  return <GameGrid onFavorite={handleFavorite} />;  
}
```

À chaque nouveau rendu, handleFavorite peut avoir une nouvelle référence.

Pourquoi cela peut poser problème ?

Cela devient intéressant notamment avec un composant mémorisé :

```
const GameGrid = memo(GameGridComponent);
```

Si une fonction passée en prop change de référence à chaque rendu, elle peut faire considérer la prop comme différente.

Dans certains cas, cela réduit l'intérêt de memo.

Utiliser useCallback

```
const handleFavorite = useCallback(  
  (game: Game) => {  
    toggleFavorite(game);  
  },  
  [toggleFavorite]  
);
```

useCallback permet de conserver une même référence de fonction tant que ses dépendances ne changent pas.

useCallback mémorise une fonction, tandis que useMemo mémorise le résultat d'un calcul.

useMemo vs useCallback

Hook	Mémoire
useMemo	une valeur calculée
useCallback	une fonction

```
const total = useMemo(  
  () => calculateTotal(items),  
  [items]  
);
```

```
const handleClick = useCallback(  
  () => saveItem(item),  
  [item]  
);
```

19.7 — Quand les utiliser ?

useMemo

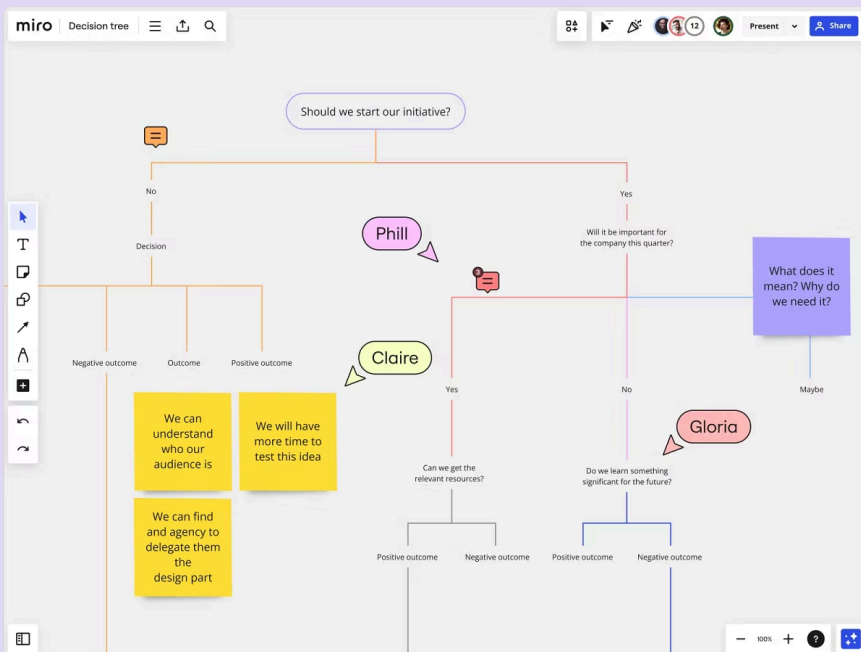
- Optimise les calculs coûteux et les traitements de données.
- Prévenir des re-renders inutiles de composants enfants si la valeur mémorisée est une prop.

Il ne faut pas utiliser `useMemo` pour des calculs non coûteux, car la mémorisation a un coût de mémoire et de comparaison.

useCallback

- C'est utile pour optimiser les performances des composants enfants qui dépendent de cette fonction (ex: un enfant enveloppé dans `React.memo`).
- Préserve l'identité référentielle des fonctions, évitant ainsi des re-renders inutiles.

Il ne faut pas utiliser `useCallback` partout : comme `useMemo`, la mémorisation a un coût et ne devrait être appliquée qu'aux fonctions qui sont coûteuses à recréer ou qui causent des re-renders inutiles de composants enfants.

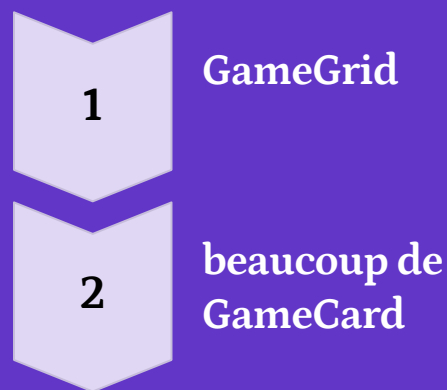


React.memo : quand ?

memo peut être pertinent lorsqu'un composant :

- est rendu très fréquemment
- est relativement coûteux
- reçoit souvent les mêmes props
- bénéficie réellement de l'absence de certains re-renders

Exemple potentiel dans GameHub :



Une optimisation peut être intéressante si les cartes sont nombreuses et coûteuses.

and Concentration in Sp

omponents of Concentration

ation: Focusing on relevant cues and ignoring irrelevant distractors (e.g., crowd noise).

entional Focus: Keeping focus on the task at hand and preventing the mind wander.

ireness: Understanding what is relevant (e.g., the sense).

ional Focus: The agility to switch focus when demands change.

useMemo : quand ?

useMemo peut être pertinent pour :

- un calcul réellement coûteux
- une transformation importante d'un grand volume de données
- une valeur dont la stabilité apporte un bénéfice mesurable

Exemple :

```
const visibleGames = useMemo(  
  () => filterAndSortGames(games, filters),  
  [games, filters]  
);
```



useCallback : quand ?

useCallback peut être pertinent lorsqu'une fonction :

- est passée à un composant mémorisé
- participe à une optimisation avec memo
- doit conserver une référence stable pour une raison précise

Il n'est pas nécessaire de mémoriser toutes les fonctions d'une application.



19.8 — Quand ne pas les utiliser ?

useMemo

Il ne faut pas utiliser `useMemo` pour des calculs non coûteux, car la mémorisation a un coût de mémoire et de comparaison.

- Évitez-le pour des calculs triviaux ou qui s'exécutent très rapidement.
- Le coût de la mémorisation (allocation de mémoire, comparaison des dépendances) peut dépasser le bénéfice pour les opérations simples.

useCallback

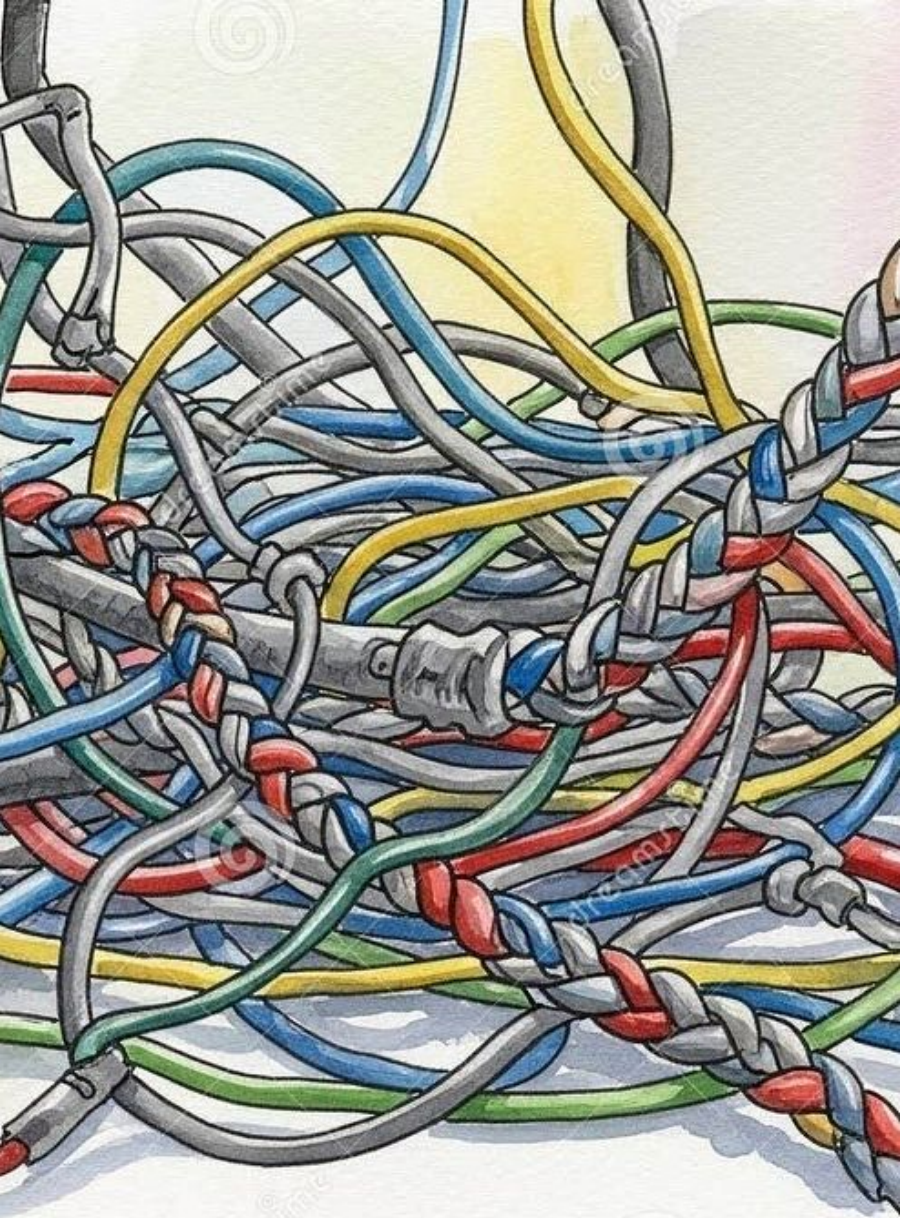
Il ne faut pas utiliser `useCallback` partout : comme `useMemo`, la mémorisation a un coût et ne devrait être appliquée qu'aux fonctions qui sont coûteuses à recréer ou qui causent des re-renders inutiles de composants enfants.

- Inutile si la fonction n'est pas passée à un composant enfant mémorisé (`React.memo`) ou à un autre hook (comme `useEffect` ou `useMemo`) nécessitant une référence stable.
- Le sur-usage peut alourdir le code sans gain de performance notable.

React.memo

`React.memo` n'est pas toujours nécessaire et peut même ajouter une surcharge inutile si un composant :

- Est rarement rendu.
- Est très léger à rendre (peu de calculs, peu d'éléments DOM).
- Reçoit des props qui changent fréquemment et sont souvent des objets ou fonctions non mémorisés (ce qui annule l'intérêt de la mémorisation).
- Ne bénéficie pas significativement de l'absence de certains re-renders (le coût de la comparaison des props peut être plus élevé que le re-render lui-même).



L'optimisation a elle-même un coût

memo, useMemo et useCallback ajoutent de la complexité. Il faut notamment :

- gérer les dépendances
- comprendre les références
- maintenir davantage de code
- raisonner sur le comportement mémorisé

Une optimisation inutile peut rendre le code plus compliqué sans le rendre plus rapide.

Exemple : optimisation inutile

Pour un petit tableau :

```
const total = useMemo(  
  () => price * quantity,  
  [price, quantity]  
);
```

Le calcul est extrêmement simple. Ajouter useMemo apporte probablement peu ou pas de bénéfice.

Il est plus clair d'écrire :

```
const total = price * quantity;
```

La simplicité est souvent la meilleure optimisation lorsqu'aucun problème n'est mesuré.

Attention à l'over-optimization

Éviter de transformer automatiquement :

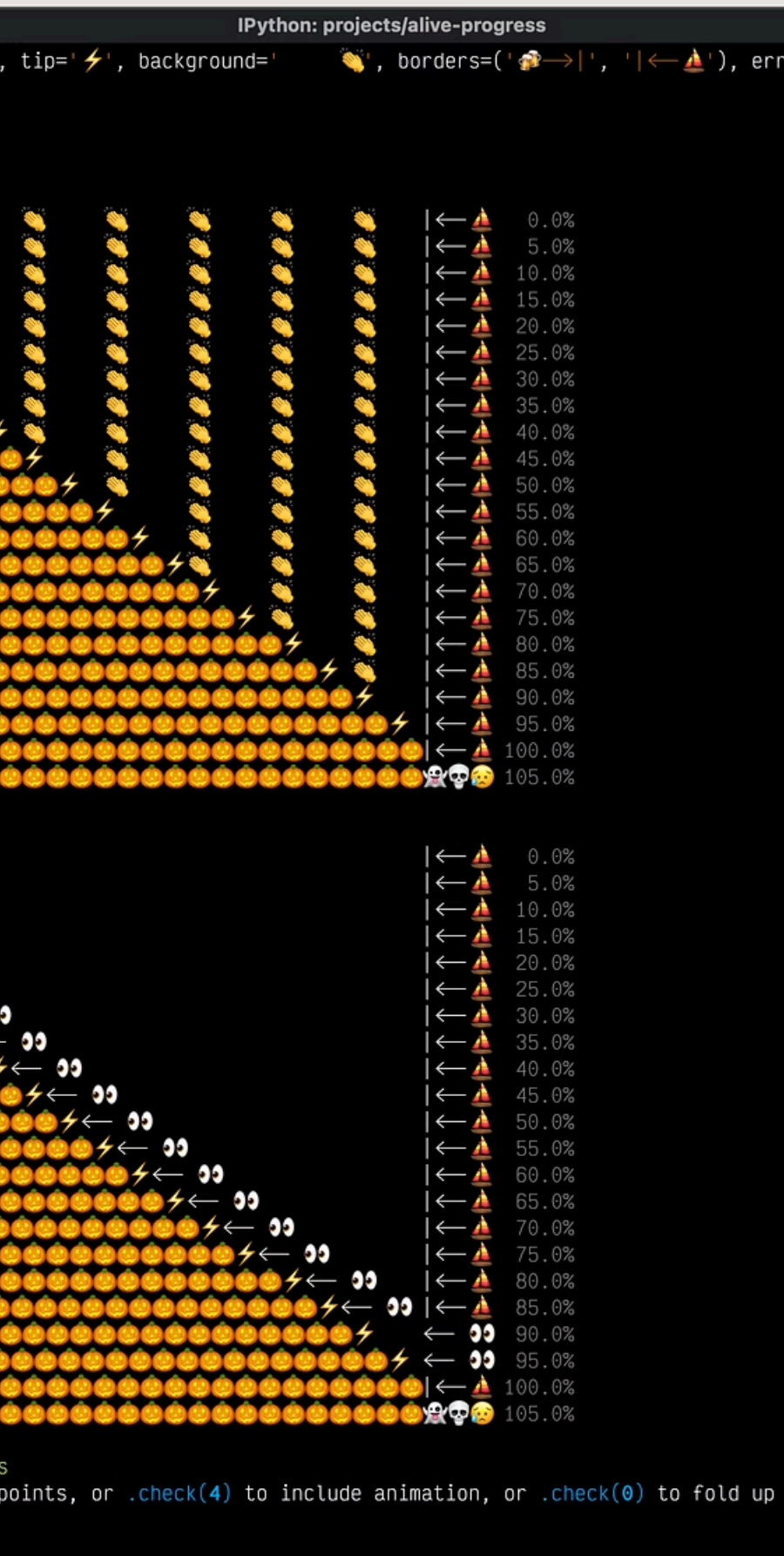
```
function GameCard() {  
  // ...  
}
```

en :

```
const GameCard = memo(/* ... */);
```

et d'ajouter partout : `useMemo(...)` `useCallback(...)` sans raison.

La question doit toujours être : Quel problème cette optimisation résout-elle ?



19.9 — Lazy loading

Le lazy loading (chargement paresseux) est une technique d'optimisation qui permet de charger des composants ou des modules de votre application uniquement lorsqu'ils sont nécessaires, au lieu de les charger tous au démarrage. Cela réduit la taille du bundle initial et améliore le temps de chargement de l'application, en particulier pour les applications volumineuses.

En React, le lazy loading est généralement mis en œuvre à l'aide des fonctions `React.lazy()` et du composant `Suspense`.

Exemple d'utilisation avec `React.lazy` et `Suspense` :

```
import React, { lazy, Suspense } from 'react';

// Chargement paresseux du composant 'About'
const About = lazy(() => import('./About'));
const Contact = lazy(() => import('./Contact'));

function App() {
  return (
    <div>
      <h1>Bienvenue sur mon site</h1>
      <Suspense fallback={<div>Chargement...</div>}>
        <About />
        <Contact />
      </Suspense>
    </div>
  );
}
```

Dans cet exemple, les composants `About` et `Contact` ne seront chargés que lorsque l'application tentera de les afficher pour la première fois. Pendant le chargement, le `fallback` de `Suspense` s'affiche.

Le problème du chargement initial

Une application peut contenir beaucoup de code. Si tout est chargé immédiatement, le navigateur doit télécharger et traiter davantage de JavaScript avant que l'application soit complètement disponible.

GameHub peut avoir plusieurs pages :

- accueil
- jeux
- détails
- bibliothèque
- profil

L'utilisateur n'a pas nécessairement besoin de tout charger immédiatement.

Charger une page à la demande

React permet d'utiliser lazy :

```
const ProfilePage = lazy(  
  () => import("./pages/ProfilePage")  
);
```

Puis :

```
<Suspense fallback={<p>Chargement...</p>}>  
  <ProfilePage />  
</Suspense>
```

Le code de ProfilePage peut ainsi être chargé lorsqu'il devient nécessaire.

Lazy loading avec les routes

Pour une application comme GameHub, le lazy loading peut être particulièrement intéressant pour certaines pages :

```
const ProfilePage = lazy(  
  () => import("./pages/ProfilePage")  
);  
  
const LibraryPage = lazy(  
  () => import("./pages/LibraryPage")  
);
```

Puis les routes utilisent ces composants. Cela permet de réduire le code nécessaire au chargement initial.

Le lazy loading n'est pas magique

Le lazy loading ajoute également une situation à gérer : la page demandée est en cours de chargement. Il faut donc prévoir un fallback :

```
<Suspense fallback={<Loading />}>  
  <ProfilePage />  
</Suspense>
```

Comme pour une API, l'utilisateur doit savoir que quelque chose est en cours.

19.10 — Optimiser GameHub

Prioriser la performance là où ça compte

L'optimisation doit être stratégique, ciblée et basée sur des mesures concrètes pour maximiser l'impact sans introduire de complexité inutile.

Pour GameHub, cela signifie:

- **Éviter l'over-optimization:** N'appliquez pas systématiquement `memo`, `useMemo` ou `useCallback` sans une raison mesurable. Chaque optimisation a un coût en complexité et en ressources.
- **Identifier les goulots d'étranglement:** Utilisez des outils de profilage pour identifier les composants ou les calculs qui sont réellement coûteux à rendre ou à recalculer.
- **Appliquer la mémorisation judicieusement:** Utilisez `useMemo` pour les calculs intensifs, `useCallback` pour les fonctions passées à des enfants mémorisés (`React.memo`), et `React.memo` pour les composants qui re-rendent inutilement.
- **Adopter le Lazy Loading pour les routes:** Pour une application comme GameHub, le lazy loading des pages (ex: Profil, Bibliothèque) peut améliorer significativement le temps de chargement initial et l'expérience utilisateur.
- **Prévoir des fallbacks:** Lors de l'utilisation du lazy loading, toujours prévoir un composant de fallback (`Suspense`) pour informer l'utilisateur du chargement en cours.

Première règle : mesurer

Avant toute optimisation de GameHub :

1. identifier une lenteur
2. reproduire le problème
3. mesurer
4. identifier la cause
5. appliquer une optimisation
6. mesurer à nouveau

Ne pas optimiser simplement parce qu'une technique existe.

Optimisations possibles dans GameHub

Rendu

- React.memo pour certains composants adaptés
- Éviter les calculs inutiles
- Structurer correctement les composants

Calculs

- useMemo pour certains calculs coûteux

Selon les problèmes réellement observés.

Chargement

- Lazy loading des pages ou fonctionnalités adaptées

Exemple GameHub : recherche

Une recherche simple peut rester très simple :

```
const results = games.filter((game) =>
  game.title
    .toLowerCase()
    .includes(query.toLowerCase())
);
```

Avec quelques dizaines ou centaines de jeux, il n'est pas nécessaire de commencer par useMemo.

Si l'application devient réellement coûteuse, on mesure d'abord. Puis on peut envisager :

```
const results = useMemo(() => {
  return games.filter((game) =>
    game.title
      .toLowerCase()
      .includes(query.toLowerCase())
  );
}, [games, query]);
```

Exemple GameHub : cartes de jeux

Si GameHub affiche un très grand nombre de cartes et qu'une interaction provoque beaucoup de rendus, on peut étudier :

```
const GameCard = memo(  
  function GameCard({  
    game,  
    onToggleFavorite,  
  }: GameCardProps) {  
    // ...  
  }  
);
```

Mais memo n'est pas automatiquement nécessaire. Il faut vérifier que cette optimisation apporte réellement quelque chose.

Performance : les bonnes priorités

01

Écrire un code simple

02

Éviter les calculs réellement inutiles

03

Réduire les données inutiles

04

Optimiser les images et ressources

05

Analyser les re-renders

06

Utiliser memo, useMemo ou useCallback si nécessaire

07

Utiliser le lazy loading lorsque le chargement initial le justifie



Exercice — Trouver l'optimisation utile

Pour chaque situation, proposer une stratégie.

1

Un calcul très coûteux est répété alors que ses entrées n'ont pas changé.

2

Une fonction passée à un composant mémorisé change inutilement de référence.

3

Une page rarement visitée alourdit fortement le chargement initial.

4

Une petite addition $\text{price} \times \text{quantity}$ est recalculée à chaque rendu.

Correction

Situation	Solution possible
Un calcul très coûteux est répété alors que ses entrées n'ont pas changé.	useMemo
Une fonction passée à un composant mémorisé change inutilement de référence.	useCallback
Une page rarement visitée alourdit fortement le chargement initial.	lazy loading
Une petite addition $\text{price} \times \text{quantity}$ est recalculée à chaque rendu.	aucune optimisation particulière

Le point important est le mot possible. Une technique d'optimisation doit être choisie en fonction du problème réel.

À retenir — Partie 19

Re-renders normaux: Les re-renders sont un fonctionnement normal de React. Ils deviennent un problème lorsqu'ils entraînent un travail réellement coûteux.

Mesurer d'abord: mesurer avant d'optimiser

React.memo: concerne les composants et leurs props

useMemo: mémorise une valeur calculée

useCallback: mémorise une fonction

Coût des outils: ces outils ont un coût et ajoutent de la complexité

Lazy loading: permet de différer certains chargements

Pas universel: toutes les applications n'ont pas besoin de toutes ces optimisations

La meilleure optimisation n'est pas celle qui utilise le plus de Hooks : c'est celle qui résout un problème réel sans compliquer inutilement le code.

Partie 20 — Tests

1

Tests unitaires

2

Tests d'intégration

3

React Testing Library

4

Vitest

5

Scénarios utilisateur



20.1 – Pourquoi tester une interface ?

Pourquoi tester ?

Une application peut fonctionner correctement aujourd'hui et être cassée demain après une modification.

1

Modification du bouton Favoris

2

La fonctionnalité Favoris fonctionne

3

Mais la recherche ne fonctionne plus

Les tests permettent de détecter automatiquement certains problèmes.
Un test vérifie qu'un comportement attendu fonctionne réellement.

Que veut-on tester ?

Pour une interface, on peut vérifier :

- qu'un élément est affiché
- qu'un texte apparaît
- qu'un bouton fonctionne
- qu'un formulaire réagit correctement
- qu'un état change
- qu'une erreur est affichée
- qu'une action utilisateur produit le résultat attendu

On teste donc principalement ce que l'utilisateur peut observer ou provoquer.

Un test comme scénario utilisateur

Préparer

Une page GameHub est affichée.

Agir

L'utilisateur clique sur « Ajouter aux favoris ».

Vérifier

Le jeu apparaît dans les favoris.

Cette approche permet de relier directement le test à une fonctionnalité.

ete

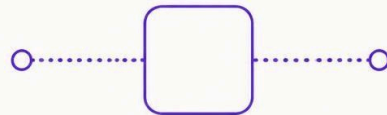
re



20.2 — Tests unitaires vs tests d'intégration

Tests unitaires

les tests unitaires se concentrent sur des composants individuels isolés



Tests d'intégration

les tests d'intégration vérifient les interactions entre plusieurs composants.



Test unitaire

Un test unitaire vérifie une petite unité de comportement de manière isolée.

```
function formatScore(score: number) {  
  return `${score}/5`;  
}
```

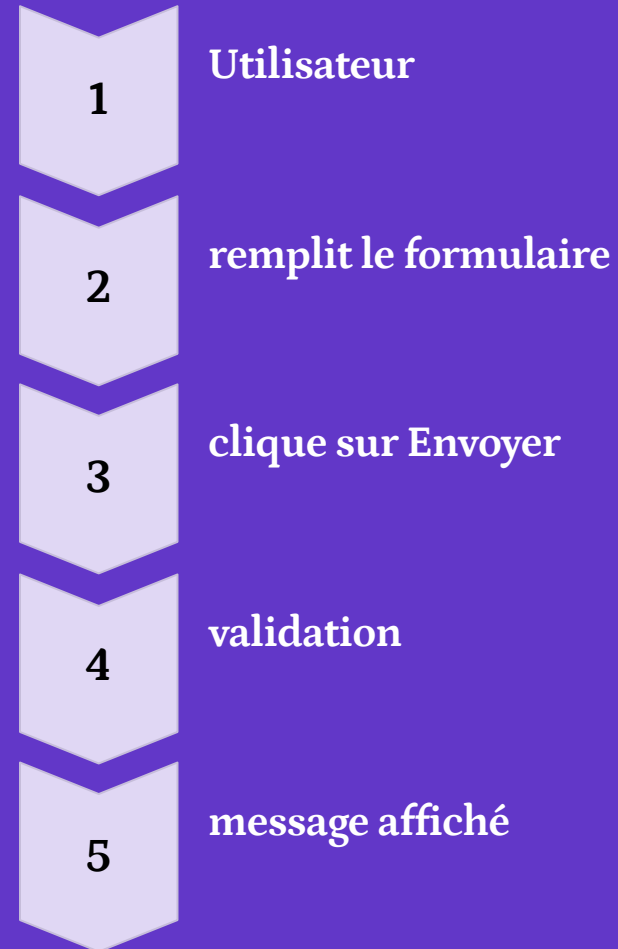
On peut tester : `formatScore(4)` → `'4/5'`

L'objectif est de vérifier une petite partie du programme.

Test d'intégration

Un test d'intégration vérifie plusieurs éléments qui fonctionnent ensemble.

Exemple GameHub — flux complet :



On teste alors davantage le comportement réel de l'interface.

Comparaison

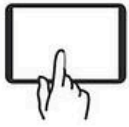
Test unitaire	Test d'intégration
petite unité	plusieurs éléments
très ciblé	comportement plus complet
rapide	généralement plus coûteux
logique isolée	composants + interactions

Dans une application React, les deux approches peuvent être utiles.

R APP DEVELOPMENT VOL 2



SMARTPHONE



TABLET



INCOME



WEARABLE



AUGMENTED REALITY



3D MODEL



SIGN-OFF



REWORK REQUIRED



PROTOCOL



MOBILE SECURITY



CLOUD INFRASTRUCTURE



PERFORMANCE

20.3 — Présentation de l'outillage

Pour tester efficacement une interface utilisateur, une variété d'outils et de frameworks sont disponibles. Cette section présentera les principaux outils que nous allons utiliser pour écrire des tests pour notre application React.

Quels outils pour React ?

Vitest

exécute les tests

React Testing Library

aide à tester l'interface

jsdom

fournit un DOM simulé

Le rôle des outils est différent.

Installer l'outillage

Dans un projet Vite, l'installation dépend de la configuration choisie.

```
npm install -D vitest jsdom  
npm install -D @testing-library/react @testing-library/jest-dom
```

L'objectif pédagogique n'est pas de mémoriser les commandes. Il faut comprendre quel outil répond à quel besoin.

Structure d'un test

Un test peut suivre cette structure :

```
describe("GameCard", () => {  
  it("affiche le titre du jeu", () => {  
    // préparation  
    // action  
    // vérification  
  });  
});
```

On retrouve souvent :

- describe : groupe de tests
- it : décrit un comportement
- expect : vérifie un résultat

```
self.app.get(BASE_URL)
assertEqual(resp.status_code, status.HTTP_200_OK)
resp.get_json()
assertEqual(len(data), 5)

def test_order(self):
    """a single Order"""
    """the id of a order"""
    order = self._create_orders(1)[0]
    self.app.get(
        f"/orders/{order.id}", content_type=0

    assertEqual(resp.status_code, status.HTTP_200_OK)
    resp.get_json()
    assertEqual(data["name"], test_order.name)

def test_order_not_found(self):
    """a Order thats not found"""
    self.app.get("/orders/0")
    assertEqual(resp.status_code, status.HTTP_404_NOT_FOUND)

def create_order(self):
    """create a new Order"""
    order = OrderFactory()
    debug(test_order)
    self.app.post(
        BASE_URL, json=test_order.serialize(), content_type=0

    assertEqual(resp.status_code, status.HTTP_201_CREATED)
    """order is set"""
    """None"""
```

20.4 — Tester un composant

Tester un composant React implique de le rendre dans un environnement de test, puis d'interagir avec lui et de vérifier son comportement attendu. Cela nous permet de nous assurer que le composant fonctionne correctement, quelle que soit la manière dont il est utilisé dans l'application.

L'objectif est de simuler les actions de l'utilisateur et d'observer les réactions du composant, en se basant sur ce qu'il affiche et comment il réagit aux événements.

Exemple simple de test de composant :

```
import { render, screen } from
  '@testing-library/react';
import MyComponent from
  './MyComponent';

describe('MyComponent', () => {
  it('affiche le texte correct', () => {
    render(<MyComponent
      text="Bonjour le monde" />);

    expect(screen.getByText(/Bonjour
      le
      monde/i)).toBeInTheDocument();
  });
});
```

Dans cet exemple, on s'assure que le composant affiche bien le texte qu'on lui passe via ses props.

Premier composant à tester

Imaginons :

```
function GameCard({ game }: GameCardProps) {  
  return (  

```

{game.title}

```
); }
```

Nous voulons vérifier : Le titre du jeu est affiché.

Rendre le composant dans le test

```
render(  
  <GameCard  
    game={{  
      id: 1,  
      title: "Hades",  
    }}  
  />  
);
```

render permet de rendre le composant dans l'environnement de test.
Le test peut ensuite rechercher ce que l'utilisateur pourrait voir.

Vérifier le contenu

```
expect(  
  screen.getByText("Hades")  
)toBeInTheDocument();
```

Le test vérifie que : GameCard → affiche « Hades ». On ne vérifie pas la structure interne de React. On vérifie le résultat observable.

20.5 — Tester une interaction

Tester un bouton

GameCard possède maintenant un bouton :

```
<button onClick={onToggleFavorite}>  
  Favoris  
</button>
```

Nous voulons vérifier que le clic déclenche bien l'action attendue.

Simuler une interaction

Avec Testing Library, on peut utiliser `userEvent`.

```
const user = userEvent.setup();

await user.click(
  screen.getByRole("button", {
    name: "Favoris",
  })
);
```

Le test reproduit une action utilisateur.

Vérifier l'appel

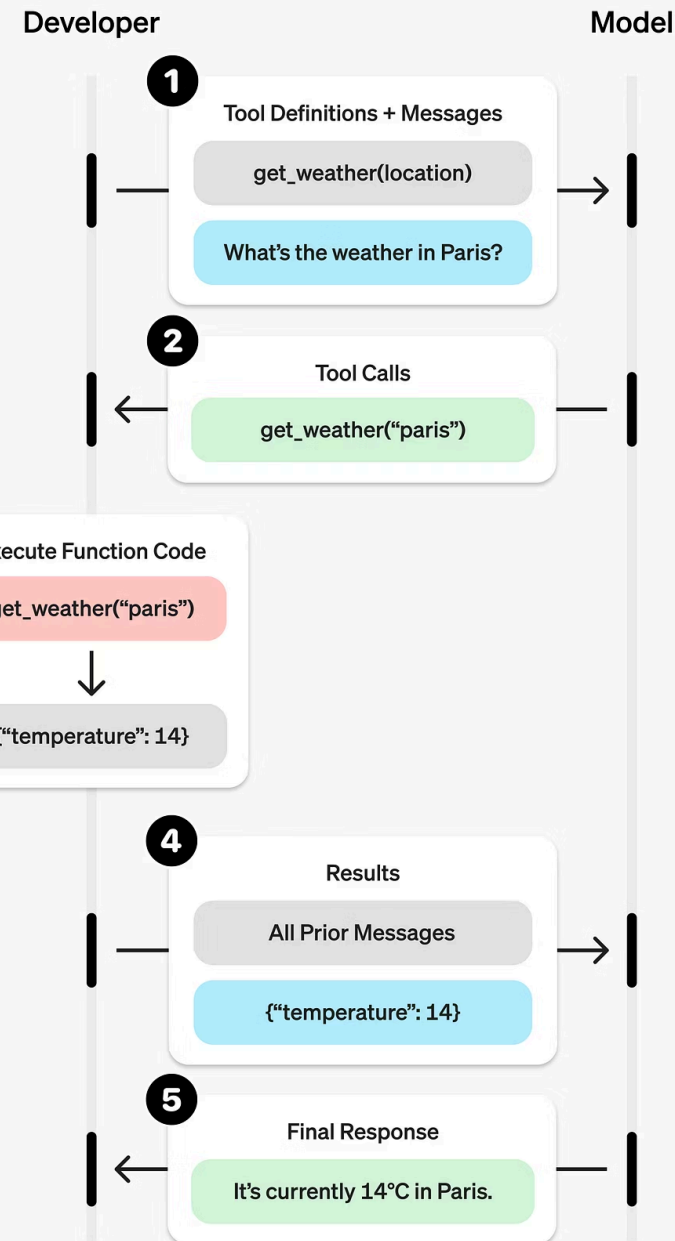
On peut fournir une fonction espion :

```
const onToggleFavorite = vi.fn();
```

Puis :

```
expect(onToggleFavorite)  
  .toHaveBeenCalled();
```

Le test vérifie donc : Quand l'utilisateur clique, la fonction attendue est appelée.



TION

+



20.6 — Tester un formulaire

Que tester dans un formulaire ?

Pour le profil GameHub, on peut vérifier :

les champs sont présents

l'utilisateur peut les remplir

la validation fonctionne

le formulaire peut être envoyé

les erreurs apparaissent

les données valides sont acceptées

Tester la saisie

Exemple :

```
const user = userEvent.setup();

await user.type(
  screen.getByLabelText("Nom"),
  "Alice"
);
```

Puis :

```
expect(
  screen.getByLabelText("Nom")
).toHaveValue("Alice");
```

On vérifie le comportement du champ, pas son implémentation interne.

Tester la validation

Si le nom est obligatoire :

```
await user.click(
  screen.getByRole("button", {
    name: "Enregistrer",
  })
);
```

Puis :

```
expect(
  screen.getByText("Le nom est
obligatoire.")
).toBeInTheDocument();
```

Le test vérifie le comportement attendu pour une donnée invalide.

First name (required)

Rodriguez



Email (required)



Please fill out this field.

Purchase Details (required)

Purchase date (must be within the last year)

mm/dd/yyyy



20.7 — Tester un état

Tester l'état d'un composant React consiste à vérifier que son interface utilisateur reflète correctement les modifications de son état interne. Cela est crucial pour les composants interactifs qui maintiennent des données ou des configurations au fil du temps, comme un compteur, un interrupteur ou un formulaire.

L'objectif est de simuler des actions utilisateur qui modifient l'état du composant et de s'assurer que le rendu visuel du composant est mis à jour comme prévu. On se concentre sur le comportement observable plutôt que sur l'implémentation interne de la gestion de l'état (par exemple, comment les hooks sont utilisés).

Exemple : Tester un compteur

Considérons un composant simple qui affiche un nombre et un bouton pour l'incrémenter.

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Counter from './Counter';
```

```
describe('Counter', () => {
  it('incrémente le compteur lors du clic', async () => {
    render(<Counter />);
    const user = userEvent.setup();
```

```
    const incrementButton =
      screen.getByRole('button', { name:
        /incrémenter/i });
    const countDisplay =
      screen.getByText('Count: 0');
```

```
    expect(countDisplay).toBeInTheDocument();
```

```
    await
      user.click(incrementButton);
```

```
    expect(screen.getByText('Count:
      1')).toBeInTheDocument();
  });
});
```

Dans cet exemple, nous vérifions que le texte affichant le compte passe de "Count: 0" à "Count: 1" après avoir cliqué sur le bouton.

Tester un changement d'état

Un composant peut avoir :

```
const [favorite, setFavorite] =  
  useState(false);
```

Au départ : Ajouter aux favoris

Après un clic : Retirer des favoris

Le test peut vérifier cette évolution.

Tester le comportement plutôt que le state

On ne cherche pas forcément à vérifier directement : `favorite === true`.
On peut plutôt vérifier ce que voit l'utilisateur :

```
await user.click(
  screen.getByRole("button", {
    name: "Ajouter aux favoris",
  })
);

expect(
  screen.getByRole("button", {
    name: "Retirer des favoris",
  })
).toBeInTheDocument();
```

Le test reste centré sur l'interface.

```
e>
_a_server()
/myproject/some_other_file.py", line 7, in some_
/myproject/server.py", line 12, in start_server
/myproject/venv/lib64/python3.7/site-packages/bo
/myproject/venv/lib64/python3.7/site-packages/bo
f.port, app, server_cls, handler_cls)
/simple_server.py", line 153, in make_server
t), handler_class)
server.py", line 449, in __init__
/simple_server.py", line 50, in server_bind
server.py", line 137, in server_bind
nd(self)
server.py", line 463, in server_bind
ress)
in use
```

20.8 — Tester les cas d'erreur

Tester uniquement le succès est insuffisant

Pour GameHub, il faut aussi tester :

- API indisponible
- formulaire invalide
- aucun résultat
- erreur de chargement
- données absentes

Une fonctionnalité robuste doit fonctionner dans les situations normales et prévues comme problématiques.

Tester un état d'erreur

Supposons que la page affiche :

```
<p>Impossible de charger les jeux.</p>
```

Le test doit vérifier que ce message apparaît lorsque le chargement échoue.

```
expect(  
  screen.getByText(  
    "Impossible de charger les jeux."  
  )  
)toBeInTheDocument();
```

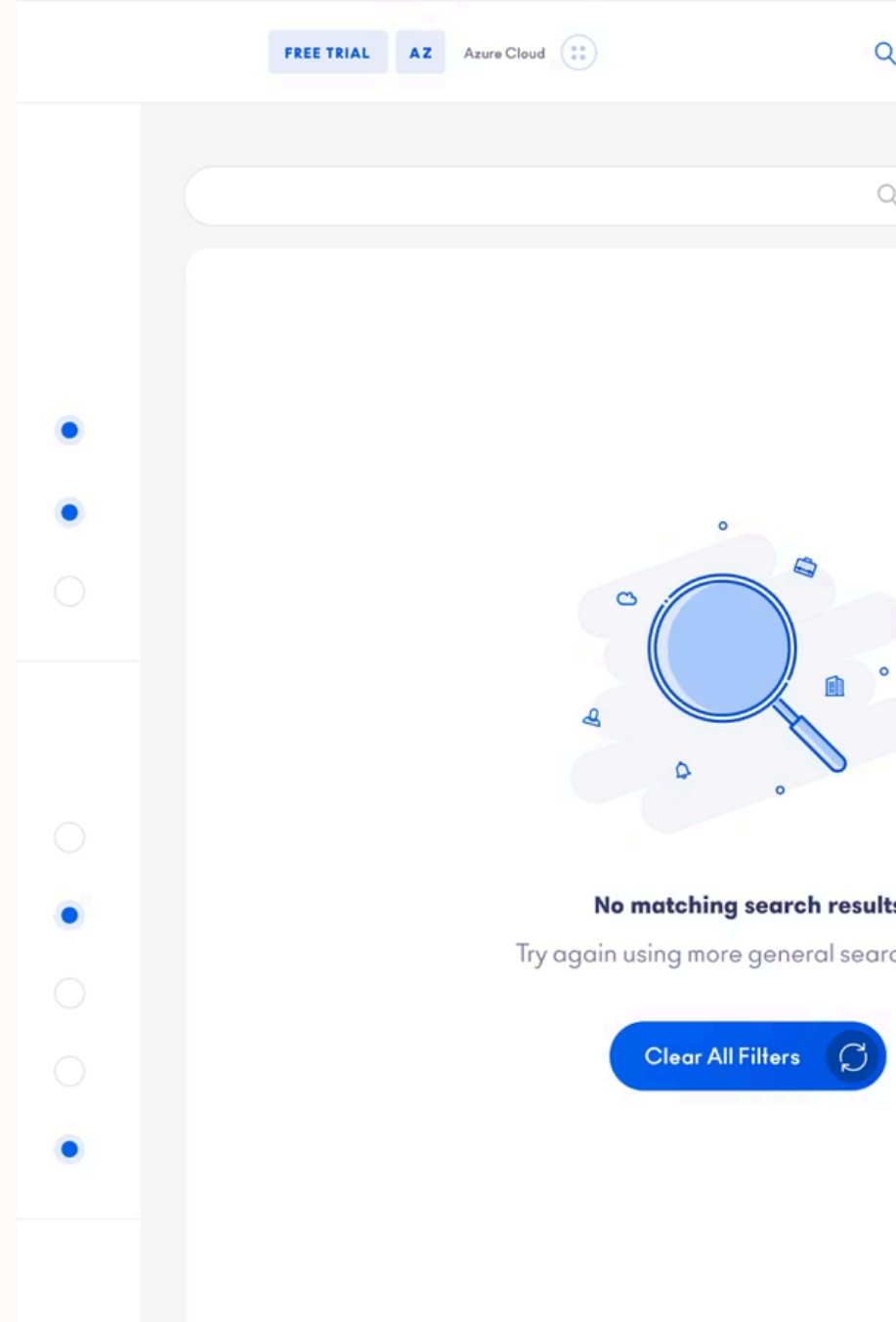
On vérifie ce que l'utilisateur voit en cas d'erreur.

Tester un état vide

Pour une recherche sans résultat :

```
expect(  
  screen.getByText(  
    "Aucun jeu trouvé."  
  )  
)  
  .toBeInTheDocument();
```

Ce test garantit que l'interface ne reste pas simplement vide.



Tester loading, error et success

Situation	Ce que le test vérifie
Loading	indicateur affiché
Success	jeux affichés
Error	message + action possible
Empty	message adapté

Ces tests correspondent directement aux états fonctionnels vus dans la partie précédente.



20.9 — Tests de GameHub

Quelles fonctionnalités tester ?

Pour GameHub, on peut sélectionner les fonctionnalités importantes :

GameCard

- titre affiché
- plateforme affichée
- bouton favori fonctionnel

Recherche

- saisie d'une recherche
- résultats filtrés
- aucun résultat

Formulaire

- saisie
- validation
- soumission
- erreur

API

- chargement
- succès
- erreur

Exemple de scénario GameHub

Scénario : Ajouter un jeu aux favoris.

01

Afficher GameCard

02

Vérifier le bouton

03

Cliquer

04

Vérifier le nouvel état

L'objectif est de reproduire une interaction réelle.

Exemple de scénario : recherche

Scénario : Rechercher « Zelda ».

01

Afficher plusieurs jeux

02

Saisir « Zelda »

03

Vérifier les résultats

04

**Vérifier qu'un jeu sans
correspondance disparaît**

Ce test vérifie le comportement de la
fonctionnalité de recherche.



Exemple de scénario : formulaire

Scénario : Modifier son profil.

01

Afficher le formulaire

02

Remplir le nom

03

Remplir l'email

04

Soumettre

05

Vérifier le résultat

On peut également ajouter un scénario avec des données invalides.



Que faut-il réellement tester ?

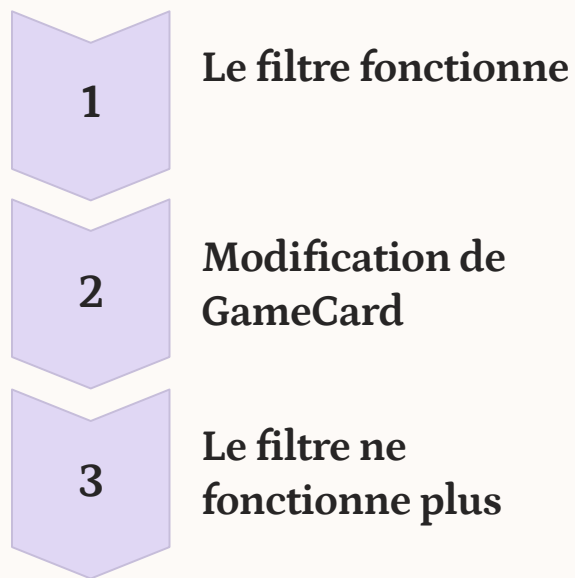
Il n'est pas nécessaire de tester absolument tout. Prioriser :

1. fonctionnalités importantes
2. comportements complexes
3. logique susceptible de casser
4. cas d'erreur
5. fonctionnalités utilisées fréquemment

Un test doit apporter une valeur de sécurité au projet.

Tests et régression

Une régression correspond à une fonctionnalité qui fonctionnait auparavant mais qui ne fonctionne plus après une modification.



Les tests automatisés permettent de détecter ce type de problème plus rapidement.

SHIFT LEFT TESTING

In the traditional SDLC, testing happens towards the end, which means:

- ✗ Bugs are found late
- ✗ Fixing them costs more time + money
- ✗ Releases get delayed



SHIFT LEFT

testing moves earlier in the development cycle:

- ✓ QA collaborates with devs from day 1
- ✓ Unit tests + API tests catch issues before UI testing
- ✓ Defects are fixed when they're cheapest to fix

Curious to hear:

Has your team adopted **SHIFT LEFT TESTING** yet?
If yes, what changes did you see in your delivery process ?



Key Benefits:

- 1 Faster releases without compromising quality
- 2 Reduced cost of bug fixes
- 3 Stronger collaboration between Dev, QA, and Product
- 4 Improved confidence in every build

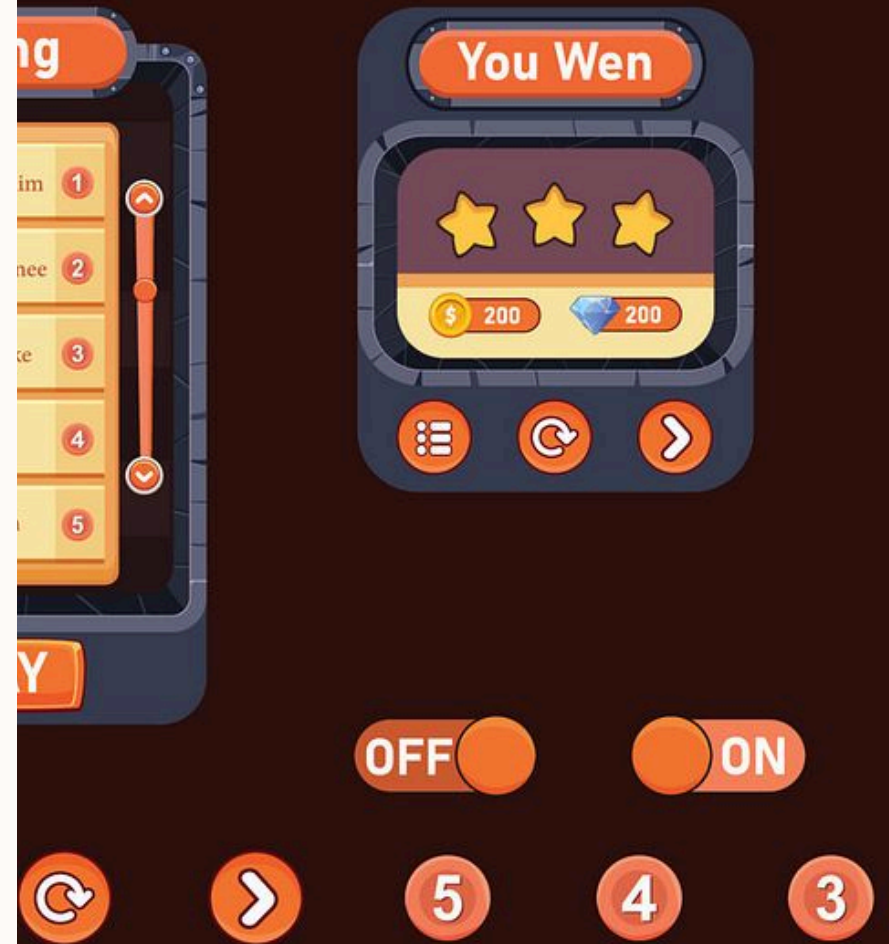


Exercice — Tester GameCard

Écrire un test qui vérifie :

1. le titre du jeu est affiché
2. le bouton « Ajouter aux favoris » est présent
3. un clic appelle onToggleFavorite

Puis ajouter un test vérifiant l'état après modification.



Correction — idées essentielles

```
render(  
  <GameCard  
    game={game}  
    onToggleFavorite={onToggleFavorite}  
  />  
);
```

Puis rechercher le contenu visible :

```
screen.getByText(game.title);
```

Simuler l'action :

```
await user.click(  
  screen.getByRole("button", {  
    name: "Ajouter aux favoris",  
  })  
);
```

Puis vérifier :

```
expect(onToggleFavorite)  
  .toHaveBeenCalled();
```

À retenir — Partie 20

Comportement: tester un comportement plutôt qu'une implémentation

Types de tests: distinguer tests unitaires et tests d'intégration

Interactions: tester les interactions utilisateur

Formulaires: tester les formulaires

États: tester les différents états

Erreurs: tester les erreurs

Priorités: prioriser les fonctionnalités importantes

Régressions: utiliser les tests pour prévenir les régressions

Un bon test répond à une question simple : « Si quelqu'un modifie le code demain, comment puis-je vérifier automatiquement que cette fonctionnalité fonctionne toujours ? »

Partie 21 — Architecture et bonnes pratiques

Organisation du projet

Séparation des responsabilités

Réutilisabilité

Nommage

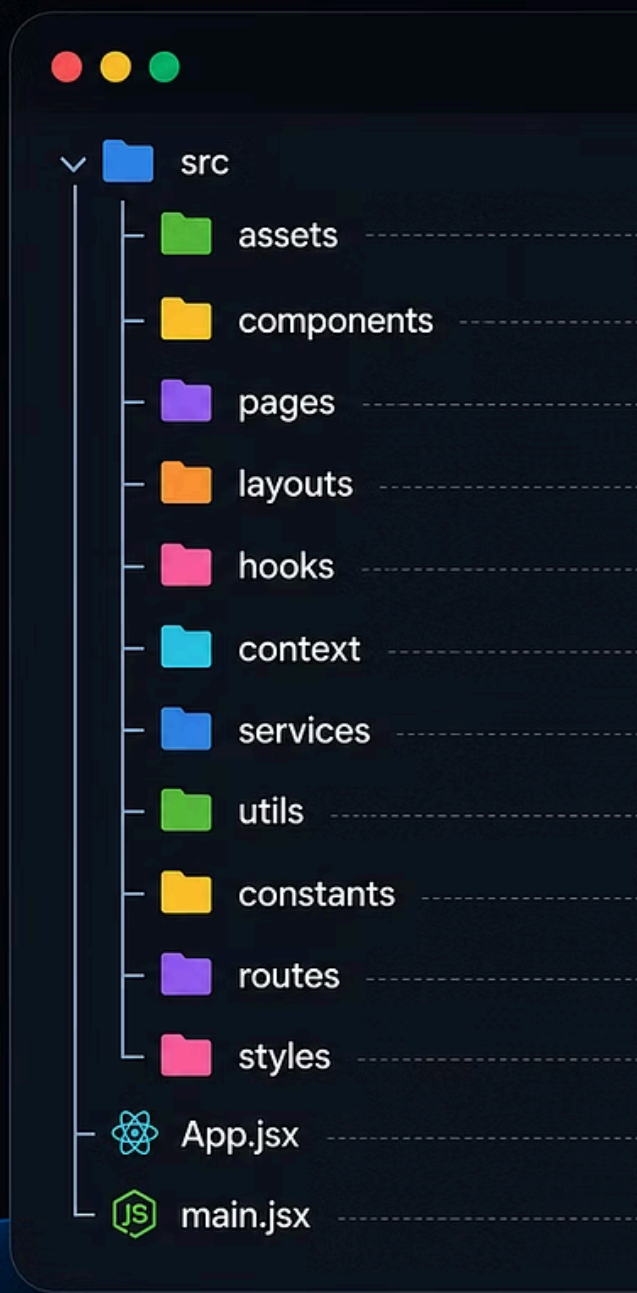
Typage

Variables d'environnement

URE
ERS



AVAILABLE
OBJECTS



21.1 — Organiser son projet React

Pourquoi organiser son projet ?

Au début, une application React peut tenir dans quelques fichiers. Puis le projet grandit :

- davantage de composants
- plusieurs pages
- des Hooks
- des appels API
- des types
- des formulaires
- du contexte

Sans organisation, il devient difficile de savoir : « Où dois-je mettre ce code ? » Une bonne organisation doit rendre le projet facile à parcourir et à faire évoluer.

Une organisation possible

Pour GameHub :

```
src/  
├── components/  
├── pages/  
├── hooks/  
├── context/  
├── services/  
├── types/  
├── App.tsx  
└── main.tsx
```

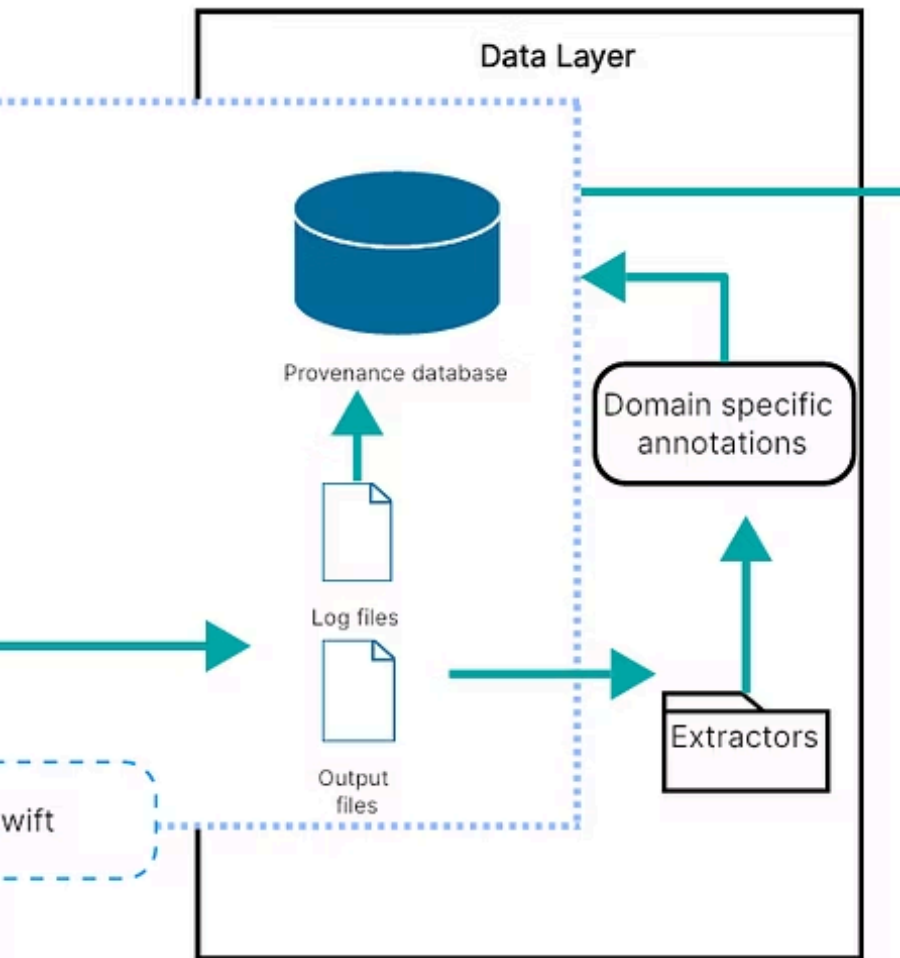
Ce n'est pas une règle obligatoire. C'est une structure possible pour séparer les différentes responsabilités du projet.

Le rôle des dossiers

Dossier	Rôle
components/	composants réutilisables
pages/	écrans/pages de l'application
hooks/	logique réutilisable sous forme de Hooks
context/	Context React
services/	communication avec des services externes
types/	types TypeScript

L'objectif est de pouvoir rapidement retrouver une fonctionnalité.

Conceptual Architecture Diagram



**21.2 – Components / Pages /
Hooks / Context / Services**

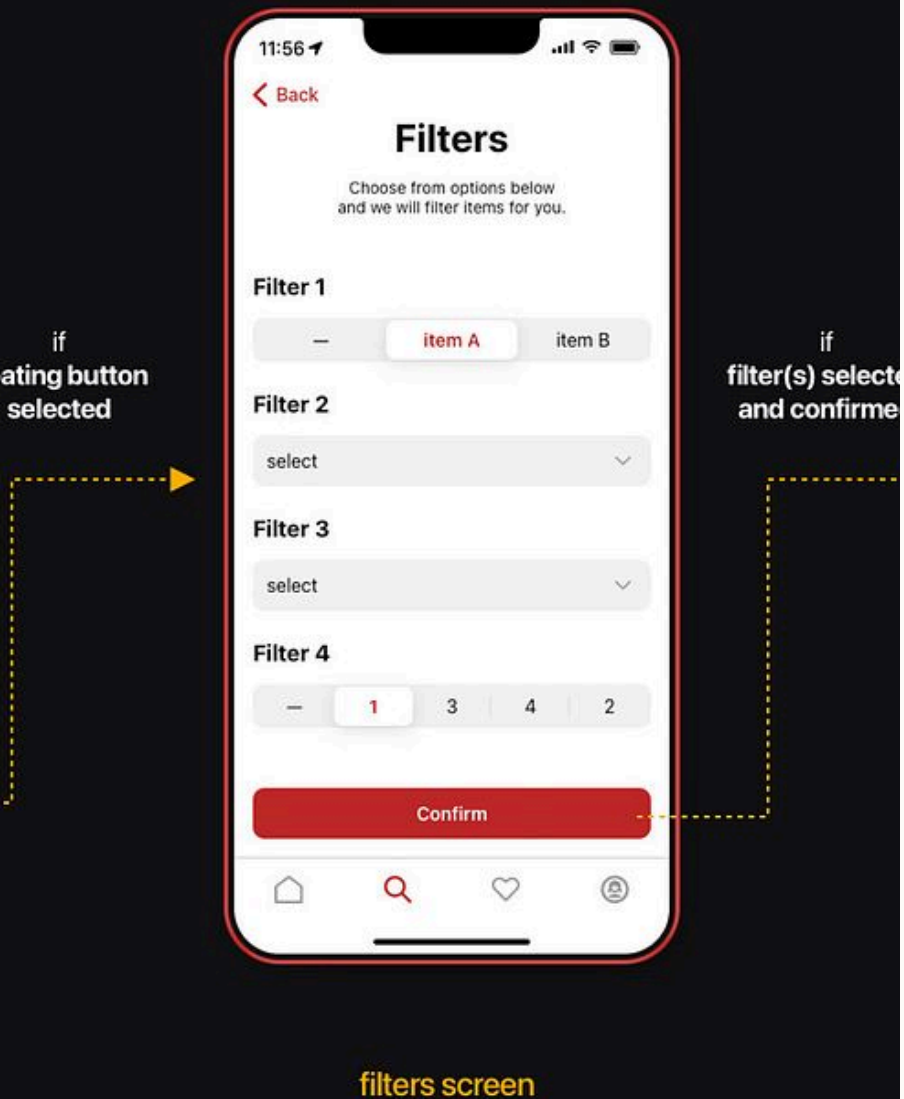
Components

Un composant représente généralement une partie de l'interface.

Exemples GameHub : NavBar / GameCard / GameGrid / SearchBar /
RatingForm / Button

```
function GameCard({ game }: GameCardProps) {  
  return (  
    <article>  
      <h2>{game.title}</h2>  
    </article>  
  );  
}
```

Un composant doit rester compréhensible et avoir une responsabilité claire.



Pages

Une page représente généralement un écran accessible dans l'application.
Exemples : HomePage / GamesPage / GameDetailsPage / FavoritesPage / LibraryPage / ProfilePage

Une page peut composer plusieurs composants :

```
function GamesPage() {  
  return (  
    <>  
    <SearchBar />  
    <GameGrid />  
  </>  
);  
}
```

La page orchestre l'écran.

Hooks

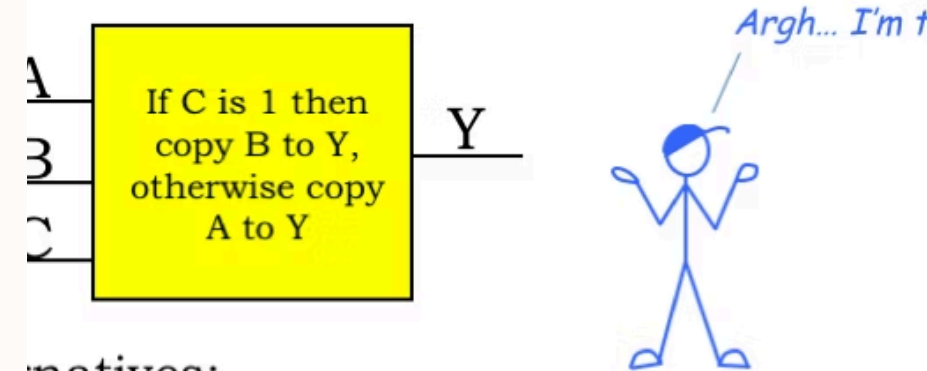
Les Hooks encapsulent une logique réutilisable. Exemples : `useGames()` / `useFetch()` / `useDebounce()`

Un Hook n'est pas un composant. Il ne retourne généralement pas directement du JSX.

```
const {
  data,
  loading,
  error,
} = useGames();
```

Functional Specifications

many ways of specifying the function of a physical device, for example:



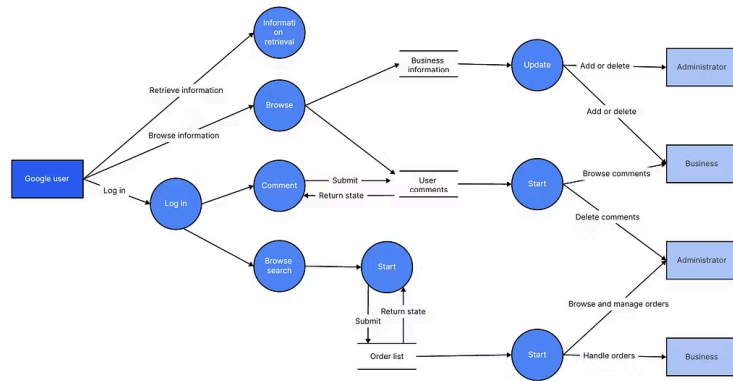
alternatives:

are a concise description of the physical system's function.

expressions form an algebra whose operations are AND (multiplication), OR (addition), and inversion (overbar).

$$Y = \bar{C} \cdot \bar{B} \cdot A$$

any Boolean function can be specified by a truth table or an equivalent sum-of-products expression!



Context

Le Context permet de rendre une information disponible à plusieurs composants sans transmettre systématiquement les props à travers toute la hiérarchie.

Exemples possibles dans GameHub :

- utilisateur connecté
- thème
- préférences globales
- état partagé

```
const user =
useContext(UserContext);
```

Le Context doit cependant être utilisé lorsqu'il répond réellement à un besoin de partage.

Services

Un service peut centraliser la communication avec une API ou un système externe.

```
export async function getGames():  
  Promise<Game[]> {  
    const response = await  
    fetch("/api/games");  
  
    if (!response.ok) {  
      throw new Error("Erreur API");  
    }  
  
    return response.json();  
  }
```

La page n'a alors pas besoin de connaître tous les détails de la requête HTTP.

Qui fait quoi ?

Élément	Responsabilité principale
Component	interface
Page	composition d'un écran
Hook	logique réutilisable
Context	données partagées
Service	communication externe
Type	description des données

Cette séparation permet de réduire le mélange des responsabilités.



21.3 — Séparation des responsabilités

Éviter de tout mettre au même endroit

Un composant comme celui-ci devient rapidement difficile à maintenir :

```
function GamesPage() {  
  // fetch  
  // validation  
  // transformation des données  
  // gestion des favoris  
  // recherche  
  // tri  
  // affichage  
  // formulaire  
}
```

Le problème n'est pas le nombre exact de lignes. Le problème est le nombre de responsabilités différentes.

Une responsabilité claire

Logique

```
const { games } = useGames();
```

Transformation

```
const results = filterGames(games, query);
```

Affichage

```
<GameGrid games={results} />
```

Chaque partie a un rôle plus précis.

La question à se poser

Lorsqu'un composant devient compliqué : « Est-ce que cette logique appartient réellement à ce composant ? »

On peut alors identifier :

- une logique réutilisable → Hook
- une communication API → service
- une interface répétée → composant
- une donnée réellement globale → Context
- une transformation simple → fonction



21.4 — Réutilisabilité

Pourquoi réutiliser ?

Supposons plusieurs boutons similaires. ✗ Copier le même JSX partout :

```
<button className="...">Ajouter</button>
```

Cela crée de la duplication.

On peut créer :

```
<Button>Ajouter</Button>
```

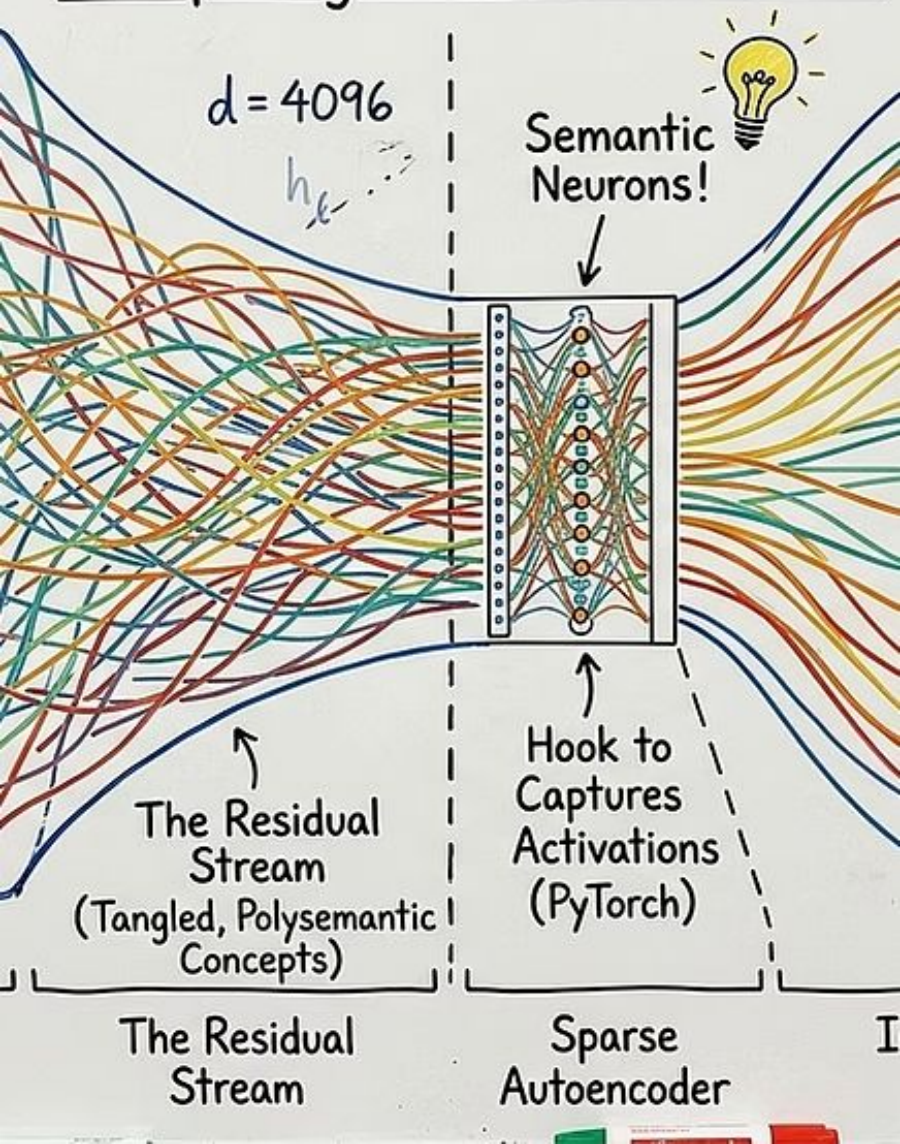
Un composant réutilisable

```
interface ButtonProps {  
  children: React.ReactNode;  
  onClick: () => void;  
}  
  
function Button({  
  children,  
  onClick,  
}: ButtonProps) {  
  return (  
    <button onClick={onClick}>  
      {children}  
    </button>  
  );  
}
```

Le composant peut maintenant être utilisé dans plusieurs contextes.

REVERSE ENGINEERING CL

Interpreting the Residual Stream



Attention à la sur-abstraction

Il ne faut pas créer un composant générique pour chaque élément. Par exemple, créer : `UniversalGameHubButtonManager` pour encapsuler un simple bouton serait excessif.

Une abstraction est utile lorsqu'elle :

- évite une vraie duplication
- simplifie le code
- possède une responsabilité claire
- peut évoluer indépendamment

e Naming tions

Side

21.5 — Nommage

Un bon nom explique le rôle

Préférer :

GameCard / GameDetailsPage / useGames / getGames / UserContext

à :

Card1 / Page2 / helper / data / thing

Un bon nom réduit le besoin de commentaires.

Conventions de nommage

Composants

GameCard.tsx

GamesPage.tsx

Hooks

useGames.ts

useFetch.ts

Services

gameService.ts

userService.ts

Types

game.ts

user.ts

L'important est surtout d'être cohérent dans tout le projet.

```
class ShowPicture {  
    public static void main(String[] args) {  
        main(String[])  
    }  
}  
  
public static void main(String[] args) {  
    JFrame frame = new JFrame();  
    ImageIcon icon = new ImageIcon("resources/jetbrains-logo.png");  
    JLabel label = new JLabel(icon);  
    frame.add(label);  
    frame.setDefaultCloseOperation  
        (JFrame.EXIT_ON_CLOSE);  
    frame.pack();  
    frame.setVisible(true);  
}
```

21.6 — Typage

TypeScript doit aider l'architecture

Les types doivent décrire les données importantes de l'application.

```
interface Game {  
  id: number;  
  title: string;  
  platform: Platform;  
  rating: number;  
}
```

On peut ensuite réutiliser Game dans :

- les composants
- les Hooks
- les services
- les pages

Éviter any

À éviter :

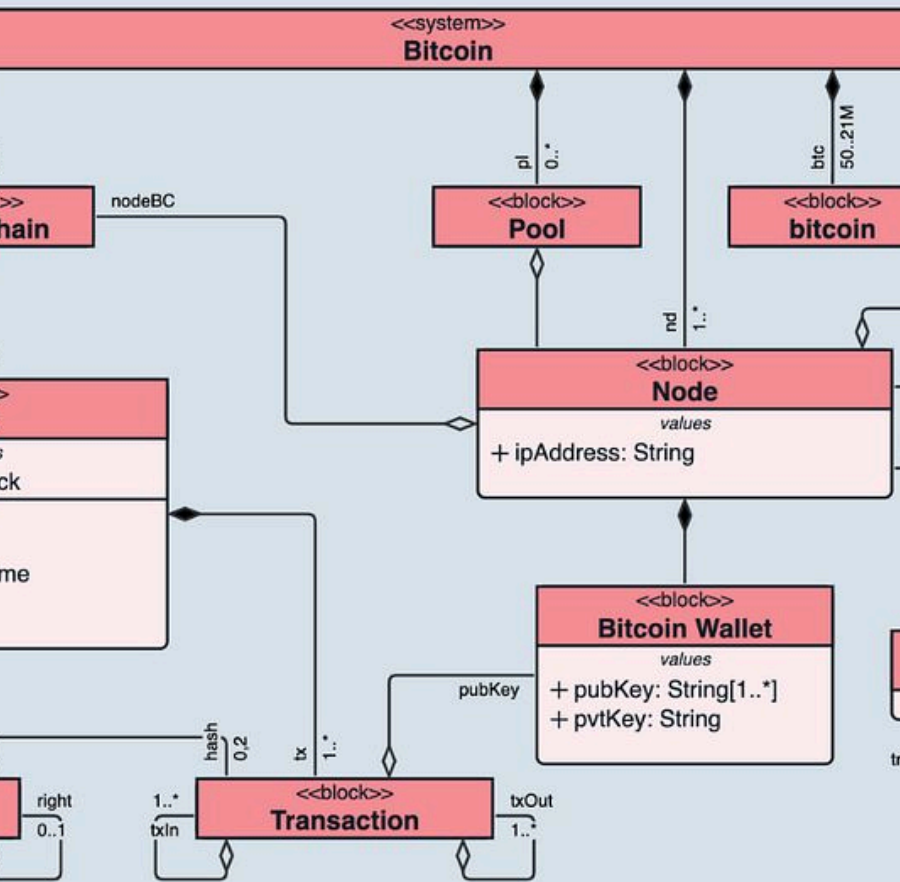
```
function GameCard(game: any) {  
  // ...  
}
```

On perd une grande partie de l'intérêt de TypeScript.

Préférer :

```
function GameCard({  
  game,  
}: {  
  game: Game;  
}) {  
  // ...  
}
```

Le type permet à l'éditeur et au compilateur de détecter davantage d'erreurs.



Typing les frontières importantes

Certaines frontières méritent particulièrement d'être typées :

- API → application
- Page → composant
- Composant → composant
- Hook → composant

Exemple :

```
async function getGames():  
  Promise<Game[]> {  
    // ...  
  }
```

Le reste du code sait immédiatement ce que le service doit retourner.



21.7 — Éviter les composants gigantesques

Comment reconnaître un composant trop gros ?

Signaux possibles :

- beaucoup de state
- beaucoup de fonctions
- plusieurs formulaires
- plusieurs appels API
- beaucoup de JSX
- plusieurs responsabilités
- modifications fréquentes de parties sans rapport

Il n'existe pas de nombre magique de lignes. La question principale est : est-ce encore facile à comprendre ?

Extraire progressivement

Au lieu de réécrire toute l'application, on peut commencer par extraire :

1

`<GameGrid />`

2

`<SearchBar />`

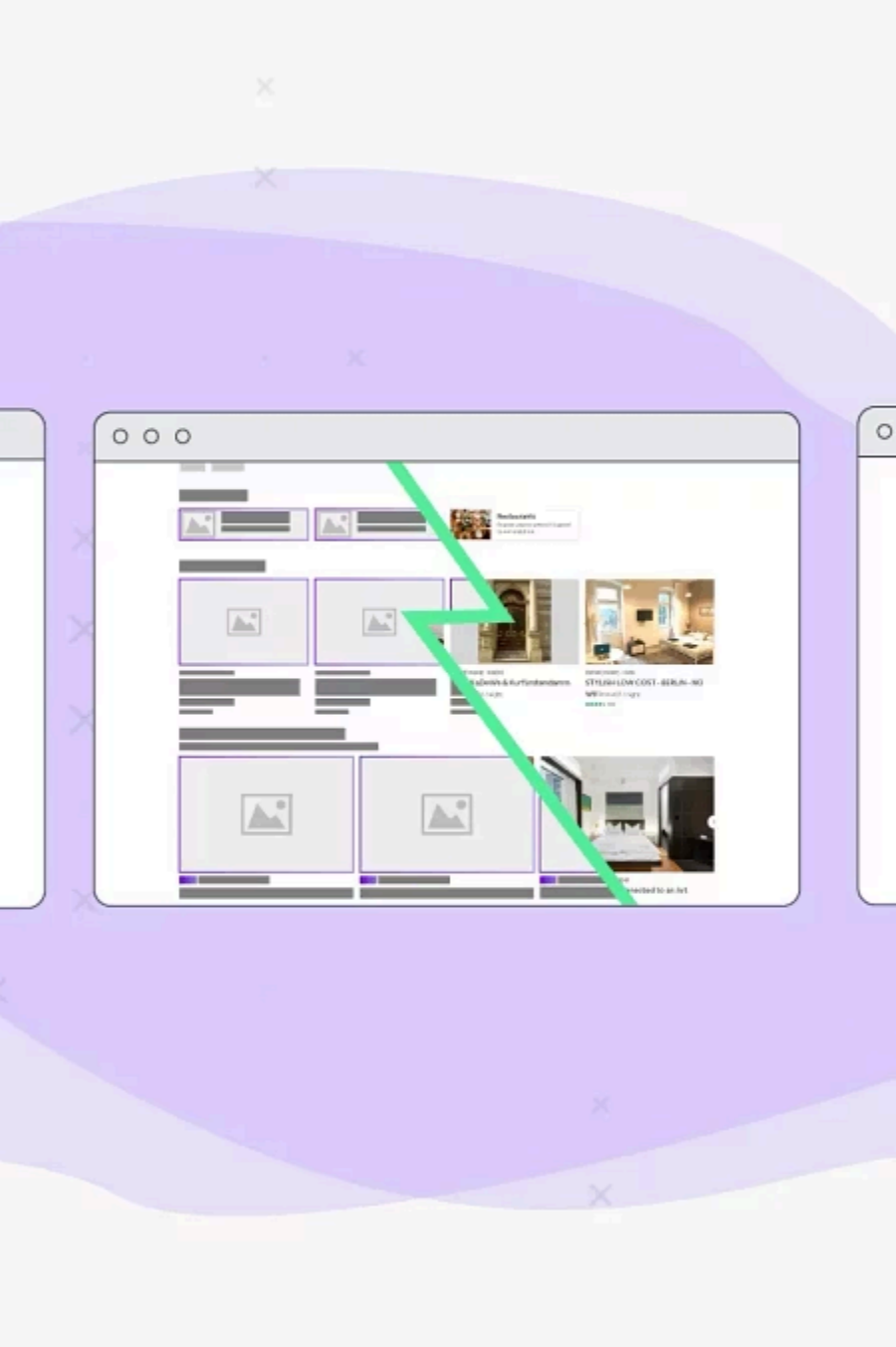
3

`const { games } = useGames();`

4

`getGames();`

L'architecture peut évoluer progressivement.



21.8 — Éviter la duplication

Identifier ce qui doit être partagé

Si une logique est réellement identique et réutilisable, on peut l'extraire.

```
function formatGameRating(rating: number) {  
  return `${rating.toFixed(1)}/5`;  
}
```

Puis utiliser cette fonction partout où c'est nécessaire.

Autres possibilités :

- composant
- Hook
- fonction utilitaire
- service

Ne pas supprimer toute répétition

Deux morceaux de code peuvent se ressembler sans devoir être fusionnés. Une abstraction trop précoce peut être plus difficile à comprendre que deux petits morceaux indépendants.

Éviter la duplication, mais éviter aussi la sur-abstraction.

All projects



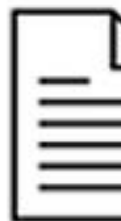
Year
2021



Month
06_June



Da
26



21.9 — Gestion des fichiers

Exemple d'organisation GameHub

src/

```
|— components/
|   |— GameCard.tsx
|   |— GameGrid.tsx
|   |— SearchBar.tsx
|
|— pages/
|   |— HomePage.tsx
|   |— GamesPage.tsx
|   |— ProfilePage.tsx
|
|— hooks/
|   |— useGames.ts
|   |— useFetch.ts
|
|— services/
|   |— gameService.ts
|
|— types/
|   |— game.ts
```

Cette organisation permet de retrouver rapidement les fichiers.

Une organisation peut évoluer

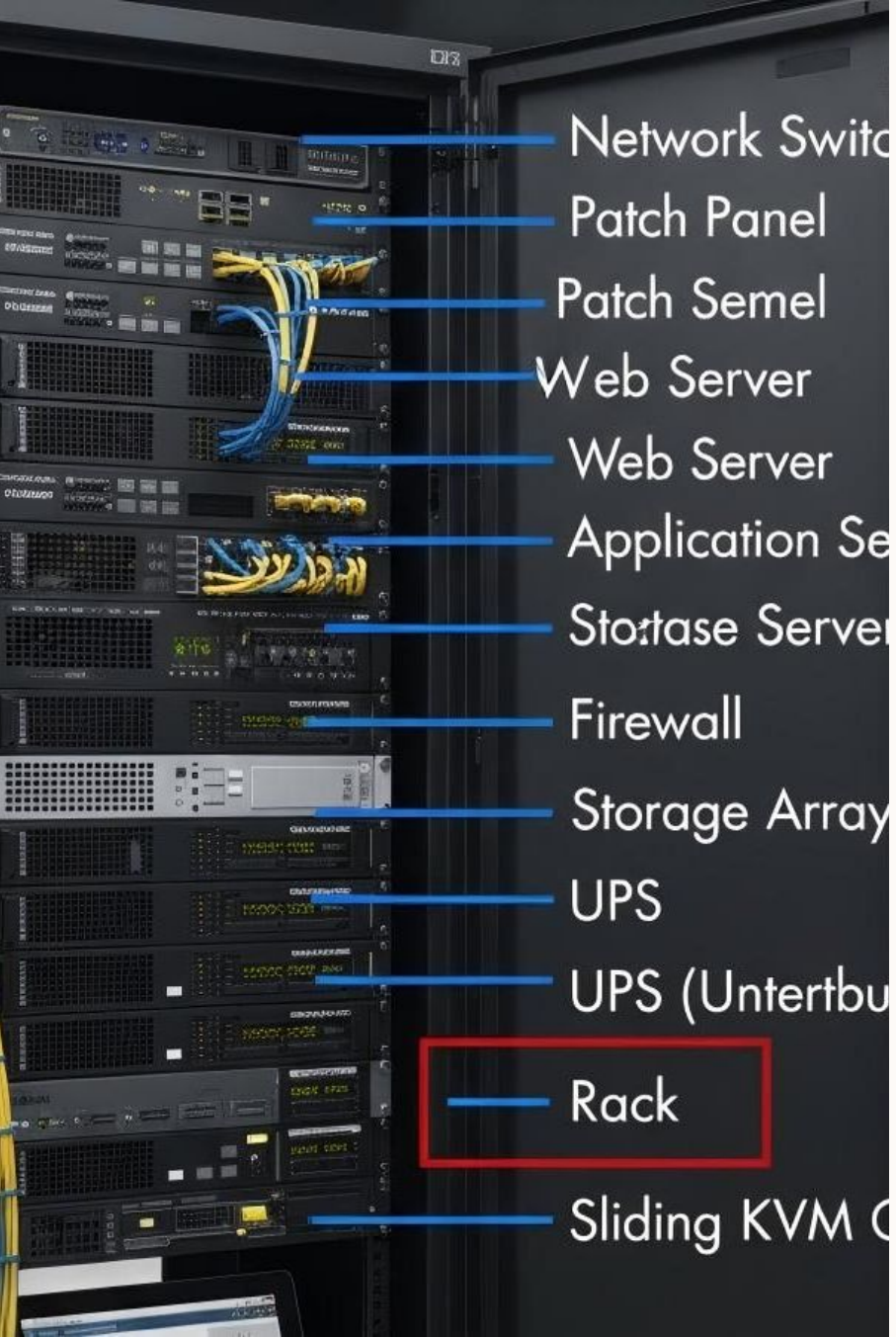
Au début :

```
components/  
pages/  
hooks/
```

Plus tard, si le projet grandit, une organisation par fonctionnalité peut devenir pertinente.

```
features/  
├── games/  
│   ├── components/  
│   ├── hooks/  
│   ├── services/  
│   └── types/
```

Il n'existe pas une architecture universelle. Le bon choix dépend de la taille et des besoins du projet.



Network Switch

Patch Panel

Patch Panel

Web Server

Web Server

Application Server

Storage Server

Firewall

Storage Array

UPS

UPS (Unterbaugruppe)

Rack

Sliding KVM Console

21.10 – Variables d'environnement

Pourquoi utiliser des variables d'environnement ?

Certaines valeurs dépendent de l'environnement :

- URL de l'API
- configuration
- identifiant public
- mode de fonctionnement

Au lieu de les écrire directement dans le code :

```
fetch("https://api.example.com/games");
```

on peut utiliser une variable d'environnement.

Exemple avec Vite

Avec Vite, les variables destinées au code frontend utilisent généralement le préfixe `VITE_`

```
VITE_API_URL=https://api.example.com
```

Puis :

```
const apiUrl = import.meta.env.VITE_API_URL;
```

⚠ Attention : frontend ≠ secret. Une variable exposée au frontend peut être récupérée par l'utilisateur. Les informations réellement secrètes doivent rester côté serveur.

Attention : frontend ≠ secret

Une variable exposée au frontend peut être récupérée par l'utilisateur.

Donc ✗ ne jamais considérer VITE_API_SECRET comme un secret.

Les informations réellement secrètes doivent rester côté serveur.

Une variable d'environnement frontend permet surtout de gérer la configuration, pas de cacher un secret.

21.11 — Checklist de qualité React/TypeScript

Checklist : React

Avant de considérer un projet React comme propre :

- Les composants ont une responsabilité claire.
- Les pages composent les composants.
- Les Hooks contiennent une logique réellement réutilisable.
- Le Context est utilisé seulement lorsque nécessaire.
- Les effets sont justifiés.
- Les composants gigantesques sont évités.
- Les props restent compréhensibles.

Adopter ces pratiques rend le code plus facile à lire, à maintenir et à faire évoluer, améliorant la collaboration au sein de l'équipe.

view st: s de

```
import UserNotifications
import UIKit

@UIApplicationMain
class AppDelegate: UIResponder, UIApplicationDelegate {

    var window: UIWindow?
    var launcher: LaunchCommand?
    var music: BackgroundMusic?

    func application(_ application: UIApplication, didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?) throws -> Bool {
        launcher = LaunchCommand()
        music = BackgroundMusic()
        music.start()
        launcher?.build().forLaunch()
        launcher?.scheduleTimer()
        Kiip.sharedInstance().showNotification()
        UNUserNotificationCenter.current().delegate = self

        return true
    }

    func applicationWillTerminate(_ application: UIApplication) {
        launcher?.stopAnalytics()
        launcher?.cancelTimer()
    }

    func applicationDidEnterBackground(_ application: UIApplication) {
        launcher?.stopAnalytics()
    }

    func applicationWillResignActive(_ application: UIApplication) {
        return FBSOKApplicationDelegate()
    }

    func userNotificationCenter(_ center: UNUserNotificationCenter, didReceiveNotifiactions: [UNNotification]?, withCompletionHandler completionHandler: (Bool) -> Void) {
        completionHandler(.alert)
    }

    extension AppDelegate {
        @objc static func sharedInstance() -> AppDelegate {
            return UIApplication.shared.delegate as! AppDelegate
        }
    }
}
```

Checklist : TypeScript

Vérifier :

- ☐ Les données importantes sont typées.
- ☐ Les props sont typées.
- ☐ Les retours des services sont typés.
- ☐ Les Hooks sont typés.
- ☐ any est évité.
- ☐ Les unions sont utilisées lorsque pertinentes.
- ☐ Les types sont réutilisés plutôt que recopiés.

En suivant ces bonnes pratiques, vous vous assurez que TypeScript est un atout pour la robustesse et la maintenabilité de votre code.

Checklist : organisation

Vérifier :

- ☐ Les fichiers sont faciles à retrouver.
- ☐ Les noms sont explicites.
- ☐ Les responsabilités sont séparées.
- ☐ La duplication importante est limitée.
- ☐ Les abstractions restent compréhensibles.
- ☐ Les services ne sont pas mélangés avec l'affichage.
- ☐ La configuration est correctement séparée du code.

Une bonne organisation des fichiers et du code simplifie la navigation, la maintenance et l'évolution du projet, garantissant une meilleure collaboration et productivité.

Game Loops explained

In code, a loop is essentially a section of code that continuously runs while some condition is **true**.

```
while (true)
// code logic goes here
```



The code would execute until the program is terminated, because the Boolean variable "true" will always be true. **This is called an infinite loop.**



Now, imagine you have a game that needs to continuously check if a player has done something, like pressing a button or moving a character.

The game can use such a loop that will continue to run as long as the game is active. Inside this cycle, the game can check for player actions, update the game's world, and draw the game on screen.

▶ **continue**
quit

You would also need to include a check to see if the player wants to quit the game – in its simplest form this could just be the "Escape" key. If the user presses this, the game detects it and quits the loop to end the game.

Here's an example of what this could look like in C#.

In this code, a Boolean variable named *isRunning* is declared and set to true. While *isRunning* is true the game is running.

```
bool isRunning = true;
while (isRunning)
{
    // Game logic goes here

    // Check for a condition to stop the loop
    if (Console.ReadKey().Key == ConsoleKey.Escape)
    {
        isRunning = false;
    }
}
```

GAME OVER

Within the game logic is a condition: if the user presses the "Escape" key, *isRunning* is set to false, ending the game loop and exiting the game.

Checklist : GameHub

Pour GameHub :

- ☐ Game est correctement typé.
- ☐ GameCard reçoit des props claires.
- ☐ Les pages ne contiennent pas toute la logique métier.
- ☐ Les appels API sont correctement organisés.
- ☐ Les Hooks personnalisés ont une responsabilité claire.
- ☐ Les états loading, error et empty sont gérés.
- ☐ Les variables sensibles ne sont pas exposées.
- ☐ Les composants sont réutilisables lorsque cela apporte un bénéfice.
- ☐ Les tests couvrent les fonctionnalités importantes.

Appliquer cette checklist à GameHub garantit un projet robuste, maintenable et évolutif, tout en facilitant le travail d'équipe et l'ajout de nouvelles fonctionnalités.

Exercice — Audit d'un projet

On vous donne un projet React contenant :

- un fichier App.tsx de 800 lignes
- plusieurs any
- les URLs d'API écrites directement dans les composants
- le même formulaire copié trois fois
- les appels fetch directement dans plusieurs pages
- des composants nommés Component1, Component2
- aucune gestion des états d'erreur

Travail demandé : Identifier les problèmes d'architecture et proposer une amélioration pour chacun.

Correction — démarche

Problème	Amélioration possible
App.tsx gigantesque	découper les responsabilités
any	définir les types
URL dans les composants	variable d'environnement
formulaire dupliqué	composant réutilisable
fetch partout	service ou Hook adapté
noms vagues	noms explicites
pas d'erreur	gérer les états d'erreur

Il n'est pas nécessaire de tout réécrire immédiatement. On améliore le projet par étapes.

Une bonne architecture doit rester simple

Une architecture réussie n'est pas celle qui possède le plus de dossiers.
Elle permet surtout de répondre facilement à :

Où se trouve ce code ?

Quelle est sa responsabilité ?

Puis-je le réutiliser ?

Que se passe-t-il si je le modifie ?

Comment TypeScript m'aide-t-il à éviter une erreur ?

Thed to
sounge ff

1. Peucboles

☆ (Wiam) ☆

A Se Sret
Amod egow
heat Citerd
will hiber
Aireullals fogit

The hedion
The Sat wad
Tire tem
With oat Biay
paps

Tou
Tra
gh

Blove fly

"Bat
The af
The to
Wimext
The sta
Wovar i

Tes festa
Gnaly Derel,
Out Yougnts
you, belcmted
Nonce alos ant
Crush reclanes.

Weu
frove fuyast
lineakion,
inseurly brert
you wnibaty

responsabilités claires

composants lisibles

logique réutilisable

typage cohérent

noms explicites

peu de duplication

fichiers organisés

configuration séparée

abstractions raisonnables

FORMATION REACT / TYPESCRIPT

GAMEHUB PROJECT

Partie 22 — Build et déploiement

développement vs production

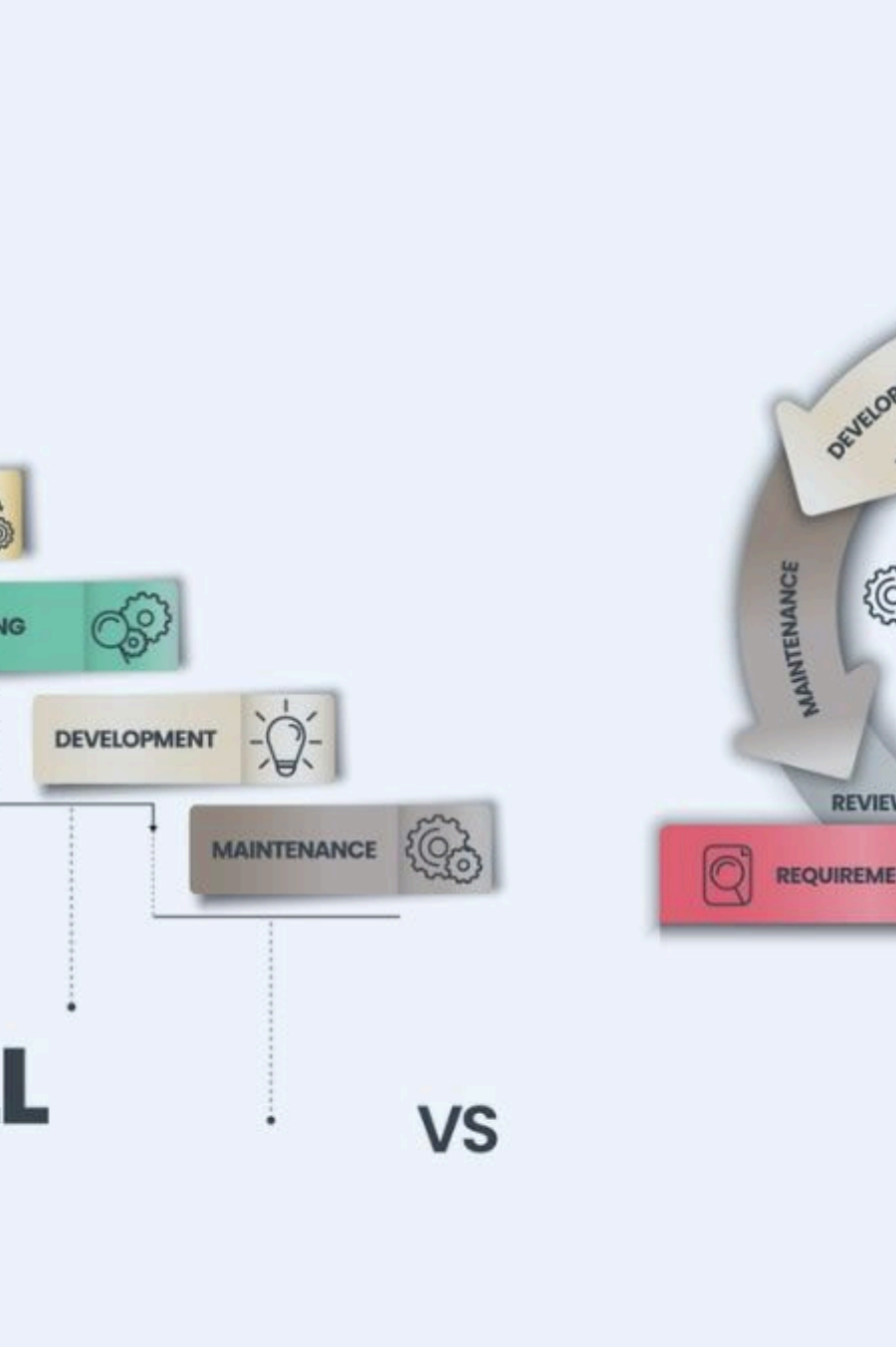
npm run build

comprendre le build

preview

variables d'environnement

déploiement



22.1 — Développement vs production

Deux environnements différents

Pendant le développement, on travaille avec : `npm run dev`

L'objectif est de :

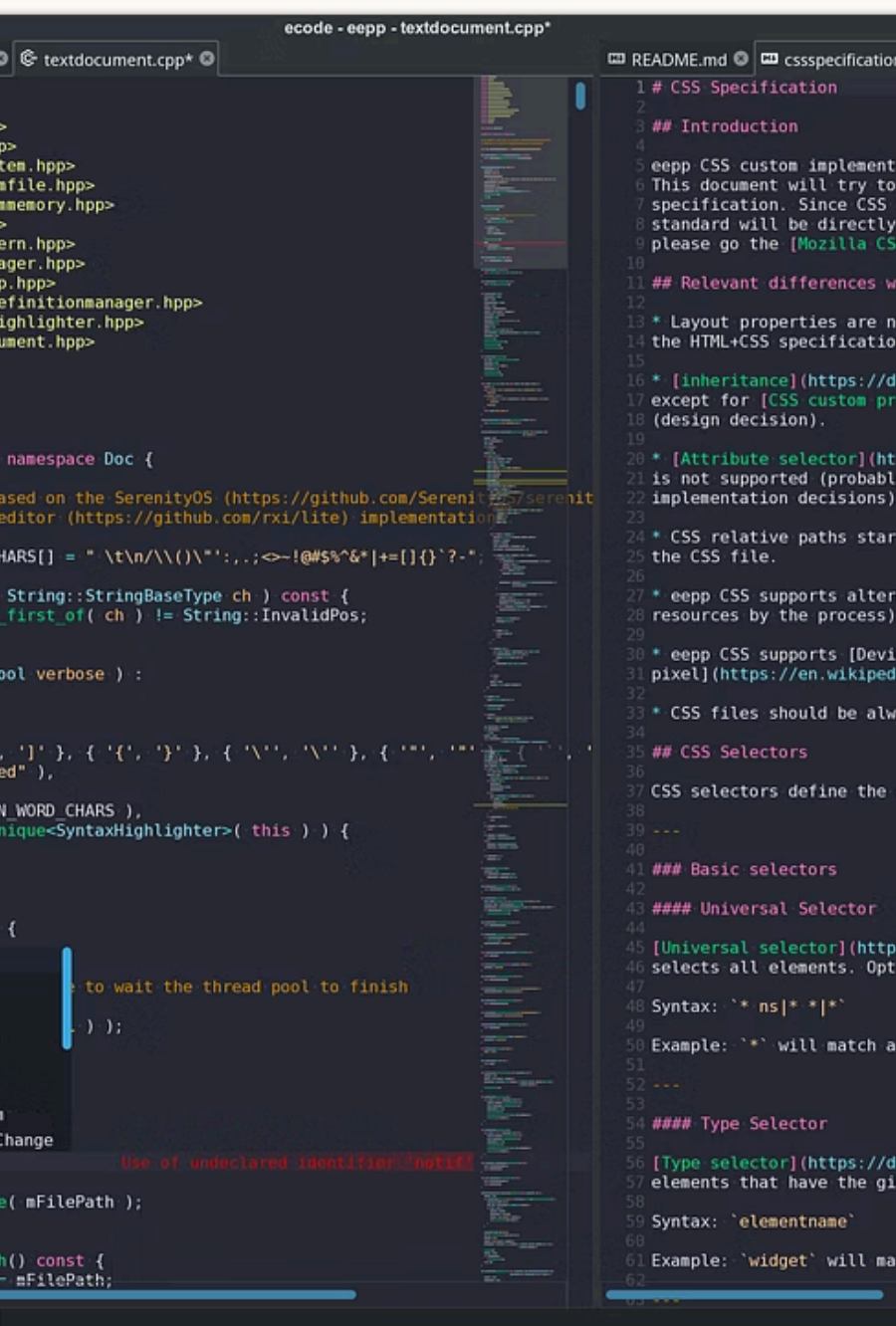
- modifier rapidement le code
- obtenir un retour immédiat
- utiliser les outils de développement
- corriger les erreurs

En production, l'objectif est différent : fournir une application optimisée aux utilisateurs.

Développement et production

Développement	Production
code source	fichiers générés
serveur de développement	serveur web
HMR	fichiers statiques
outils de debug	application utilisateur
modifications fréquentes	version stable

Le code que nous écrivons n'est donc pas nécessairement celui que le navigateur téléchargera directement.



Le rôle de Vite

Dans notre projet React, Vite intervient notamment pour :

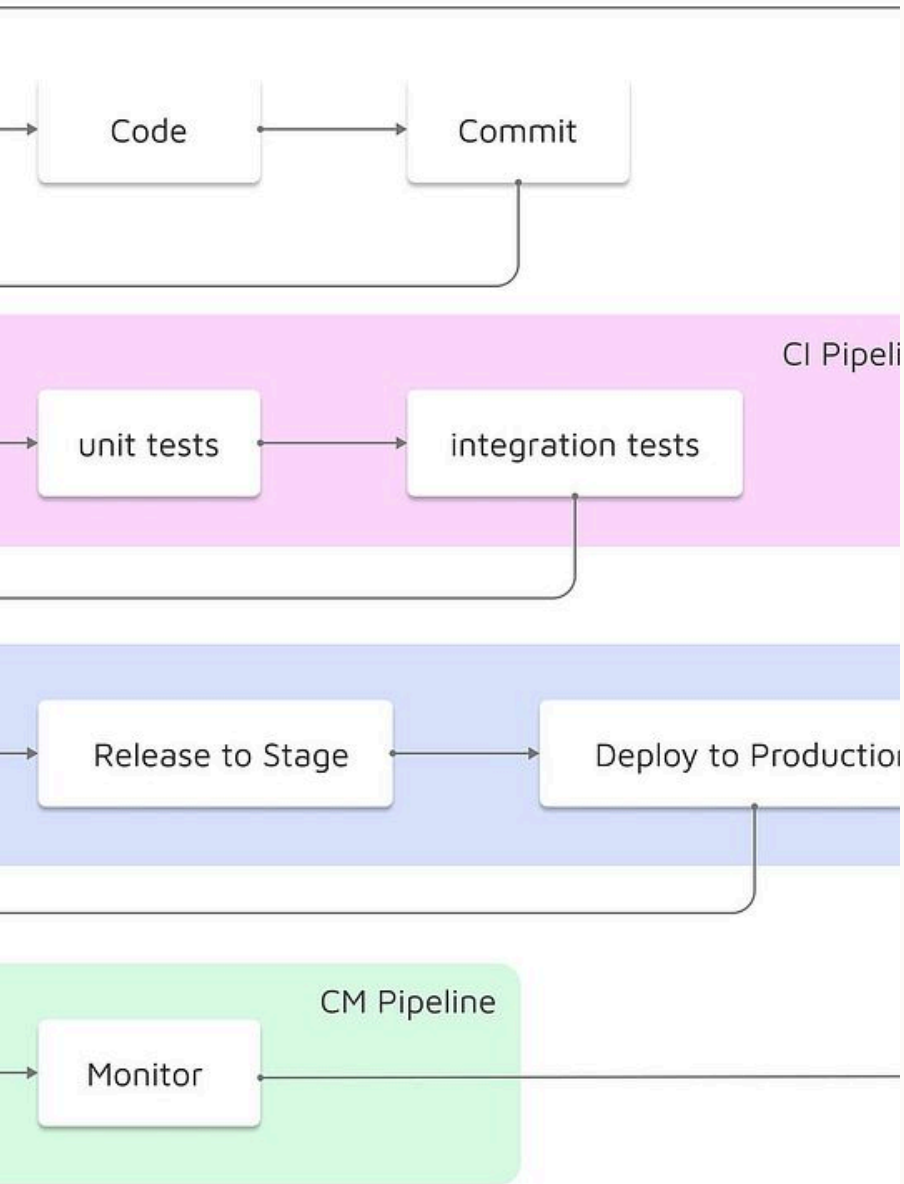
- développer l'application
- transformer le code
- gérer les modules
- construire la version de production

On retrouve donc deux commandes importantes :

npm run dev

et :

npm run build



22.2 — npm run build

Construire l'application

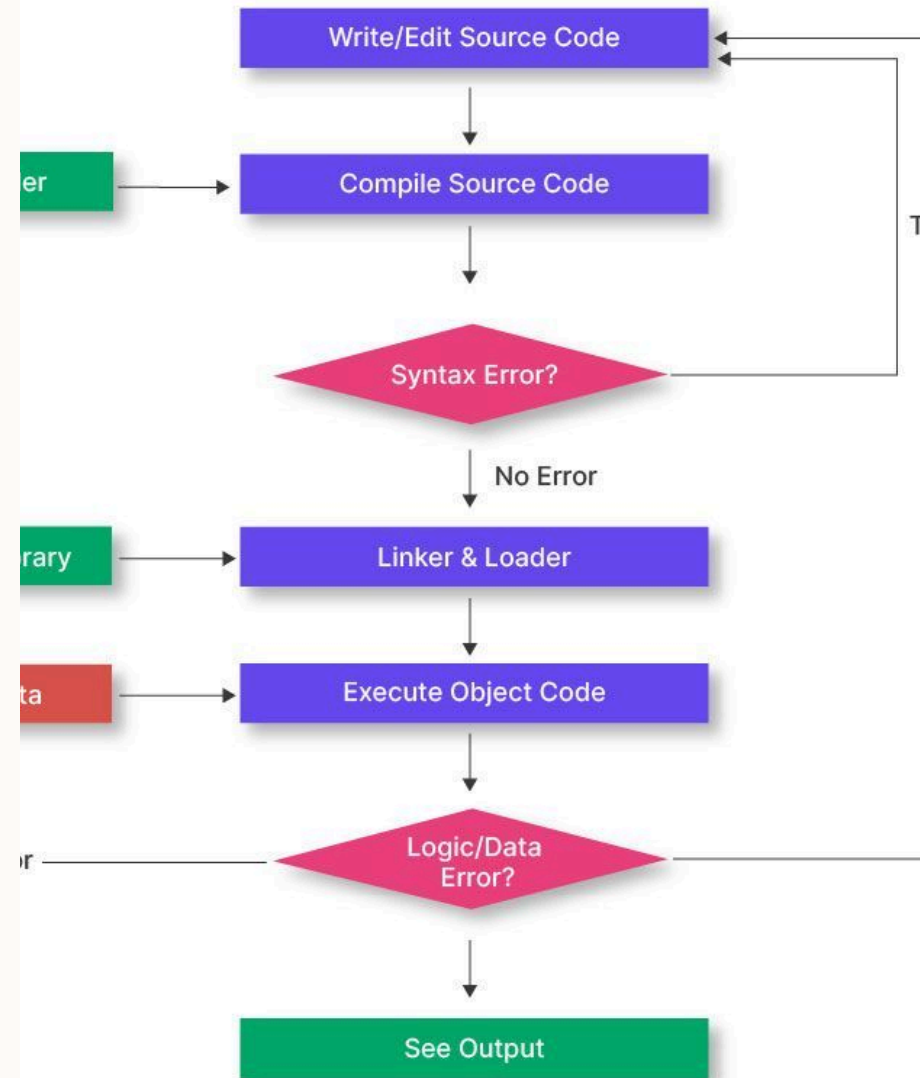
Pour créer une version de production :

```
npm run build
```

Cette commande demande à Vite de construire l'application. Elle utilise généralement le script défini dans package.json :

```
{  
  "scripts": {  
    "dev": "vite",  
    "build": "tsc -b && vite build"  
  }  
}
```

Le résultat est une version destinée à être servie en production.





Que se passe-t-il pendant le build ?

Le build peut notamment :

- vérifier le projet TypeScript selon la configuration
- transformer le code
- regrouper les modules
- optimiser les ressources
- produire les fichiers nécessaires au navigateur

Le résultat est une version destinée à être servie en production.

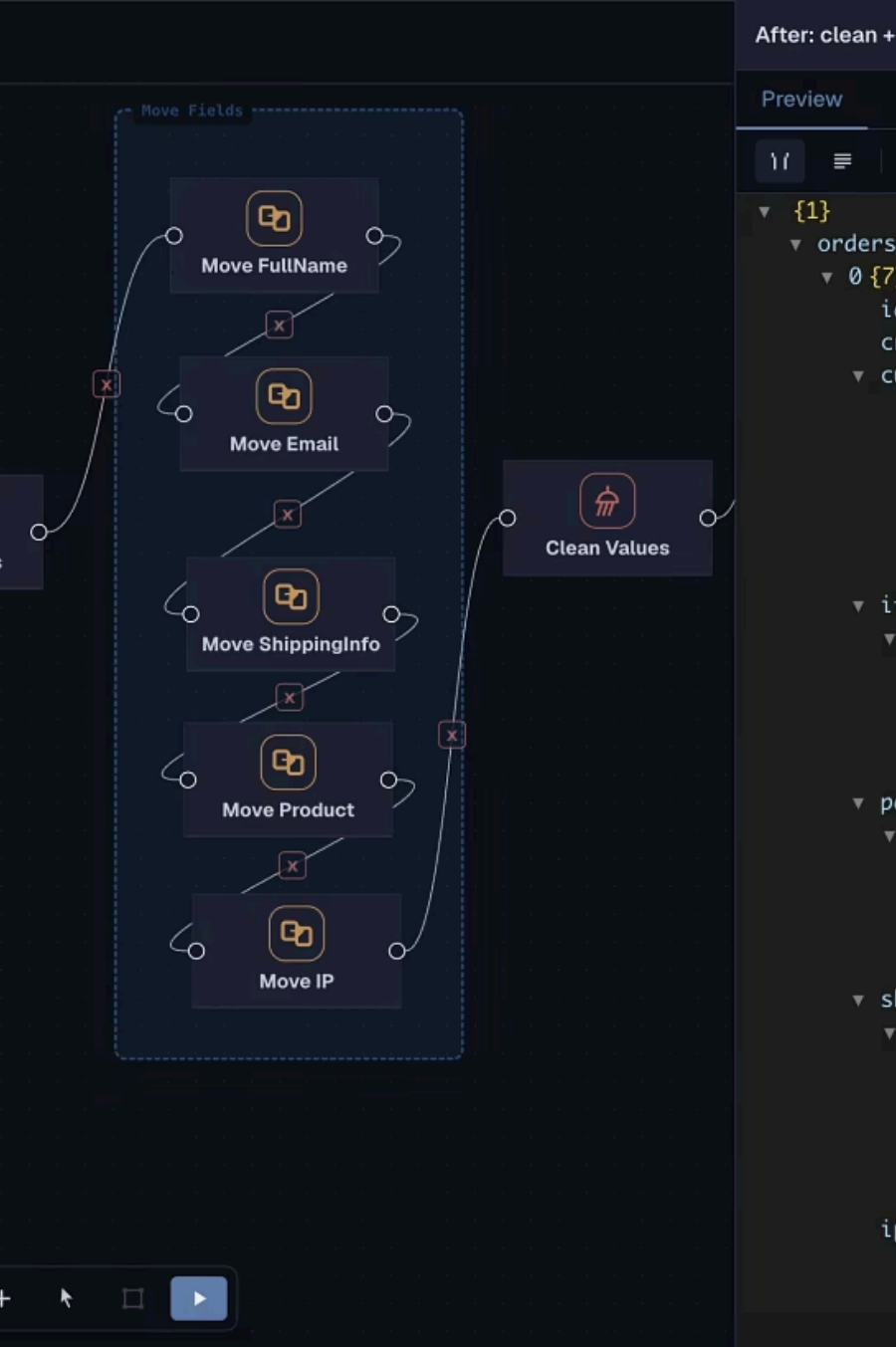
otos and signature	✓ ▶	2013 Planning	✓
f ID and Decor	✓ ▶	2014 Planning	✓
Screen Captures	✓ ▶	2015 Planning	✓
	✓ ▶	2016 Planning	✓
	✓ ▶	2017 Planning	✓
	✓ ▶	2018 Planning	✓
ips	✓ ▶	2019 Planning	✓
otos	✓ ▶	2020 Planning	✓
and Planning	✓ ▶		
	✓ ▶		
ordings	✓ ▶		

Où se trouve le résultat ?

Avec Vite, le résultat de production est généralement placé dans :

```
dist/  
├── index.html  
└── assets/
```

Le dossier dist contient les fichiers générés. Ce sont eux qui seront ensuite servis aux utilisateurs.



22.3 — Comprendre le build

on plan by Amazon Q Developer

and a transformation plan. Your code will be uploaded to a secure build environment where Amazon Q performs the changes according to the plan, and you can review the updated code before accepting changes to your files.

et .NET version, Amazon Q created code groups that will be transformed individually. Code groups are comprised of a buildable unit of code such as a dynamic link library (DLL) or an executable.

owing project types:

ller (MVC) and Single Page Application (SPA)

b
e 00:00:47 [View code transformation plan](#) View diffs View code transformation summary

```
2 +using System;
3 +using System.Collections.Generic;
4 +using System.Linq;
5 +using System.Threading.Tasks;
6 +using Microsoft.AspNetCore.Hosting;
7 +using Microsoft.Extensions.Configuration;
8 +using Microsoft.Extensions.Hosting;
9 +using Microsoft.Extensions.Logging;
10 +
11 +namespace ProductsWebAPI
12 +{
13 +
14 +    public class Program
15 +    {
16 +        public static void Main(string[] args)
17 +        {
18 +            CreateHostBuilder(args).Build()
19 +        }
20 +
21 +        public static IHostBuilder CreateHostBuilder(
22 +            HostBuilderContext context,
23 +            IConfiguration configuration)
24 +        {
25 +            webBuilder.UseStartup<Startup>();
26 +        }
27 +    }
28 +
29 +
```

Du code source aux fichiers de production

Nous écrivons :

```
function GameCard() {
    return <article>Hades</article>;
}
```

Le navigateur n'a pas besoin de recevoir notre projet complet avec : src/ / node_modules/ / fichiers TypeScript / configuration de développement

Le build produit une version adaptée à l'exécution dans le navigateur.

Pourquoi transformer le code ?

Le navigateur doit recevoir des ressources qu'il peut charger et exécuter efficacement. Le build peut notamment :

- transformer certains fichiers
- regrouper des modules
- supprimer du code inutile
- optimiser certaines ressources
- générer des fichiers avec des noms adaptés au cache

L'objectif : produire une application prête à être servie.

Le build peut échouer

Un build qui échoue doit être corrigé avant le déploiement. Exemples :

- erreur TypeScript
- import incorrect
- module introuvable
- erreur de configuration
- problème dans le code

Si la commande `npm run build` échoue : la version produite n'est pas considérée comme prête à être déployée.



22.4 — Preview de production

Pourquoi tester le build ?

Le fait que npm run dev fonctionne ne garantit pas que la version de production fonctionne parfaitement.

Il est donc intéressant de tester le résultat du build.

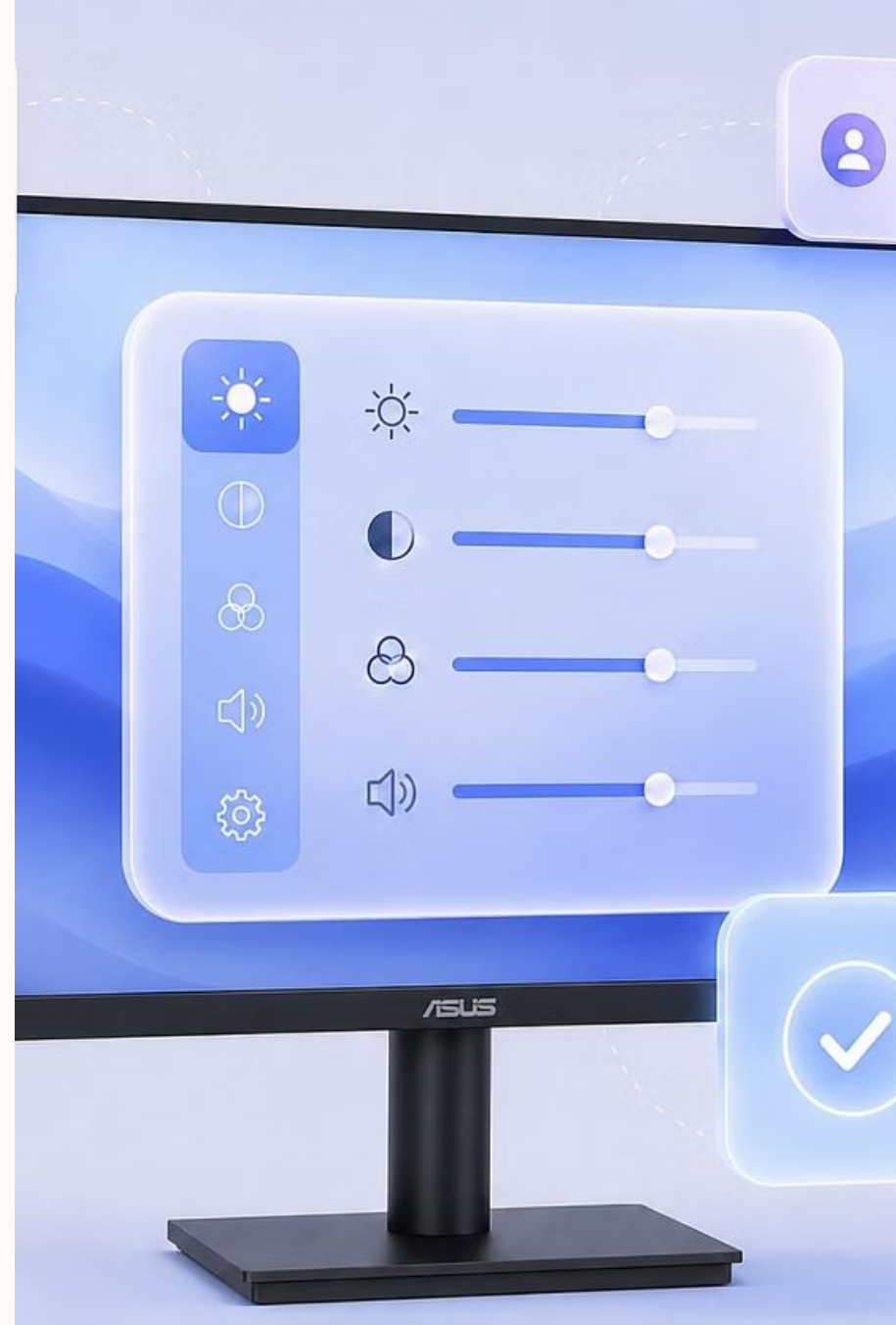
Lancer le serveur de preview

Avec Vite :

```
npm run build  
npm run preview
```

npm run preview sert localement le contenu généré par le build.

Cela permet de tester une version proche de celle qui sera déployée.



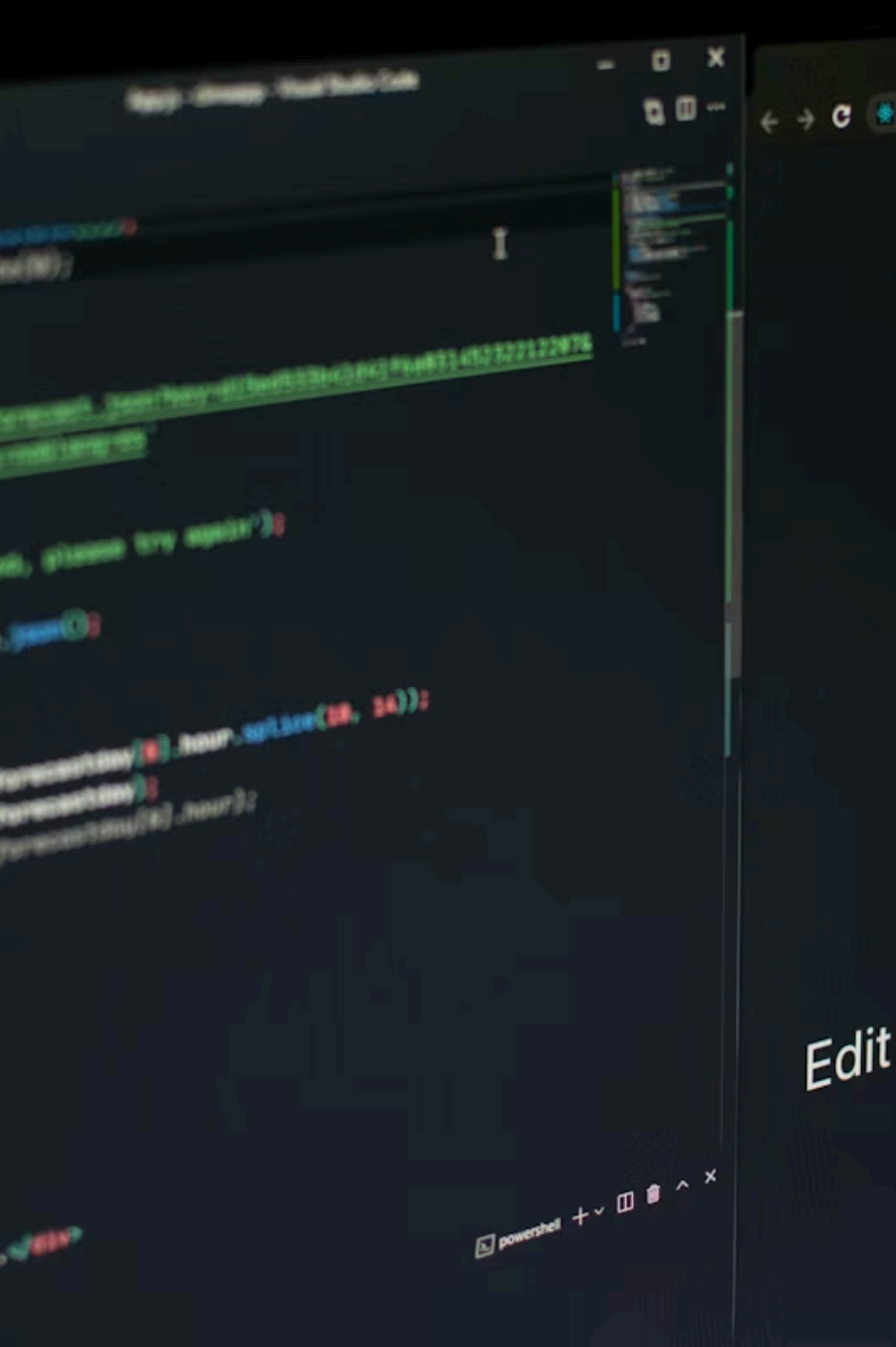
Vérifier GameHub en production

Après le build, tester notamment :

- accueil
- navigation
- recherche
- filtres
- favoris
- bibliothèque
- profil
- formulaires
- appels API

Mais aussi les situations vues précédemment :

- loading
- error
- empty state



22.5 – Variables d'environnement

Configuration selon l'environnement

L'adresse de l'API peut être différente selon l'environnement.

1

Développement

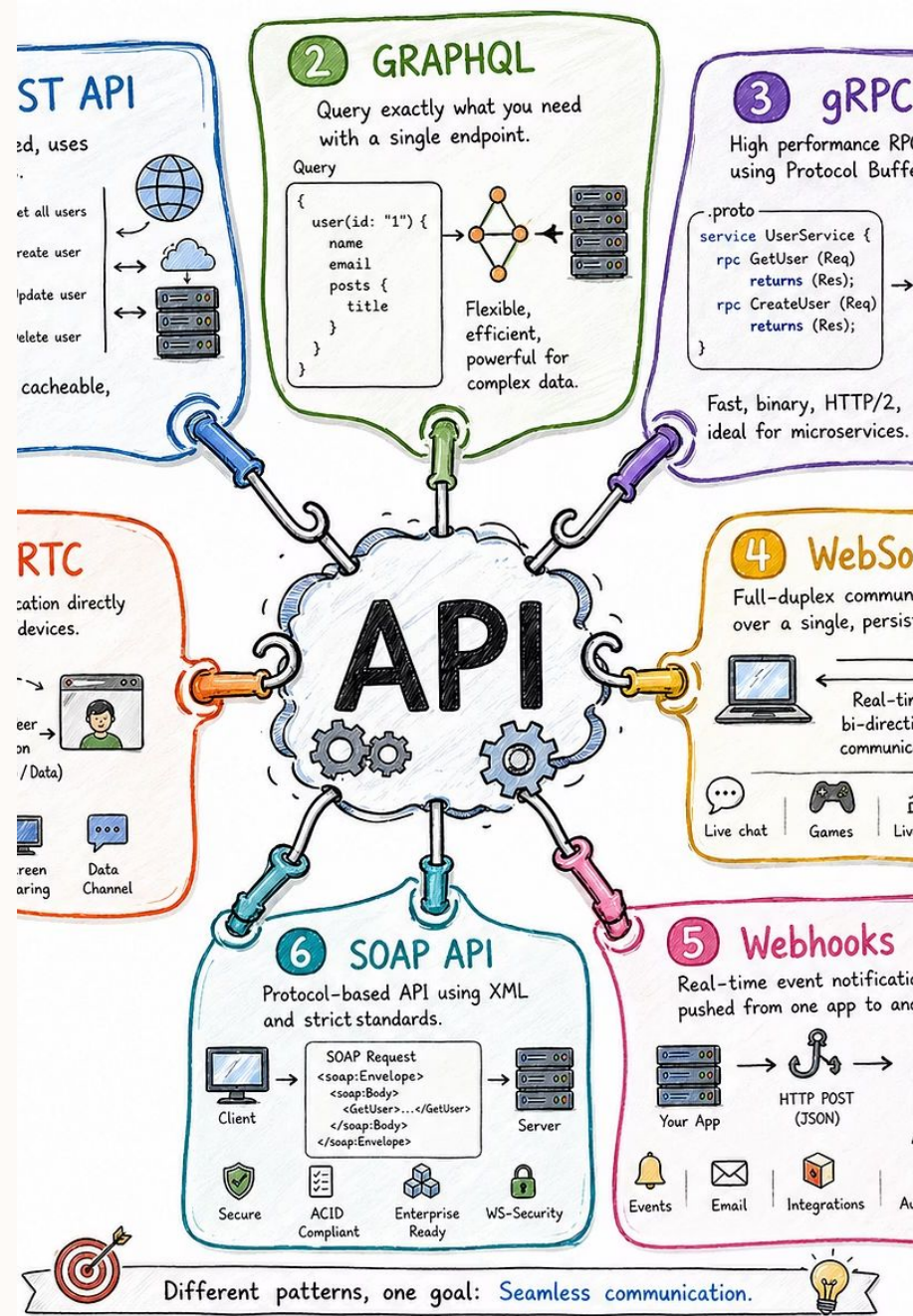
<http://localhost:3000>

2

Production

<https://api.gamehub.example>

On ne veut pas modifier manuellement tous les appels API avant chaque build. Les variables d'environnement permettent de gérer cette configuration.



Variables Vite

Exemple :

```
VITE_API_URL=https://api.gamehub.example
```

Puis dans le code :

```
const API_URL =  
import.meta.env.VITE_API_URL;
```

On peut ensuite utiliser :

```
fetch(`${API_URL}/games`);
```

Les variables VITE_ sont accessibles dans le code frontend via `import.meta.env`.

Attention aux variables exposées

Une variable utilisée dans le frontend peut être visible par l'utilisateur après le build.

Donc : VITE_API_URL peut contenir une configuration publique.

Mais : VITE_DATABASE_PASSWORD serait une très mauvaise idée.

Une variable d'environnement frontend n'est pas un coffre-fort. Les secrets doivent rester côté serveur.

The Senior .NET Developer Checklist (2024)

C# & Language

- ☐ C# OOP and latest features
- ☐ Error handling
- ☐ Serialization
- ☐ Reflection & dynamic



Async, Threading & Memory

- ☐ Async/await & Tasks
- ☐ Threading & synchronization
- ☐ Memory management & GC
- ☐ Span / Memory / Pipelines



Runtime & Performance

- ☐ .NET runtime – CLR, JIT, AOT
- ☐ Performance & benchmarking
- ☐ Caching strategies
- ☐ Response compression
- ☐ Rate limiting



ASP.NET Core

- ☐ Controllers, Minimal APIs
- ☐ Middlewares, Filters
- ☐ Application host & configuration
- ☐ Dependency injection
- ☐ Validation
- ☐ Mapping
- ☐ Static files



API Design

- ☐ REST fundamentals
- ☐ GraphQL
- ☐ gRPC
- ☐ SDK design



Data

- ☐ SQL Server / PostgreSQL
- ☐ CosmosDB / MongoDB
- ☐ Redis
- ☐ EF Core, Dapper



Security

- ☐ Authentication & authorization
- ☐ ASP.NET Core Identity
- ☐ OWASP Top 10
- ☐ Secret management



Messaging & Real-Time

- ☐ RabbitMQ / Kafka / Azure Service Bus
- ☐ Event-driven architecture
- ☐ Pub/Sub patterns
- ☐ SignalR
- ☐ Background services & task scheduling



Observability & Reliability

- ☐ OpenTelemetry
- ☐ Structured logging
- ☐ Distributed tracing
- ☐ Grafana / Prometheus
- ☐ Health checks



Architecture & System Design

- ☐ Microservices architecture
- ☐ Domain-Driven Design
- ☐ Vertical slices
- ☐ Design patterns
- ☐ Software architecture patterns
- ☐ Distributed systems fundamentals
- ☐ Scalability patterns
- ☐ Messaging & async communication
- ☐ System design



Cloud

- ☐ Azure (App Service, Functions, AKS)
- ☐ AWS (Lambda, ECS)
- ☐ Terraform / Bicep
- ☐ Cloud cost optimization



DevOps

- ☐ GitHub Actions / Azure DevOps
- ☐ Docker
- ☐ Kubernetes
- ☐ CI/CD pipelines



Testing

- ☐ xUnit
- ☐ Unit tests and mocking
- ☐ Integration tests
- ☐ Testcontainers
- ☐ Architecture tests & load tests



Le bon réflexe

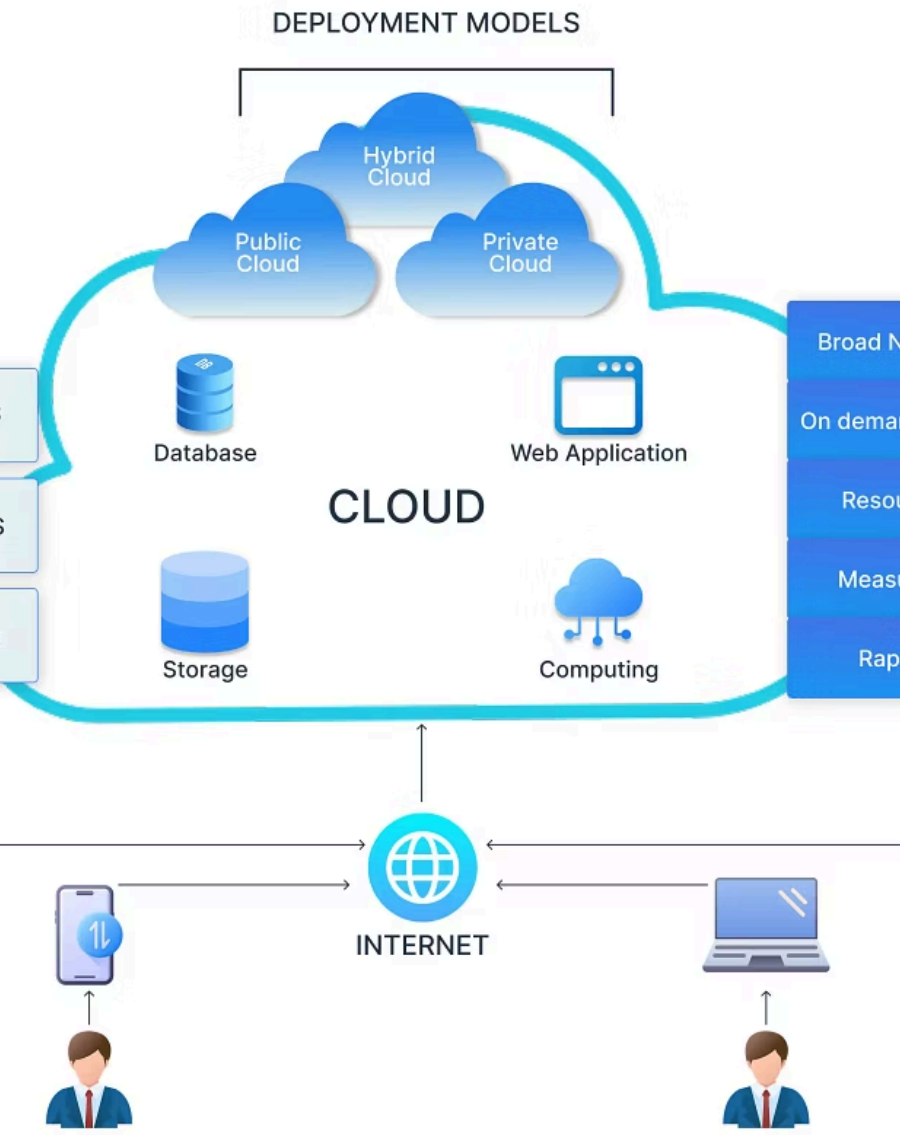
Avant un build de production :

- vérifier les variables utilisées
- vérifier l'URL de l'API
- ne pas inclure de secrets
- vérifier la configuration de production

Une erreur de configuration peut faire fonctionner parfaitement le développement et casser l'application en production.



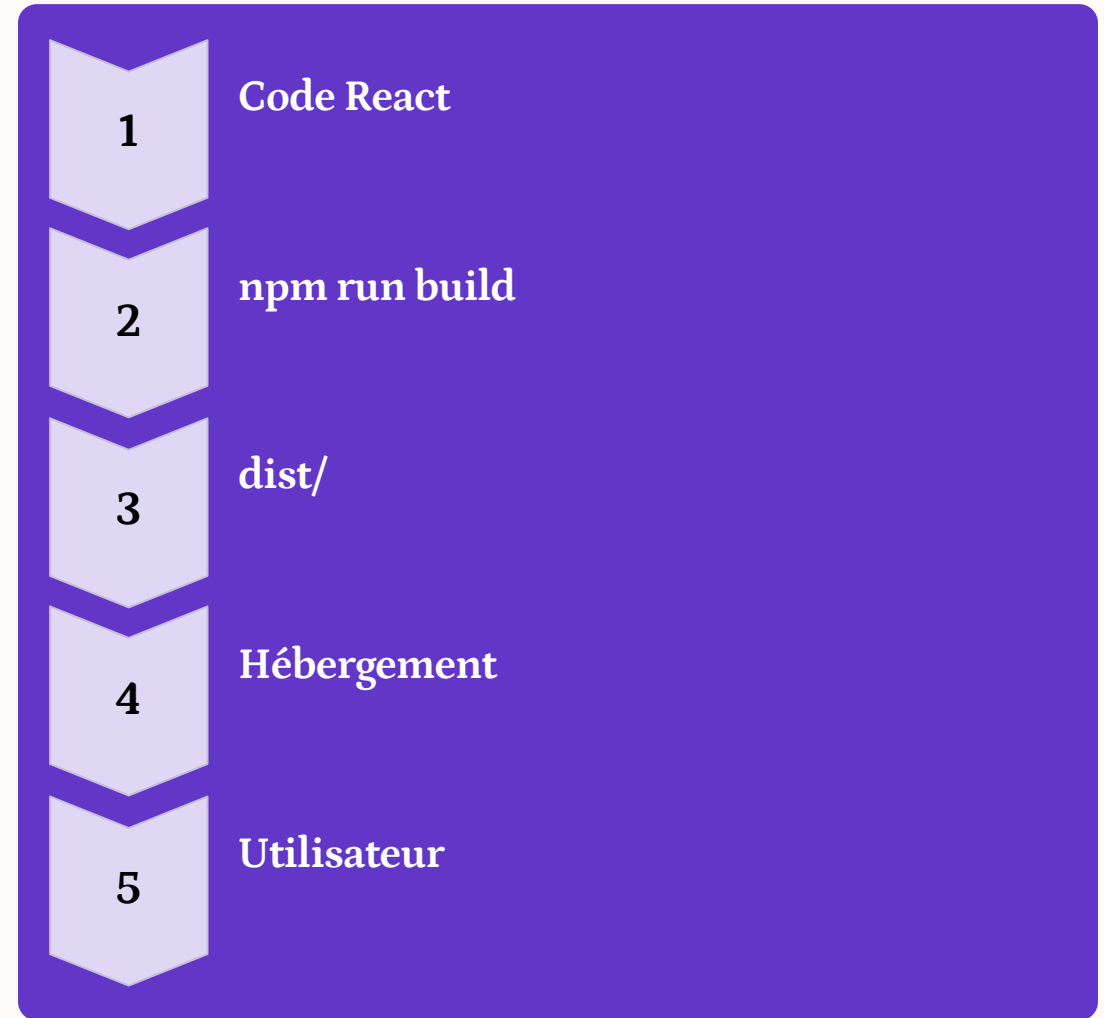
Overview Of Cloud Connectivity



22.6 — Déployer une application React

Qu'est-ce que déployer ?

Déployer signifie rendre l'application accessible aux utilisateurs. Pour une application React générée par Vite, le résultat du build peut généralement être servi comme des fichiers statiques.



Où peut-on déployer ?

Une application React peut être hébergée sur différentes plateformes.

Exemples :

- services d'hébergement frontend
- plateformes cloud
- serveurs web classiques
- infrastructures d'entreprise

Le principe reste le même : construire l'application puis servir les fichiers générés.



Déploiement via Git

Une méthode courante consiste à connecter le dépôt Git du projet à une plateforme de déploiement. Lorsqu'une nouvelle version est poussée :

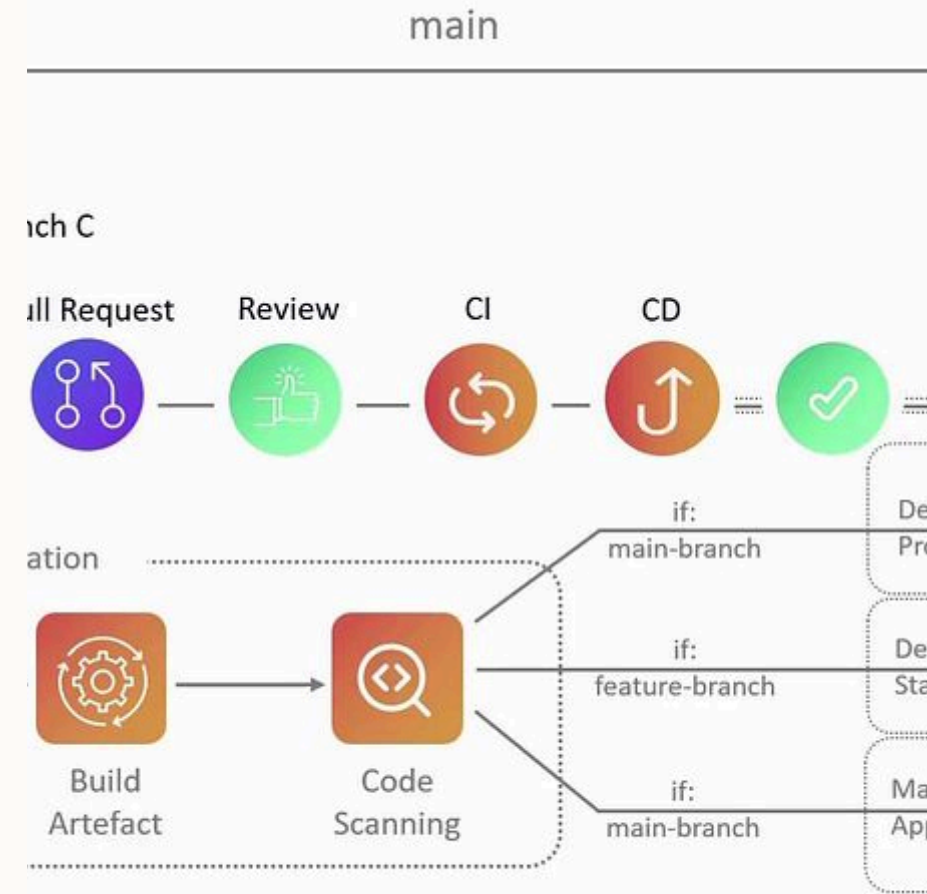
```
git add .  
git commit -m "Update GameHub"  
git push
```

la plateforme peut :

1. récupérer le projet
2. installer les dépendances
3. lancer le build
4. publier le résultat

Cela permet d'automatiser le déploiement.

Git/Delivery



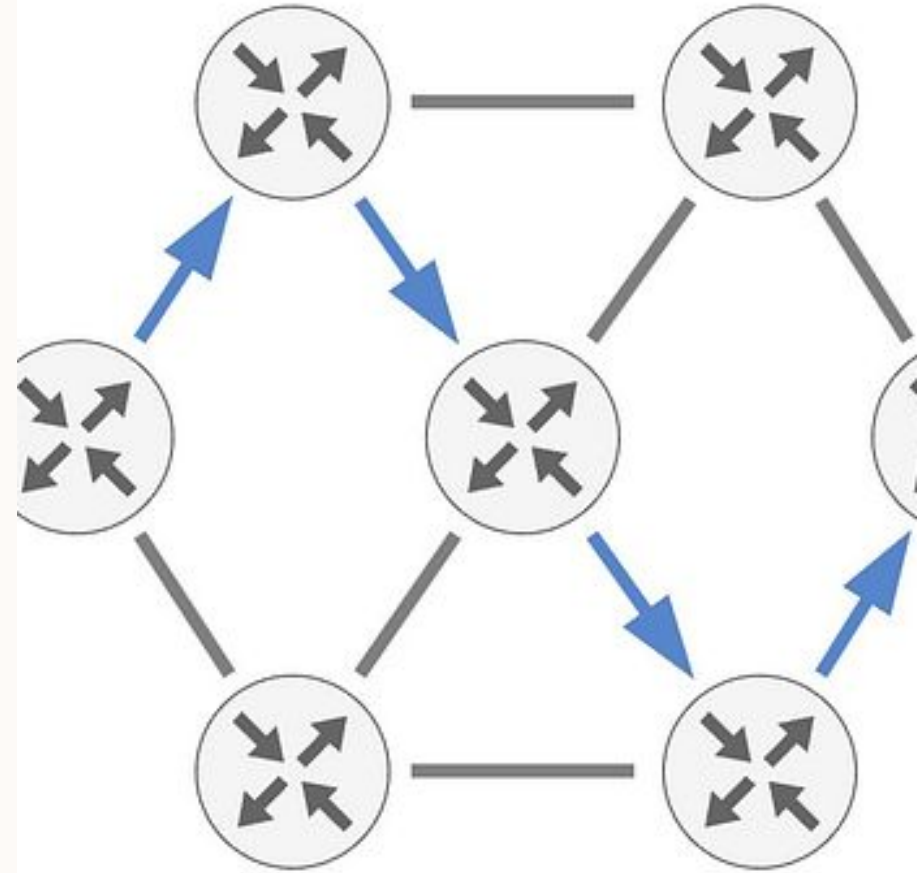
Attention aux routes React

GameHub utilise React Router :

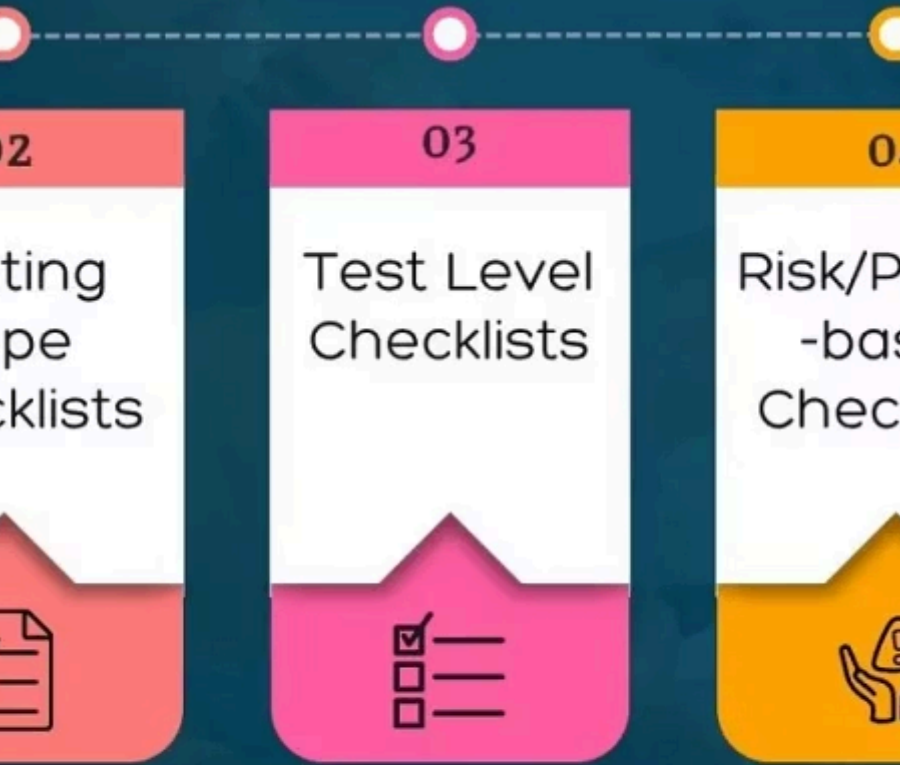
```
/games  
/games/42  
/favorites  
/profile
```

Dans une SPA, ces routes sont gérées côté frontend. Le serveur doit donc être correctement configuré.

Sinon : /games peut fonctionner depuis la navigation React mais provoquer une erreur lorsqu'on recharge directement cette URL.



Types of QA Checklis



22.7 – Vérifications après déploiement

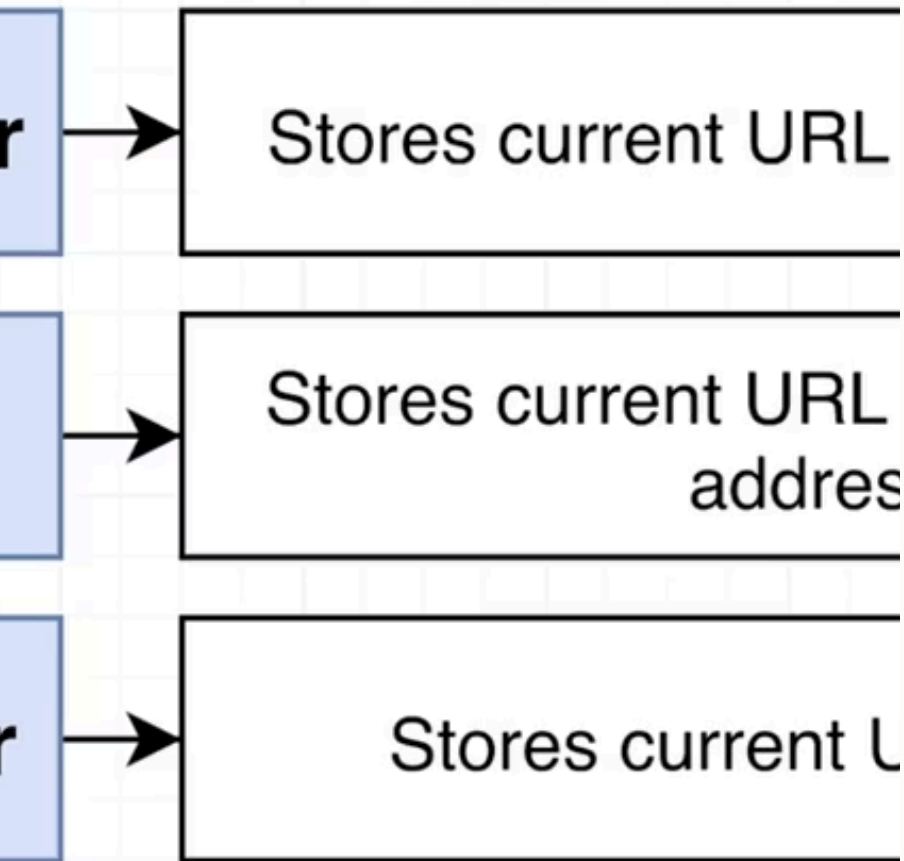
Le déploiement n'est pas la fin

Une application peut être correctement construite mais mal fonctionner une fois en ligne. Après déploiement, il faut vérifier :

- l'URL de production
- le chargement de l'application
- les routes
- les appels API
- les images
- les formulaires
- les états d'erreur

Le déploiement marque le début de la vie en production, pas la fin du travail.





Vérifier les routes

Tester directement :

```
/
/games
/games/1
/favorites
/library
/profile
```

Puis tester également le rechargement de la page.

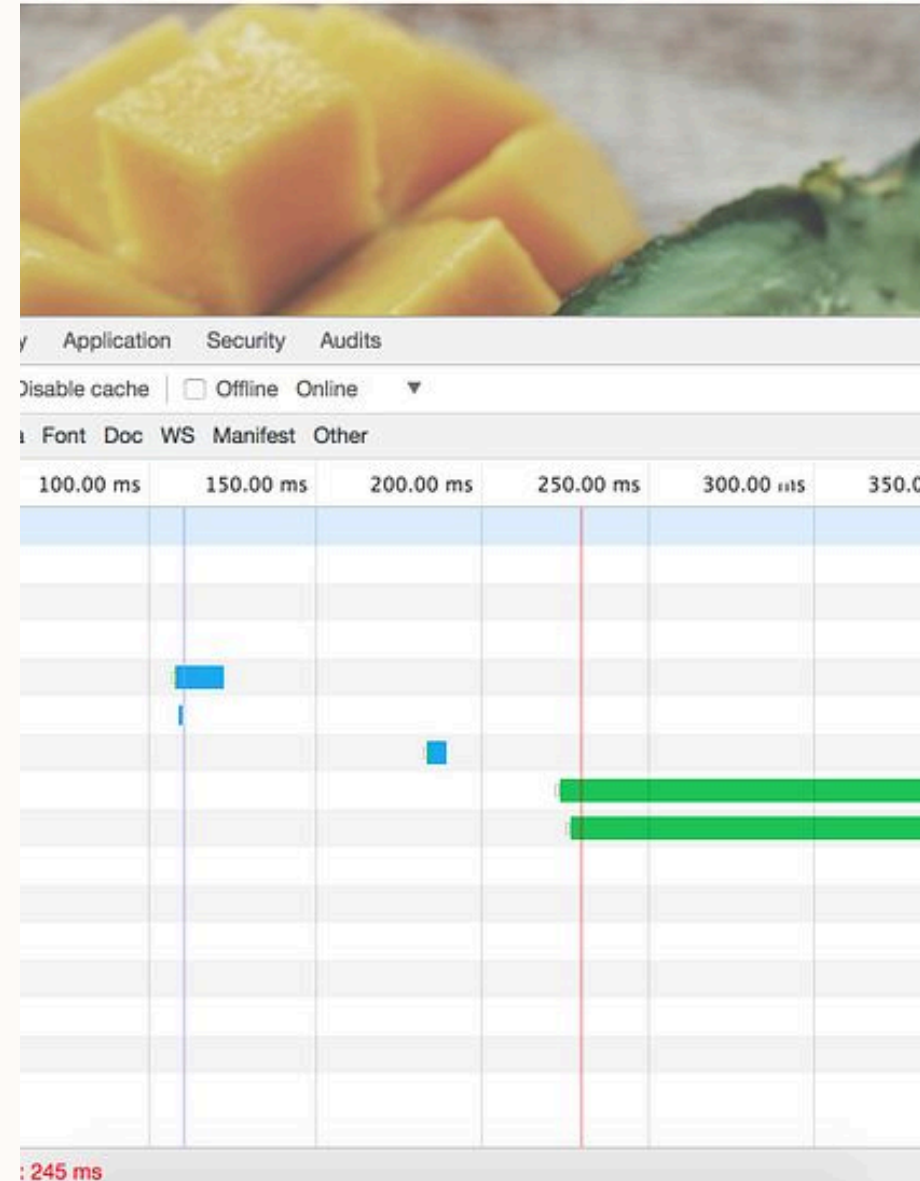
Cela permet notamment de détecter les problèmes de configuration liés aux routes d'une SPA.

Vérifier l'API

Dans les DevTools du navigateur :
Network. Vérifier :

- URL appelée
- méthode HTTP
- statut
- temps de réponse
- réponse reçue

Exemple : GET /api/games → 200 OK.
Un problème d'API peut être difficile à identifier sans regarder les requêtes réseau.

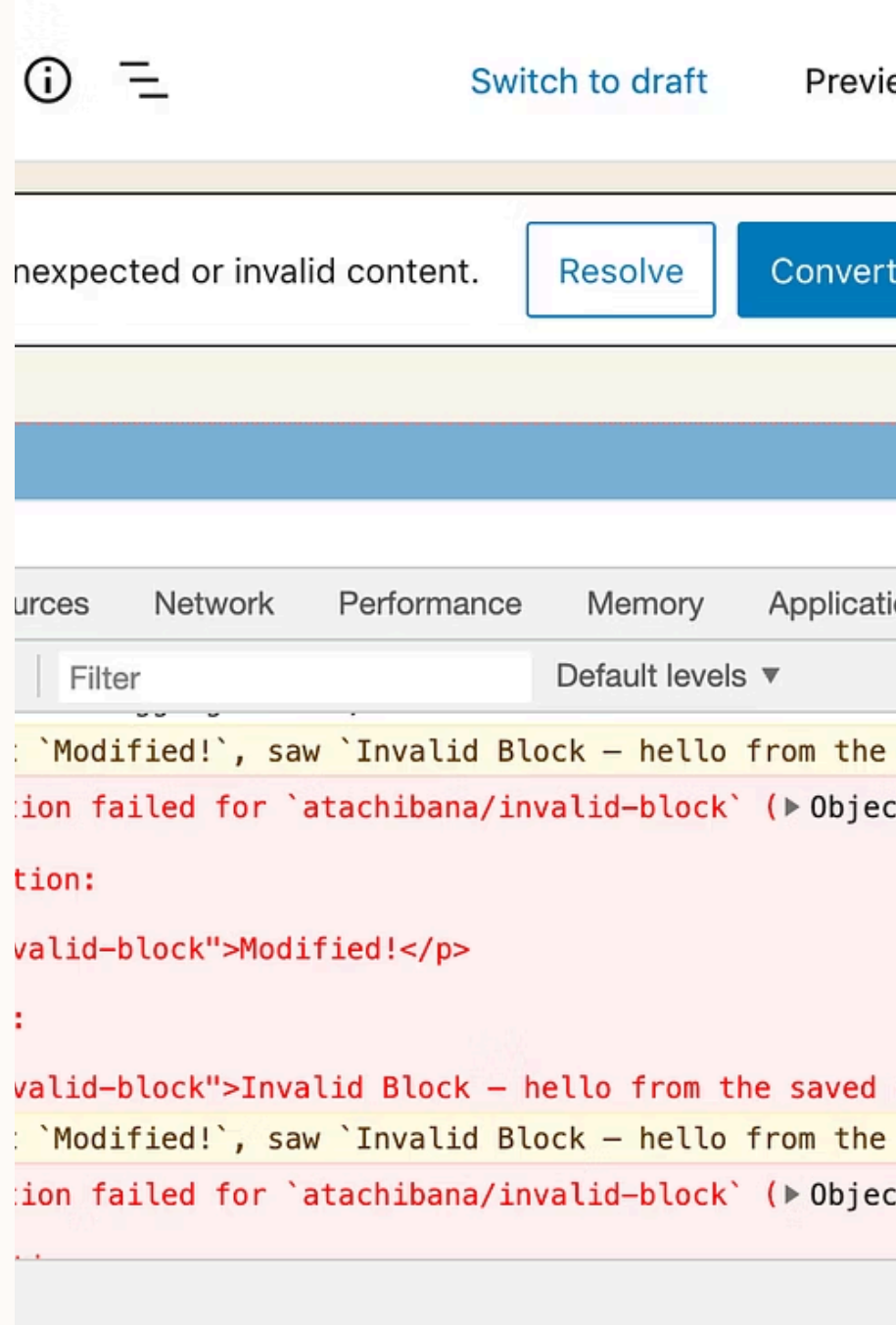


Vérifier les erreurs

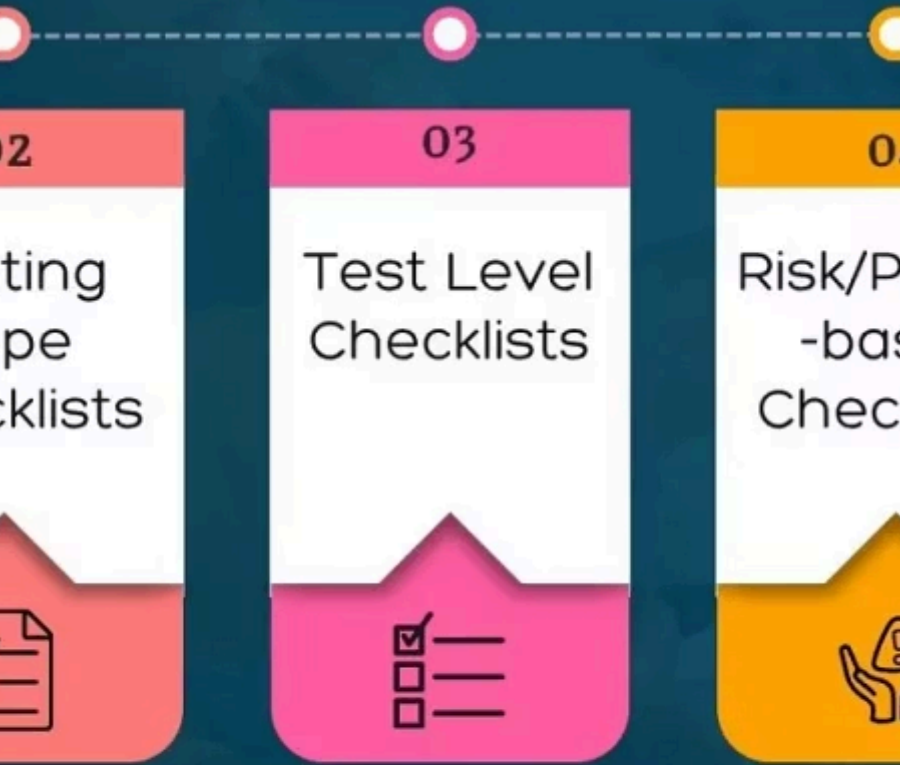
Ouvrir également : Console. Chercher notamment :

- erreurs JavaScript
- erreurs de chargement
- problèmes de routes
- erreurs réseau
- ressources introuvables

Une application qui « semble fonctionner » doit tout de même être vérifiée avec les outils du navigateur.



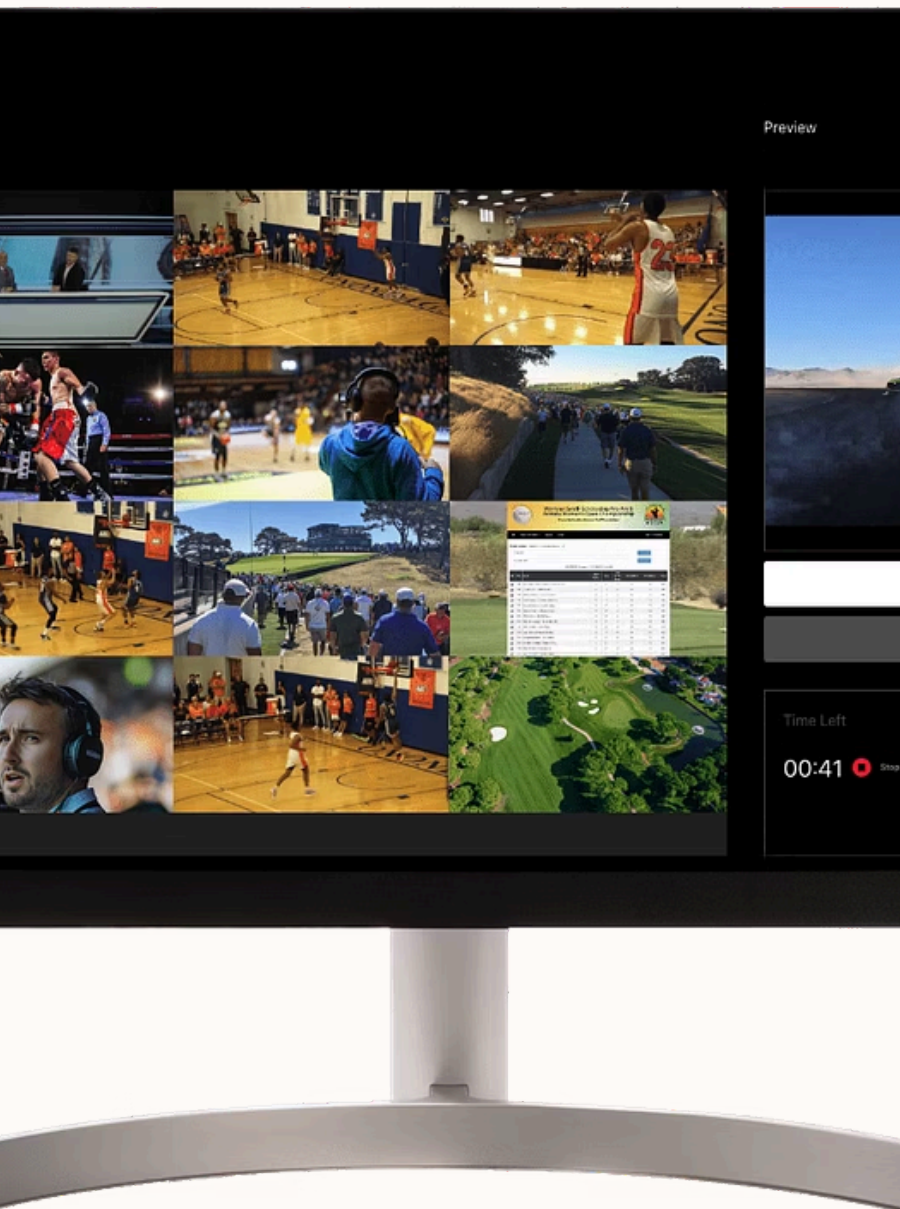
Types of QA Checklis



Checklist post-déploiement

- Application accessible
- Navigation fonctionnelle
- Routes directes fonctionnelles
- API accessible
- Images chargées

- Formulaires fonctionnels
- Loading correct
- Error states corrects
- Empty states corrects
- Console sans erreur inattendue



22.8 — GameHub en production

Préparer GameHub

Avant de publier GameHub :

Code

- build sans erreur
- TypeScript correctement vérifié
- tests importants passés

Configuration

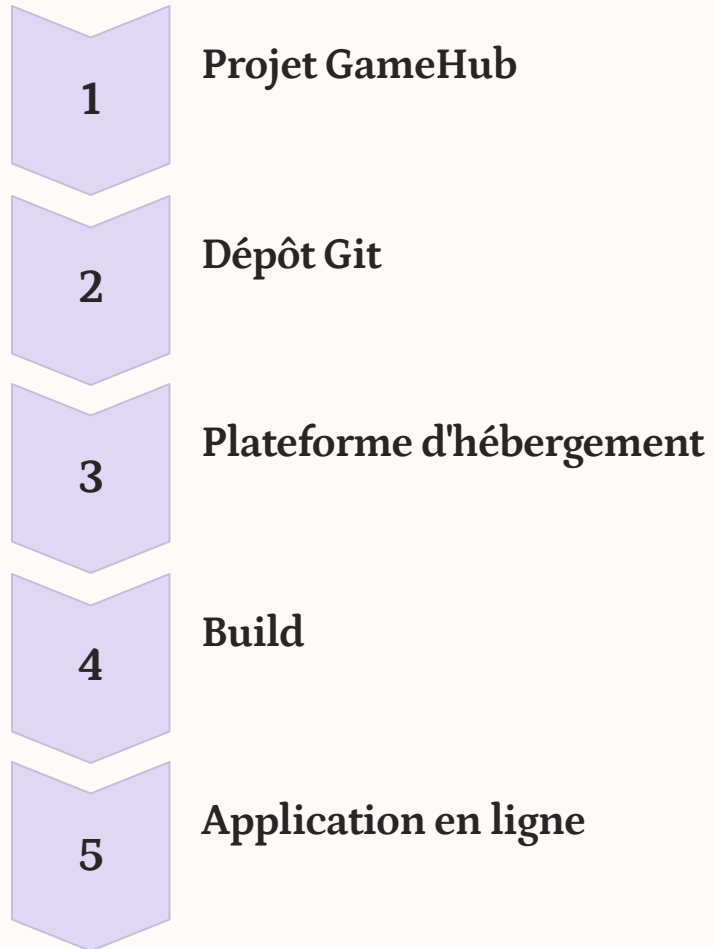
- variables d'environnement correctes
- URL de production correcte
- aucun secret dans le frontend

Interface

- routes fonctionnelles
- responsive
- loading/error/empty states

Déployer GameHub

Une fois le build validé :



Après publication, l'application devient accessible depuis son URL de production.

Tester comme un utilisateur

Ne pas uniquement vérifier : « Le site s'affiche. » Tester réellement :

- 1 Ouvrir GameHub
- 2 rechercher un jeu
- 3 ouvrir ses détails
- 4 ajouter aux favoris
- 5 consulter les favoris
- 6 modifier son profil
- 7 recharger une page
- 8 provoquer un cas d'erreur

Le test doit reproduire les usages réels.

Exercice — Préparer un déploiement

Avant de déployer une application React, établir une checklist comprenant :

1. vérification TypeScript
2. tests
3. build
4. variables d'environnement
5. preview
6. routes
7. API
8. vérifications navigateur
9. déploiement
10. vérifications post-déploiement

Objectif : Comprendre qu'un déploiement est un processus, pas simplement une commande.

Correction — ordre recommandé

Une démarche raisonnable :

01

Développer

02

Tester

03

Corriger

04

Construire

05

Prévisualiser

06

Configurer la production

07

Déployer

08

Vérifier

Si une étape échoue, on corrige le problème avant de poursuivre.

Les erreurs classiques

« Ça fonctionne avec npm run dev. »

La production n'a pas encore été testée.

« Le build fonctionne, donc tout est bon. »

Le build ne vérifie pas toute l'expérience utilisateur.

« Mon API fonctionne en local. »

L'URL et la configuration de production peuvent être différentes.

« Les routes fonctionnent quand je clique. »

Il faut également tester un accès direct et un rechargement.

À retenir — Partie 22

npm run dev sert au développement

npm run build produit une version de production

dist/ contient généralement le résultat du build Vite

npm run preview permet de tester localement le build

les variables d'environnement permettent de gérer la configuration

les secrets ne doivent jamais être exposés dans le frontend

React Router nécessite une configuration adaptée de l'hébergement

le déploiement doit être suivi de vérifications réelles

Une application n'est pas terminée lorsqu'elle compile : elle est terminée lorsque sa version de production fonctionne correctement pour ses utilisateurs.

Partie 24 — Audit Peer-to-Peer

01

Pourquoi auditer

03

Qualité TypeScript

05

Architecture

07

Performance

09

Tests

02

Méthode

04

Qualité React

06

UX

08

Gestion des erreurs

10

Grille d'audit

Pourquoi un audit ?

Un code peut fonctionner... sans être de qualité. L'audit permet de vérifier :

- lisibilité
- maintenabilité
- qualité technique
- UX
- robustesse

Un autre regard révèle souvent des problèmes invisibles pour l'auteur.

Une méthode en 5 étapes

01

Utiliser — Tester l'application comme un utilisateur.

03

Identifier — Repérer problèmes et incohérences.

05

Proposer — Formuler des améliorations concrètes.

02

Lire — Observer l'organisation du code.

04

Prioriser — Distinguer l'important du secondaire.

Observer avant de corriger

Tester notamment :

- navigation
- recherche
- formulaires
- favoris
- erreurs
- responsive

Puis seulement examiner le code.

Produit + code = audit complet

Vérifier le typage

Questions :

- Les données sont-elles typées ?
- Les props sont-elles typées ?
- Les fonctions sont-elles typées ?
- Les réponses API sont-elles décrites ?
- Les any sont-ils justifiés ?

```
const game: any = data;
```

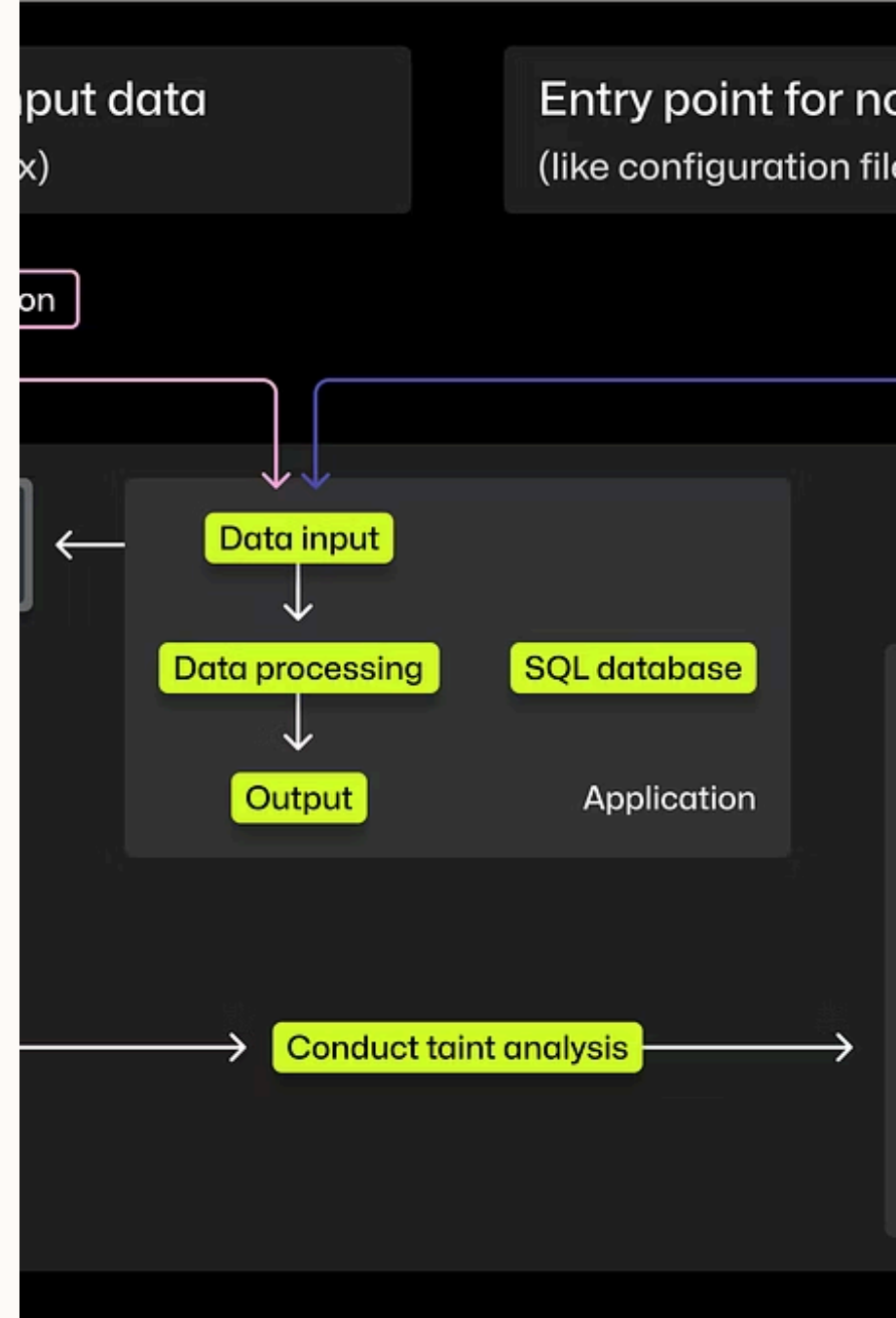
→ Signal d'alerte

Vérifier les composants

Observer :

- taille des composants
- responsabilités
- props
- state
- hooks
- réutilisation

Question clé : Un composant fait-il trop de choses ?



```

react, { useRef } from 'react';
useMediaQuery from 'beautiful-react-hooks/useMediaQuery'
useGeolocation from 'beautiful-react-hooks/useGeolocation'
useWindowSize from 'beautiful-react-hooks/useWindowSize'
useMouse from 'beautiful-react-hooks/useMouse'

```

beautiful-react-hooks to enhance and speed-up your development experier

```

meComponent = () => {
  ref = useRef()
  mouse = useMouse(ref) // returns mouse info when the provided element
  [geoState, { onChange }] = useGeolocation() // returns geolocation inf
  isTablet = useMediaQuery('(max-width: 48rem)') // media-queries in com
  onWindowSize = useWindowSize()

```

onomic approach to event callbacks

```

owResize(() => {
  kWindowSize()

```

```

(
n className="some-component" ref={ref}>
sTablet ? <TabletNavigation /> : <DesktopNavigation />
eoState.isRetrieving && <LoadingPlaceholder />
geoState.isSuppoerted && geoState.position) && (
<section className="some-component-position">
  <span>Lat: {geoState.position.coords.latitude}</span>
  <span>Lng: {geoState.position.coords.longitude}</span>
</section>

```

```

ection className="some-component-mouse">
<p>Mouse positin (x: {mouse.clientX}, y: {mouse.clientY}</p>
section>
in>

```

Vérifier les Hooks

Se demander :

- useEffect est-il nécessaire ?
- Les dépendances sont-elles correctes ?
- Le state est-il au bon endroit ?
- Les custom hooks sont-ils utiles ?
- useMemo / useCallback sont-ils justifiés ?

Pas de hook par réflexe.

Le projet est-il compréhensible ?

components/
pages/
hooks/
context/
services/
types/

Un développeur doit rapidement savoir :

- où trouver l'UI
- où trouver l'API
- où trouver les types
- où trouver la logique partagée

Responsabilités

Attention aux composants qui mélangent :

API

Recherche

State

Validation

Affichage

Navigation

Question : Peut-on modifier une fonctionnalité sans toucher à tout le composant ?

ersecurity



CATCH MISTAKES

Implement approval workflows and dual-controls to identify errors before they cause damage. Cross-validation of work reduces the risk of human mistakes.

THE PRINCIPLES OF SEPARATION OF DUTY HELPS ORGANIZATIONS PREVENT F MAINTAIN INTEGRITY OF CRITICAL SYSTEMS



Tester comme un utilisateur

Vérifier :

- navigation
- lisibilité
- feedback
- cohérence
- responsive
- accessibilité de base

Une fonctionnalité peut être techniquement correcte mais difficile à utiliser.

Les 4 états

Loading

Success

Error

Empty

Exemple : Aucun résultat trouvé. ≠ Erreur serveur. L'utilisateur doit comprendre ce qui se passe.

Error 404: Requested Resource Not Found.

The server cannot locate the requested URL. Please consult your network administrator or try again later



Error 404: This page seems to be missing.

Let's find your way back. Check out these popular destinations to continue your journey!

[Home](#)

[About Us](#)

[Featured Products](#)



Avant d'optimiser

Questions :

- Où est le ralentissement ?
- Combien de données ?
- Quels composants re-render ?
- Quels calculs sont coûteux ?

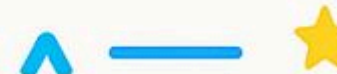
Mesurer avant d'optimiser.

a Into ads: at Matter

Q Keyword



★ Reviews



▲ Retention



! Uninstall Rate

Les optimisations

À examiner :

React.memo

useMemo

useCallback

lazy

Mais leur présence n'est pas automatiquement positive. Question :
Quel problème cette optimisation résout-elle ?

Tester les échecs

Provoquer volontairement :

- API indisponible
- erreur HTTP
- formulaire invalide
- route inconnue
- recherche sans résultat

Puis observer le comportement.

Une bonne erreur

Éviter :

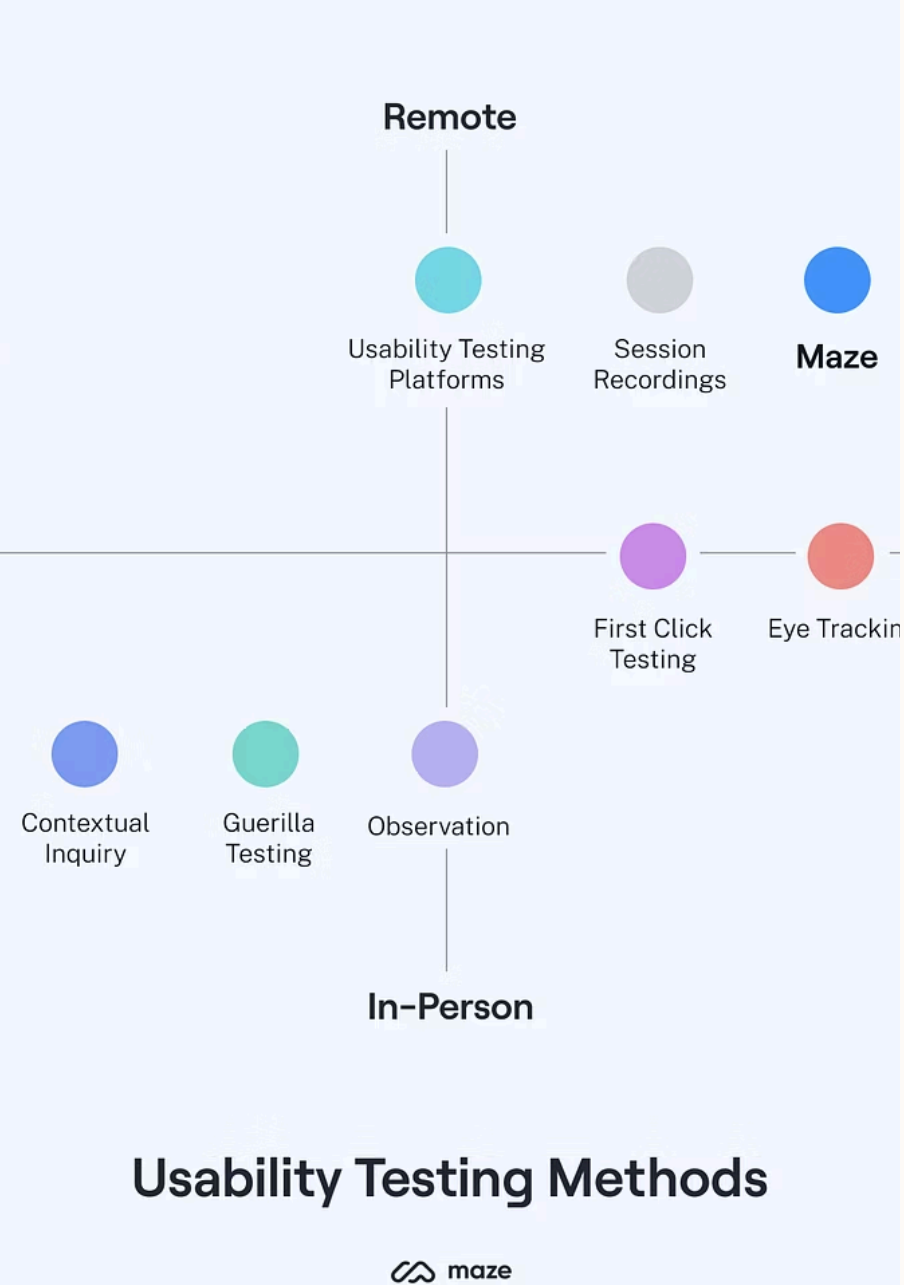
Failed **to** fetch

Préférer :

Impossible de charger les jeux.

[Réessayer]

Une erreur doit être compréhensible et actionnable.



Que tester ?

Priorité aux fonctionnalités importantes :

- recherche
- favoris
- formulaires
- validation
- navigation
- loading / error

Le but n'est pas de tester chaque ligne.
Tester les comportements importants.

Tester le comportement

```
await user.click(  
  screen.getByRole("button", {  
    name: /favori/i  
  })  
);
```

Puis vérifier ce que voit l'utilisateur.

Tester le résultat, pas l'implémentation interne.

Grille Peer-to-Peer

Domaine	OK	À améliorer
TypeScript		
React		
Architecture		
UX		
Responsive		
Performance		
Erreurs		
Tests		
Code		

Prioriser

Critique

l'application ne fonctionne pas.

Important

fonctionnalité majeure dégradée.

Amélioration

code ou UX perfectible.

Cosmétique

détail secondaire.

Identifier les 3 améliorations prioritaires.

Restitution

Chaque binôme présente :

1. 1 point positif
2. 1 problème
3. 1 proposition
4. 1 priorité

Exemple : « La structure des composants est claire. La logique de recherche est dupliquée. Elle pourrait être extraite dans un hook.
Priorité : importante. »

Ce que nous avons appris

JavaScript

Variables · objets · fonctions · tableaux · async

TypeScript

Types · interfaces · unions · generics

React

JSX · composants · props · state · hooks

Application

Router · API · formulaires · Context

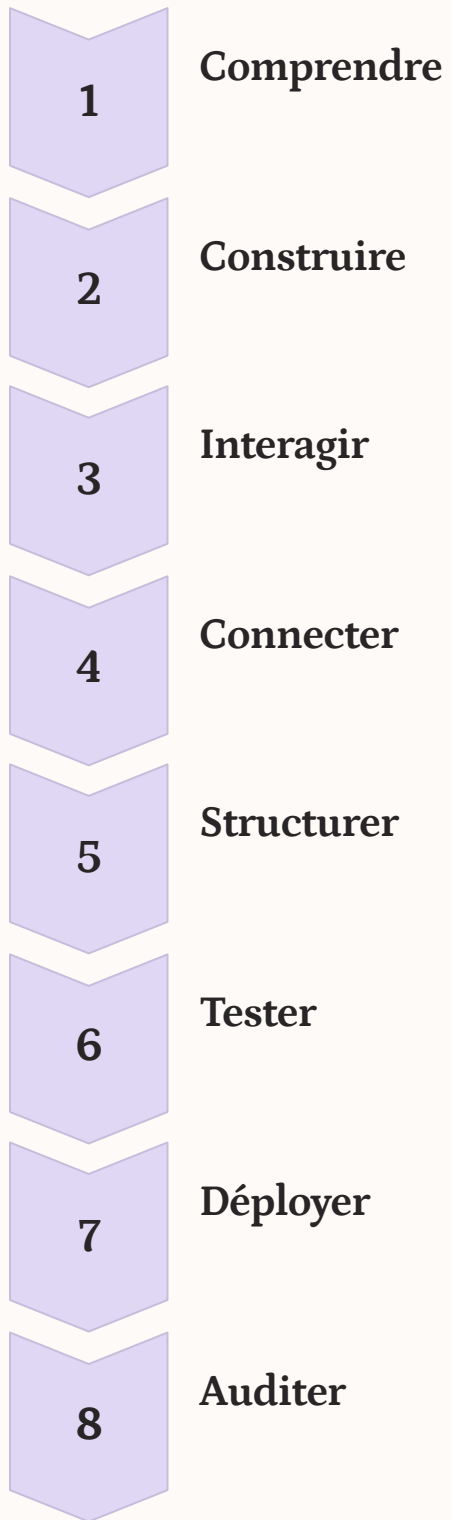
Qualité

Tests · erreurs · performance · architecture

Production

Tailwind · build · déploiement · audit

Notre progression



Chaque étape répond à un problème concret rencontré dans GameHub.

Les réflexes à conserver

Face à un problème :

Quel est le besoin ?

Où doit vivre la donnée ?

Quelle solution est la plus simple ?

Comment gérer les erreurs ?

Comment vérifier que ça fonctionne ?

Comment rendre le code maintenable ?

React n'est pas une liste de recettes

Ne pas apprendre : « Pour ce problème, j'utilise forcément useEffect. »

Mais apprendre à raisonner : Problème → solution → fonctionnement → limites

Même logique pour :

- useState
- useReducer
- Context
- custom hooks
- useMemo
- useCallback

GameHub : le résultat

GameHub réunit :

- navigation
- recherche
- filtres
- jeux
- favoris
- bibliothèque
- profil
- API
- formulaires
- gestion des erreurs
- tests
- responsive
- déploiement

Un projet concret pour mettre en pratique les concepts React + TypeScript.

Conclusion finale

Construire une application ne suffit pas. Un bon développeur doit aussi savoir :

comprendre le besoin

structurer le code

anticiper les erreurs

tester les comportements

mesurer les performances

améliorer l'expérience

déployer l'application

relire son propre travail

L'objectif final : devenir autonome pour construire et faire évoluer une application React + TypeScript.

GameHub — React.js & TypeScript

Comprendre → Construire → Tester → Améliorer → Déployer

Merci

Questions / démonstration finale / retour sur le projet